

Documentation Portfolio



Portfolio refers to any combination of financial assets held by a trader. In the world of vectorbt, "portfolio" is a multidimensional structure capable of simulating and tracking multiple independent but also dependent portfolio instances. The main function of a portfolio is to apply a trading logic on a set of inputs to simulate a realistic trading environment, also referred to as "simulation". The outputs of such a simulation are orders and other information that can be used by the user in assessing the portfolio's performance, also referred to as "reconstruction" or "post-analysis". Both phases are isolated in nature, which enables various interesting use cases for quantitative analysis and data science.

The main class concerned with simulating and analyzing portfolios (i.e., with the actual backtesting) is **Portfolio**, which is a regular Python class subclassing **Analyzable** and having a range of Numba-compiled functions at its disposal. It's built similarly to other analyzable classes in the way that it has diverse class methods for instantiation from a range of inputs (such as **Portfolio.from\_signals** taking signals), it's a state-full class capable of wrapping and indexing any Pandas-like objects contained inside it, and it can compute metrics and display (sub-)plots for quick introspection of the stored data.

## Simulation

So, what's a simulation? It's just a sophisticated loop! 🔄

A typical simulation in vectorbt takes some inputs (such as signals), gradually iterates over their rows (time steps in the real world) using a for-loop, and at each row, runs the trading logic by issuing and executing orders, and updating the current state of the trading environment such as the cash balance and position size. If we think about it, it's the exact same way we would approach algorithmic trading in reality: at each minute/hour/day (= row), decide what to do (= trading logic), and place an order if you decided to change your position in the market.

Now, let's talk about execution. The core of the vectorbt's backtesting engine is fully Numba-compiled for best performance. The functionality of the engine is distributed across many functions in an entire sub-package - **portfolio.nb**, ranging from core order execution commands to calculation of P&L in trade records. Remember that those functions aren't meant to be used directly (unless specifically desired) but are used by Python functions higher in the stack that know how to properly pre-process their input data and post-process the output data.

In the following parts, we'll discuss order execution and processing, and gradually implement a collection of simple pipelines to better illustrate various simulation concepts.

## Primitive commands

Remember that vectorbt is an exceptionally raw backtester: its primary commands are "buy" 🟢 and "sell" 🔴. This means that any strategy that can be translated into a set of those commands is also supported out of the box. This also means that more complex orders such as limit and SL orders must be implemented manually. In contrast to other backtesting frameworks where processing is monolithic and functionality is written in an **object-oriented manner**, Numba forces vectorbt to implement most of the functionality in a procedural manner.

### Info

Even though Numba supports OOP by compiling Python classes with `@jitclass`, they are treated as functions, must be statically typed, and have performance drawbacks that don't allow us to jump on the wagon just yet.

Functions related to order execution are primarily located in **portfolio.nb.core**. The functions implementing our primary two commands are **buy\_nb** and **sell\_nb**. Among the requested size and price of an order, the primary input of each of these functions is the current account state of type **AccountState**, which contains the current cash balance, position size, and other information about the current environment. Whenever we buy or sell something, the function creates and returns a new state of the same type. Furthermore, it returns an order result of type **OrderResult**, which contains the filled size, price adjusted with slippage, transaction fee, order side, status information on whether the order succeeded or failed, and valuable information about why it failed.

## Buying

The buy operation consists of two distinct operations: "long-buy" implemented by **long\_buy\_nb** and "short-buy" implemented by **short\_buy\_nb**. The first one opens or increases a long position, while the second one decreases a short position. By chaining these two operations, we can reverse a short position, which is done automatically by **buy\_nb**: it checks the position we're in (if any), and calls the responsible function.

Let's say we have \$100 available and want to buy 1 share at the price of \$15:

```
>>> from vectorbtpro import *

>>> account_state = vbt.pf_enums.AccountState(
...     cash=100.0,
...     position=0.0,
...     debt=0.0, ①
...     locked_cash=0.0, ②
...     free_cash=100.0 ③
... )
>>> order_result, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     size=1.0,
...     price=15.0
... )
>>> vbt.pprint(order_result)
OrderResult(
```

```

        size=1.0,
        price=15.0,
        fees=0.0,
        side=0,
        status=0,
        status_info=-1
    )
>>> vbt.pprint(new_account_state)
AccountState(
  cash=85.0,
  position=1.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=85.0
)

```

- ❶ Debt is non-zero only when leveraging or shorting
- ❷ Locked cash is non-zero only when leveraging or shorting
- ❸ Free cash deviates from the regular cash balance only when leveraging or shorting

The returned state indicates that we spent \$15 and increased our position by 1 share. The order result contains details about the executed order: we bought 1 share for \$15, with no transaction fees. Since order side and status are of named tuple type `OrderSide` and `OrderStatus` respectively, we can query the meaning behind those numbers as follows:

```

>>> vbt.pf_enums.OrderSide._fields[order_result.side]
'Buy'

>>> vbt.pf_enums.OrderStatus._fields[order_result.status]
'Filled'

```

#### Info

If any value is `-1` and cannot be found in the named tuple, the information is unavailable.

Now, based on the new state, let's execute a transaction that uses up the remaining cash:

```

>>> order_result, new_account_state2 = vbt.pf_nb.buy_nb(
...     account_state=new_account_state, ❶
...     size=np.inf, ❷
...     price=15.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=5.666666666666667,
  price=15.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state2)
AccountState(
  cash=0.0,
  position=6.666666666666667,
  debt=0.0,
  locked_cash=0.0,
  free_cash=0.0
)

```

- ❶ Use the previous account state as input
- ❷ Infinity means using up the entire balance

Since vectorbt was originally tailored to cryptocurrency and fractional shares, the default behavior is buying as much as possible (here `5.67`), even if the amount is below of that requested. But what happens if we wanted to have the entire share instead? Let's specify the size granularity of 1, indicating that only integer amounts should be allowed:

```

>>> order_result, new_account_state = vbt.pf_nb.buy_nb(
...     account_state,
...     size=np.inf,
...     price=15.0,
...     size_granularity=1
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=6.0,
  price=15.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=10.0,
  position=6.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=10.0
)

```

```
)
```

This has bought exactly 6 shares. Given the new account state, let's run the same transaction again:

```
>>> order_result, new_account_state2 = vbt.pf_nb.buy_nb(
...     new_account_state, ❶
...     size=np.inf,
...     price=15.0,
...     size_granularity=1
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=np.nan,
  price=np.nan,
  fees=np.nan,
  side=-1,
  status=1,
  status_info=5
)
>>> vbt.pprint(new_account_state2)
AccountState(
  cash=10.0,
  position=6.0,
  debt=0.0,
  free_cash=10.0
)
```

- ❶ Use the account state from the previous operation

The account state remains unchanged, while so many NaNs in the order result hint at a failed order. Let's query the meaning behind the status and status information numbers using `OrderStatus` and `OrderStatusInfo` named tuple respectively:

```
>>> vbt.pf_enums.OrderStatus._fields[order_result.status]
'Ignored'

>>> vbt.pf_enums.OrderStatusInfo._fields[order_result.status_info]
'SizeZero'

>>> vbt.pf_enums.status_info_desc[order_result.status_info] ❶
'Size is zero'
```

- ❶ There is a list with more elaborative descriptions of different statuses

Here, the status "Size is zero" means that by considering our cash balance and after applying the size granularity, the (potentially) filled order size is zero, thus the order should be ignored. Ignored orders have no effect on the trading environment and are simply, well, *ignored*. But sometimes, when the user has specific requirements and vectorbt cannot execute them, the status will become "Rejected", indicating that the request could not be fulfilled and an error can be thrown if wanted.

For example, let's try to buy more than possible:

```
>>> order_result, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     size=1000.0,
...     price=15.0,
...     allow_partial=False
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=np.nan,
  price=np.nan,
  fees=np.nan,
  side=-1,
  status=2,
  status_info=12
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=100.0,
  position=0.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=100.0
)

>>> vbt.pf_enums.OrderStatus._fields[order_result.status]
'Rejected'

>>> vbt.pf_enums.status_info_desc[order_result.status_info]
'Final size is less than requested'
```

There are many other parameters to control the execution. Let's use 50% of cash, and apply 1% in fees and slippage:

```
>>> order_result, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     size=np.inf,
...     price=15.0,
...     fees=0.01, ❶
...     slippage=0.01, ❷
...     percent=0.5 ❸
... )
>>> vbt.pprint(order_result)
```

```

OrderResult(
  size=3.2676534980230696,
  price=15.15,
  fees=0.4950495049504937,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=50.0,
  position=3.2676534980230696,
  debt=0.0,
  locked_cash=0.0,
  free_cash=50.0
)

```

- ❶ 0.01 = 1%. Paid always in cash. To specify fixed fees, use `fixed_fees` instead.
- ❷ 0.01 = 1%. Applied on the price. By artificially increasing the price, you always put yourself at a disadvantage, but this might be wanted to make backtesting more realistic.
- ❸ 0.01 = 1%. Applied on the resources used to open or increase a position.

The final fees and the price adjusted with the slippage are reflected in the order result.

Whenever we place an order, we can specify any price. Thus, it may sometimes happen that the provided price is (by mistake of the user) higher than the highest price of that bar or lower than the lowest price of that bar. Also, if the user wanted the price to be closing, and he specified a slippage, this would also be quite unrealistic. To avoid such mistakes, the function performs an OHLC check. For this, we need to specify the `price_area` of type `PriceArea` with the price boundaries, and what should be done if a boundary violation happens via `price_area_vio_mode` of type `PriceAreaVioMode`:

```

>>> price_area = vbt.pf_enums.PriceArea(
...     open=10,
...     high=14,
...     low=8,
...     close=12
... )
>>> order_result, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     size=np.inf,
...     price=np.inf,
...     price_area=price_area,
...     price_area_vio_mode=vbt.pf_enums.PriceAreaVioMode.Error
... )
ValueError: Adjusted order price is above the highest price

```

## Selling

The sell operation consists of two distinct operations: "long-sell" implemented by `long_sell_nb` and "short-sell" implemented by `short_sell_nb`. The first one decreases a long position, while the second one opens or increases a short position. By chaining these two operations, we can reverse a long position, which is done automatically by `sell_nb`: it checks the position we're in (if any), and calls the responsible function.

The function for selling takes the same arguments but uses them in the opposite direction. Let's remove 2 shares from a position of 10 shares:

```

>>> account_state = vbt.pf_enums.AccountState(
...     cash=0.0,
...     position=10.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=0.0
... )
>>> order_result, new_account_state = vbt.pf_nb.sell_nb(
...     account_state=account_state,
...     size=2.0,
...     price=15.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=2.0,
  price=15.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=30.0,
  position=8.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=30.0
)

```

The size in the order result remains positive but the side has changed from 0 to 1:

```

>>> vbt.pf_enums.OrderSide._fields[order_result.side]
'Sell'

```

## Shorting

Shorting is a regular sell operation with `sell_nb`, but with one exception: it now involves the debt as well as the locked cash balance. Whenever we short, we are borrowing shares and selling them to buyers willing to pay the market price. This operation increases the cash balance and turns the position size negative. It also registers the received cash amount as a debt, and subtracts it from the free cash balance. Whenever we buy some shares back, the debt decreases proportionally to the value of the shares bought back, while the free cash might increase depending upon whether the price was higher or lower than the average short-selling price. Whenever we cover the short position entirely, the debt becomes zero and the free cash balance returns to the same level as the regular cash balance.

### Note

You shouldn't treat debt as an absolute amount of cash you owe since you owe shares, not cash; it's used for calculating the average leverage and entry price of the short position, which is then used to calculate the change in the free cash balance with each trade.

To borrow any shares, we need a positive free cash balance to be used as a collateral. The exact amount of free cash needed for a shorting operation depends on margin; by default, we need the same amount of funds available in our margin account as the value of to-be-borrowed shares. For example, if we short a stock and the new position had a value of \$100, we would be required to have the \$100 that came from the short sale plus an additional \$100 in cash, for a total of \$200, which depending on the definition is either a 100% (before sale) or 200% (after sale) initial margin requirement. Maintenance margin and liquidation checks are the responsibility of the user (for now).

```
>>> account_state = vbt.pf_enums.AccountState(
...     cash=100.0,
...     position=0.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=100.0
... )
>>> order_result, new_account_state = vbt.pf_nb.sell_nb(
...     account_state=account_state,
...     size=np.inf, ❶
...     price=15.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=6.666666666666667,
  price=15.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=200.0,
  position=-6.666666666666667,
  debt=100.0,
  locked_cash=100.0,
  free_cash=0.0
)
```

### ❶ Short-sell as much as possible

### Info

Infinity is a special value in vectorbt and usually means "go as far as you can".

Here's what happened. First, we've converted all the available free cash (\$100) into the locked one such that it becomes the collateral of our shorting operation. Because the default leverage is 1, we've borrowed the same value in shares (\$100), which has been added to the regular cash balance as well registered as a debt. This value corresponds to (minus = borrowed) 6.67 shares. But here's the problem: since we've doubled the regular cash balance, it may be used by other assets. To avoid that, all operations are done strictly on the free cash balance! But it's zero right now, so how do we buy back the borrowed shares? Remember that the debt and locked cash represent the total amount of cash that we used in the first place; by adding both to the free cash, we get our cash limit for the current buy operation, which nicely matches the regular cash when dealing with only one asset.

To change the margin, we can use the argument `leverage`. For example, setting it to 2 will allow us to borrow twice as many shares as can be covered by the current free cash:

```
>>> account_state = vbt.pf_enums.AccountState(
...     cash=100.0,
...     position=0.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=100.0
... )
>>> order_result, new_account_state = vbt.pf_nb.sell_nb(
...     account_state=account_state,
...     size=np.inf,
...     price=15.0,
...     leverage=2
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=13.333333333333334,
  price=15.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
```

```
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=300.0,
  position=-13.333333333333334,
  debt=200.0,
  locked_cash=100.0,
  free_cash=0.0
)
```

The debt-to-locked-cash ratio is now 2, which corresponds to the leverage that we specified.

#### Info

We can specify a different leverage for each short-sell order, even in the same position.

Let's try to run the same operation again, but now on the new account state:

```
>>> order_result, new_account_state2 = vbt.pf_nb.sell_nb(
...     account_state=new_account_state,
...     size=np.inf,
...     price=15.0,
...     leverage=2
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=np.nan,
  price=np.nan,
  fees=np.nan,
  side=-1,
  status=2,
  status_info=6
)
>>> vbt.pprint(new_account_state2)
AccountState(
  cash=300.0,
  position=-13.333333333333334,
  debt=200.0,
  locked_cash=100.0,
  free_cash=0.0
)

>>> vbt.pf_enums.OrderStatus._fields[order_result.status]
'Rejected'

>>> vbt.pf_enums.status_info_desc[order_result.status_info]
'Not enough cash'
```

We see that vectorbt prevents the free cash balance to become negative.

To order any quantity possible, we can use the unlimited leverage:

```
>>> order_result, new_account_state = vbt.pf_nb.sell_nb(
...     account_state=account_state,
...     size=1000,
...     price=15.0,
...     leverage=np.inf
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=1000.0,
  price=15.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=15100.0,
  position=-1000.0,
  debt=15000.0,
  locked_cash=100.0,
  free_cash=0.0
)
```

What's the effective leverage of this operation?

```
>>> new_account_state.debt / new_account_state.locked_cash
150.0
```

If we had to calculate the current portfolio value, it would still default to the initial cash since no transaction costs were involved and no additional trades were made:

```
>>> new_account_state.cash + new_account_state.position * order_result.price
100.0
```

As we see, the positive cash balance and the negative position size keep the total value in balance. Now, let's illustrate buying back some shares using `buy_nb`. First, we'll borrow 10 shares with 2x leverage and sell them for the price of \$10 per share:

```
>>> order_result, new_account_state = vbt.pf_nb.sell_nb(
...     account_state=account_state,
...     size=10.0,
...     price=15.0,
...     leverage=2
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=10.0,
  price=15.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=250.0,
  position=-10.0,
  debt=150.0,
  locked_cash=75.0,
  free_cash=25.0
)
```

Let's buy back 5 shares for the price of \$30 per share (my condolences):

```
>>> order_result, new_account_state2 = vbt.pf_nb.buy_nb(
...     account_state=new_account_state,
...     size=5.0,
...     price=30.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=5.0,
  price=30.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state2)
AccountState(
  cash=100.0,
  position=-5.0,
  debt=75.0,
  locked_cash=37.5,
  free_cash=-12.5
)
```

We executed the order for \$150, which have been deducted from the regular cash balance. The position has been reduced by half, to 5 borrowed shares. Along with the position, the debt and locked cash have also been reduced by half. Given the absolute amount of released debt (\$75), we can then compute the P&L, which is the total spent cash subtracted from the total released debt, or -\$75. But this operation has also released some locked cash - \$37.5, such that the amount of cash added back to our free cash balance is -\$37.5, which makes it negative. A negative free cash means that we won't be able to buy any other assets apart from reducing any short positions and thus potentially releasing additional funds (i.e., profits as well as losses are shared among all assets within the same group with cash sharing). Even though we've got a negative free cash, we can still buy back more shares because the sum of `debt`, `locked_cash`, and `free_cash` is greater than zero.

Not only the debt and locked cash can be used to compute the effective leverage, but we can also derive the average entry price of the entire position:

```
>>> new_account_state2.debt / abs(new_account_state2.position)
15.0
```

Let's say instead of jumping, the price has dipped to \$10 per share (my congratulations!):

```
>>> order_result, new_account_state2 = vbt.pf_nb.buy_nb(
...     account_state=new_account_state,
...     size=5.0,
...     price=10.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=5.0,
  price=10.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state2)
AccountState(
  cash=200.0,
  position=-5.0,
  debt=75.0,
  locked_cash=37.5,
  free_cash=87.5
)
```

We see that the debt and locked cash have decreased to the same level as previously (because we've bought back the same relative amount of shares), but the free cash balance is now \$200, netting \$25 in profit! Again, the calculation is simple: we just take the total amount of spent cash ( $5 * 10 = \$50$ ) and subtract it

from the total released debt ( $0.5 * \$150 = \$75$ ) to get the P&L. By adding the P&L, the released locked cash ( $0.5 * \$75 = \$37.5$ ), and the current free cash (\$25) together, we get the new free cash of \$87.5 immediately available for all other assets to use.

Let's compute the equity to validate the profit:

```
>>> st0 = account_state
>>> st1 = new_account_state2
>>> avg_entry_price = st1.debt / abs(st1.position) ❶
>>> new_equity = st1.cash + st1.position * avg_entry_price
>>> new_equity - st0.cash
25.0
```

❶ The remaining shares are still valued at \$15

Let's close out the open short position using the same price:

```
>>> order_result, new_account_state3 = vbt.pf_nb.buy_nb(
...     account_state=new_account_state2,
...     size=5.0,
...     price=10.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=5.0,
  price=10.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state3)
AccountState(
  cash=150.0,
  position=0.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=150.0
)
```

The free cash balance equals to the regular cash balance, and we are debt-free! Additionally, the last two operations have brought us \$50 in profit, or  $(15 - 10) * 10 = \$50$ .

Finally, let's try to close out the position using an astronomically high price!

```
>>> order_result, new_account_state3 = vbt.pf_nb.buy_nb(
...     account_state=new_account_state2,
...     size=5.0,
...     price=100.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=2.0,
  price=100.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state3)
AccountState(
  cash=0.0,
  position=-3.0,
  debt=45.0,
  locked_cash=22.5,
  free_cash=-67.5
)
```

We could buy back only 2 shares out of the remaining 5. If we try the same operation again, we would get the "Not enough cash" message because `debt + locked_cash + free_cash` is below or equal to zero. We also witness the regular cash balance going to zero, which means that we've exhausted all of our capital; but we shouldn't rely on it to make trading decisions! If any other asset buys shares with leverage, regular cash may go negative, which doesn't necessarily mean that we have no cash left - only free cash (with debt and locked cash when covering shorts) provide us with the right information.

## Leverage

Even though vectorbt allows setting an arbitrary (even an infinite) cash and ordering as many shares as required by the user, this constellation is associated with some drawbacks: an infinite cash leads to an infinite portfolio value, which makes certain operations on that value impossible, such as the conversion from a target percentage into a target amount of shares; but also, the more cash we have, the smaller are potential contributions of the positions to the portfolio value, thus lowering the magnitude of the portfolio returns. What we need though is to multiply those contributions without inflating the cash balance, which can be effectively done using leverage.

Leverage involves borrowing additional funds to buy shares. In main contrast to shorting, leverage is applied to long positions and borrows cash instead of shares. The underlying mechanism is quite similar though. First, we multiply the available free cash by `leverage`. Then, we derive the order value and the fraction of it to be borrowed. Finally, we move the borrowed cash to `debt` while declaring a part of the free cash as the collateral of the operation and moving it to `locked_cash`. But since the locked cash must be spent to buy a part of the shares, it changes the way the effective leverage can be calculated from `debt / locked_cash` to `debt / locked_cash + 1`.



Let's say we have \$100 in our margin account and want to buy \$200 worth of shares. As we learned previously, we can specify an infinite leverage and vectorbt will derive the effective leverage internally for us:

```
>>> account_state = vbt.pf_enums.AccountState(
...     cash=100.0,
...     position=0.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=100.0
... )
>>> order_result, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     size=20,
...     price=10.0,
...     leverage=np.inf
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=20,
  price=10.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=-100.0,
  position=20.0,
  debt=100.0,
  locked_cash=100.0,
  free_cash=0.0
)
```

We can see that \$100 were deducted from our free cash balance and additional \$100 were borrowed, thus bringing the effective leverage to 2:

```
>>> new_account_state.debt / new_account_state.locked_cash + 1
2.0
```

Buying 10 shares instead would use no leverage since the transaction can be entirely covered by the free cash, even if the leverage is infinity:

```
>>> order_result, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     size=10,
...     price=10.0,
...     leverage=np.inf
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=10,
  price=10.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=0.0,
  position=10.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=0.0
)
```

Is there a way to use only a subset of our own free cash while borrowing the rest? Yes! The command **buy\_nb** takes the argument `leverage_mode` of the type **LeverageMode**, which supports two modes: "lazy" and "eager" leveraging. The first mode is the default one and enables leverage only if the quantity to be bought cannot be fulfilled using own resources. The second mode enables leverage for any quantity and requires the leverage to be set explicitly, that is, an infinite leverage would raise an error.

#### Note

Shorting supports "lazy" leveraging only.

Let's buy 10 shares with a 3x leverage:

```
>>> order_result, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     size=10,
...     price=10.0,
...     leverage=3,
...     leverage_mode=vbt.pf_enums.LeverageMode.Eager
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=10,
  price=10.0,
  fees=0.0,
  side=0,
  status=0,
```

```

        status_info=-1
    )
>>> vbt.pprint(new_account_state)
AccountState(
  cash=0.0,
  position=10.0,
  debt=66.66666666666667,
  locked_cash=33.333333333333336,
  free_cash=66.66666666666666
)

```

We've used only \$33.3 from our free cash balance as the collateral to borrow additional \$66.6, making a total of \$100 that were spent to buy the desired quantity.

Now, how do we repay the debt? When selling, the debt and locked cash balances decrease proportionally to the number of shares sold, which is exactly the same procedure as during (partially-) closing a short position. The main difference is in the calculation of the P&L: we take the total of the released debt and locked cash and subtract them from the cash retrieved from the operation.

First, we'll use a 2x leverage to buy 10 shares for the price of \$20 per share:

```

>>> order_result, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     size=10,
...     price=20.0,
...     leverage=2
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=10,
  price=20.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=-100.0,
  position=10.0,
  debt=100.0,
  locked_cash=100.0,
  free_cash=0.0
)

```

Let's sell 5 shares for the price of \$5 per share (my condolences):

```

>>> order_result, new_account_state2 = vbt.pf_nb.sell_nb(
...     account_state=new_account_state,
...     size=5.0,
...     price=5.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=5.0,
  price=5.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state2)
AccountState(
  cash=-75.0,
  position=5.0,
  debt=50.0,
  locked_cash=50.0,
  free_cash=-25.0
)

```

We've retrieved \$25 from this operation, which have been added to the regular cash balance. The debt and locked cash have been cut in half because the half of the leveraged position has been closed out. The P&L of this operation is the gained cash (\$25) minus the released debt (\$50) and locked cash (\$50), which makes it a loss of \$75. By adding this number to the released locked cash, we arrive at the new free cash of -\$25 - a change that is propagated to all other assets using the same cash balance and thus preventing them to open or increase their positions.

Another way of calculating the P&L using the equity:

```

>>> st0 = account_state
>>> st1 = new_account_state2
>>> avg_entry_price = (st1.debt + st1.locked_cash) / abs(st1.position) ❶
>>> new_equity = st1.cash + st1.position * avg_entry_price
>>> new_equity - st0.cash
-75.0

```

❶ The remaining shares are still valued at \$20

Now, let's say instead of dipping, the price has jumped to \$40 per share (my congratulations!):

```

>>> order_result, new_account_state2 = vbt.pf_nb.sell_nb(
...     account_state=new_account_state,
...     size=5.0,
...     price=40.0
... )

```

```
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=5.0,
  price=40.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state2)
AccountState(
  cash=100.0,
  position=5.0,
  debt=50.0,
  locked_cash=50.0,
  free_cash=150.0
)
```

We've retrieved \$200 from this operation, which have been added to the regular cash balance. The debt and locked cash have been cut in half because the half of the leveraged position has been closed out. The P&L of this operation is the gained cash (\$200) minus the released debt (\$50) and locked cash (\$50), which makes it a profit of \$100. By adding this number to the released locked cash, we arrive at the new free cash of \$150, which can be now used by all other assets sharing the same balance.

Let's close out the remaining position using the same price:

```
>>> order_result, new_account_state2 = vbt.pf_nb.sell_nb(
...     account_state=new_account_state,
...     size=5.0,
...     price=40.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=5.0,
  price=40.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state2)
AccountState(
  cash=300.0,
  position=0.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=300.0
)
```

We've made a profit of \$200, which is just the same as we had used our own cash:

```
>>> account_state = vbt.pf_enums.AccountState(
...     cash=200.0,
...     position=0.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=200.0
... )
>>> _, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     size=10,
...     price=20.0
... )
>>> _, new_account_state2 = vbt.pf_nb.sell_nb(
...     account_state=new_account_state,
...     size=10.0,
...     price=40.0
... )
>>> new_account_state2.free_cash - account_state.free_cash
200.0
```

## Symmetry

Long and short positions behave symmetrically. For example, let's open two opposite positions using an infinite size and 10x leverage and close them out with a price difference of \$5 per share that favors the current position:

```
>>> account_state = vbt.pf_enums.AccountState(
...     cash=100.0,
...     position=0.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=100.0
... )

>>> _, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=account_state,
...     direction=vbt.pf_enums.Direction.LongOnly,
...     size=np.inf,
...     price=10.0,
...     leverage=10
... )
```

```

>>> _, new_account_state = vbt.pf_nb.sell_nb(
...     account_state=new_account_state,
...     direction=vbt.pf_enums.Direction.LongOnly,
...     size=np.inf,
...     price=15.0
... )
>>> vbt.pprint(new_account_state)
AccountState(
  cash=600.0,
  position=0.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=600.0
)

>>> _, new_account_state = vbt.pf_nb.sell_nb(
...     account_state=account_state,
...     direction=vbt.pf_enums.Direction.ShortOnly,
...     size=np.inf,
...     price=10.0,
...     leverage=10
... )
>>> _, new_account_state = vbt.pf_nb.buy_nb(
...     account_state=new_account_state,
...     direction=vbt.pf_enums.Direction.ShortOnly,
...     size=np.inf,
...     price=5.0
... )
>>> vbt.pprint(new_account_state)
AccountState(
  cash=600.0,
  position=0.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=600.0
)

```

## Reversing

Positions in vectorbt can be reversed with a single order. To reverse a position, the direction argument should stay at its default - `Direction.Both`. Let's start with a position of 10 shares, reverse it to the maximum extent in the short direction, and then reverse it to the maximum extent again but in the opposite (long) direction:

```

>>> account_state = vbt.pf_enums.AccountState(
...     cash=0.0,
...     position=10.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=0.0
... )
>>> order_result, new_account_state = vbt.pf_nb.sell_nb(
...     account_state=account_state,
...     size=np.inf,
...     price=15.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=20.0,
  price=15.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=300.0,
  position=-10.0,
  debt=150.0,
  locked_cash=150.0,
  free_cash=0.0
)

>>> order_result, new_account_state2 = vbt.pf_nb.buy_nb(
...     account_state=new_account_state,
...     size=np.inf,
...     price=15.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=20.0,
  price=15.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state2)
AccountState(
  cash=0.0,
  position=10.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=0.0
)

```

)

Both operations are symmetric in nature and cancel each other out by a repetitive call, thus ultimately we've arrived at our initial state of the account.

## Closing

To close out a position and to avoid its reversal, we can either specify the exact size, or the size of infinity and the current direction via the `direction` argument of the type **Direction**. For example, if we're in a long position and specified the long-only direction, the position won't be reversed:

```
>>> account_state = vbt.pf_enums.AccountState(
...     cash=0.0,
...     position=10.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=0.0
... )
>>> order_result, new_account_state = vbt.pf_nb.sell_nb(
...     account_state=account_state,
...     size=np.inf,
...     price=15.0,
...     direction=vbt.pf_enums.Direction.LongOnly
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=10.0,
  price=15.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=150.0,
  position=0.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=150.0
)
```

### Note

Using the **buy\_nb** and **sell\_nb** commands guarantees to execute the order in the long and short direction respectively.

We can also use the commands that are guaranteed to execute within the current position and not open an opposite one: **long\_sell\_nb** for long positions and **short\_buy\_nb** for short positions. They don't require the argument `direction` at all, just the size of infinity:

```
>>> account_state = vbt.pf_enums.AccountState(
...     cash=0.0,
...     position=10.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=0.0
... )
>>> order_result, new_account_state = vbt.pf_nb.long_sell_nb(
...     account_state=account_state,
...     size=np.inf,
...     price=15.0
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=10.0,
  price=15.0,
  fees=0.0,
  side=1,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_account_state)
AccountState(
  cash=150.0,
  position=0.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=150.0
)
```

## Pipeline/1

Even by using just these two essential commands, we can already build our own backtesting pipeline of arbitrary complexity and flexibility. As said before, a simulation is just a loop that iterates over timestamps. Let's create a simplified pipeline that puts \$1 into Bitcoin each time it discovers a **Golden Cross** entry signal, and sells \$1 otherwise. What we want is one number: the final value of the portfolio.

```
>>> @njit
... def pipeline_1_nb(close, entries, exits, init_cash=100):
...     account_state = vbt.pf_enums.AccountState( 1
...         cash=float(init_cash),
...         position=0.0,
```

```

...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=float(init_cash)
... )
... for i in range(close.shape[0]):
...     if entries[i]:
...         _, account_state = vbt.pf_nb.buy_nb( ❷
...             account_state=account_state,
...             size=1 / close[i],
...             price=close[i]
...         )
...         if exits[i]:
...             _, account_state = vbt.pf_nb.sell_nb(
...                 account_state=account_state,
...                 size=1 / close[i],
...                 price=close[i]
...             )
...     return account_state.cash + account_state.position * close[-1] ❸

>>> data = vbt.YFData.pull("BTC-USD", end="2022-01-01")
>>> sma_50 = vbt.talib("SMA").run(data.get("Close"), 50)
>>> sma_200 = vbt.talib("SMA").run(data.get("Close"), 200)
>>> entries = sma_50.real_crossed_above(sma_200)
>>> exits = sma_50.real_crossed_below(sma_200)

>>> pipeline_1_nb(
...     data.get("Close").values,
...     entries.values,
...     exits.values
... )
210.71073253390762

```

- ❶ Initial account state
- ❷ Execute the order and return a new account state
- ❸ Calculate the final portfolio value

#### Hint

Adding a suffix `_nb` to indicate a Numba-compiled function is not necessary but still a good convention in vectorbt.

We can validate the pipeline using one of the preset simulation methods:

```

>>> vbt.Portfolio.from_orders(
...     data.get("Close"),
...     size=entries.astype(int) - exits.astype(int),
...     size_type="value"
... ).final_value
210.71073253390762

```

## Order execution

Using the primitive commands is fun as long as we exactly know the direction of the order and are sure that the provided arguments make sense. But very often, we have to deal with more complex requirements such as target percentages, which change the order direction depending on the current value. In addition, the commands do not validate their arguments; for example, there won't be any error thrown in a case when the user accidentally passes a negative order price. But also, we need a better representation of an order - it's a bad practice of passing all the parameters such as slippage as keyword arguments.

All the checks and other pre-processing procedures are happening in the function `execute_order_nb`. The first input to this function is an order execution state of type `ExecState`, which contains the same information as an account state we saw above, but with additional information on the current valuation. The second input is a named tuple of type `Order` representing an order. The third argument is the price area, which we are also already familiar with.

### Order

An order in vectorbt is represented by a **named tuple**. Named tuples are alternatives to data classes in both the Python and Numba world; they are very efficient and lightweight data structures that can be easily constructed and processed. Let's create an instance of an order:

```
>>> order = vbt.pf_enums.Order()
>>> vbt.pprint(order)
Order(
  size=np.inf,
  price=np.inf,
  size_type=0,
  direction=2,
  fees=0.0,
  fixed_fees=0.0,
  slippage=0.0,
  min_size=np.nan,
  max_size=np.nan,
  size_granularity=np.nan,
  leverage=1.0,
  leverage_mode=0,
  reject_prob=0.0,
  price_area_vio_mode=0,
  allow_partial=True,
  raise_reject=False,
  log=False
)
```

The tuple allows for attribute access through the dot notation:

```
>>> order.direction
2
```

Other than this, it behaves just like any other tuple in Python:

```
>>> order[3]
2

>>> tuple(order) ❶
(inf,
 inf,
 0,
 2,
 0.0,
 0.0,
 0.0,
 0.0,
 nan,
 nan,
 nan,
 1.0,
 0,
 0.0,
 0,
 True,
 False,
 False)
```

#### ❶ Convert to a regular tuple

One issue that we still have to address when working with Numba are default arguments: although we can construct a new tuple solely with default arguments in Numba as we did above, if we want to override some values, they must be located strictly on the left in that tuple's definition. Otherwise, we must explicitly provide all the default arguments located before them:

```
>>> @njit
... def create_order_nb():
...     return vbt.pf_enums.Order() ❶

>>> create_order_nb()
Order(size=inf, price=inf, ...)

>>> @njit
... def create_order_nb(size, price):
...     return vbt.pf_enums.Order(size=size, price=price) ❷

>>> create_order_nb(1, 15)
Order(size=1, price=15, ...)

>>> @njit
... def create_order_nb(size, price, direction):
...     return vbt.pf_enums.Order(size=size, price=price, direction=direction) ❸

>>> create_order_nb(1, 15, 2)
Failed in nopython mode pipeline (step: nopython frontend)
```

- ❶ Using the default values only
- ❷ Overriding the default values of arguments on the left side
- ❸ Overriding the default values of arguments on different positions

Another issue are data types. In the example above where we provided integer size and price, Numba had no issues processing them. But as soon as we create such as order in a loop and one of the arguments is a float instead of an integer provided previously, Numba will throw an error because it cannot unify data types anymore. Thus, we should cast all arguments to their target data types before constructing an order.

Both issues are solved by using the function `order_nb`:

```
>>> @njit
... def create_order_nb(size, price, direction):
...     return vbt.pf_nb.order_nb(size=size, price=price, direction=direction)

>>> create_order_nb(1, 15, 2)
Order(size=1.0, price=15.0, ..., direction=2, ...)
```

Notice how the size and price arguments were automatically cast to floats.

#### Hint

Use `order_nb` instead of `Order` whenever possible.

To create an order that closes out the current position, we can conveniently use `close_position_nb`:

```
>>> vbt.pf_nb.close_position_nb(15) ❶
Order(size=0.0, price=15.0, size_type=6, ...)
```

❶ Uses size of zero and size type of `TargetAmount`

## Validation

Having constructed the order, `execute_order_nb` will check the arguments of that order for correct data types and values. For example, let's try passing a negative price:

```
>>> exec_state = vbt.pf_enums.ExecState(
...     cash=100.0,
...     position=0.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=100.0,
...     val_price=15.0,
...     value=100.0
... )
>>> vbt.pf_nb.execute_order_nb(
...     exec_state,
...     vbt.pf_nb.order_nb(price=-15)
... )
ValueError: order.price must be finite and 0 or greater
```

## Price resolution

After validating the inputs, `execute_order_nb` uses them to decide which command to run: buy or sell. But first, it has to do some preprocessing.

Even though vectorbt isn't associated with any particular data schema and can run on tick data just as well as on bar data, it still gives us an option to provide the current candle (`price_area`) for validation and resolution reasons. In such a case, it will consider the passed order price as a price point located within four price bounds: the opening, high, low, and closing price. Since order execution must happen strictly within those bounds, setting order price to `-np.inf` and `np.inf` will replace it by the opening and closing price respectively. Hence, next time, when you see any default price being `np.inf`, just know that it means the close price 🧐

```
>>> price_area = vbt.pf_enums.PriceArea(
...     open=10,
...     high=14,
...     low=8,
...     close=12
... )
>>> order_result, new_exec_state = vbt.pf_nb.execute_order_nb( ❶
...     exec_state=exec_state,
...     order=vbt.pf_nb.order_nb(size=np.inf, price=-np.inf),
...     price_area=price_area
... )
>>> order_result.price
10.0

>>> order_result, new_exec_state = vbt.pf_nb.execute_order_nb( ❷
...     exec_state=exec_state,
...     order=vbt.pf_nb.order_nb(size=np.inf, price=np.inf),
...     price_area=price_area
... )
>>> order_result.price
12.0

>>> order_result, new_exec_state = vbt.pf_nb.execute_order_nb( ❸
...     exec_state=exec_state,
...     order=vbt.pf_nb.order_nb(size=np.inf, price=np.inf)
... )
>>> order_result.price
nan
```

- ❶ Price gets replaced by the open price. Order executed.
- ❷ Price gets replaced by the close price (default). Order executed.
- ❸ Price gets replaced by `np.nan` since the price area is not defined. Order ignored.

## Size type conversion



Our primitive commands accept only a size in the number of shares, thus we have to convert any size type defined in `SizeType` to `Amount`. Different size types require different information for conversion; for example, `TargetAmount` requires to know the current position size, while `Value` also requires to know the current valuation price.

Let's execute an order such that the new position has 3 shares:

```
>>> order_result, new_exec_state = vbt.pf_nb.execute_order_nb(
...     exec_state=exec_state,
...     order=vbt.pf_nb.order_nb(
...         size=3,
...         size_type=vbt.pf_enums.SizeType.TargetAmount
...     ),
...     price_area=price_area
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=3.0,
  price=12.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_exec_state)
ExecState(
  cash=64.0,
  position=3.0,
  debt=0.0,
  locked_cash=0.0,
  free_cash=64.0,
  val_price=15.0,
  value=100.0
)
```

Since we're not in the market, vectorbt used `buy_nb` to buy 3 shares. If we were in the market with 10 shares, it would have used `sell_nb` to sell 7 shares.

## Direction

Function `order_nb` takes the argument `direction` for two reasons: resolve the direction of the order based on the sign of the argument `size`, and decide on whether to reverse the position or just to close out it. When the direction is `LongOnly` or `Both`, a positive size means buying and a negative size means selling. When the direction is `ShortOnly` though, the exact opposite happens: a positive size means selling and a negative size means buying. This is because a positive size is associated with increasing a position, which means buying to increase a long position and selling to increase a short position. For example, if the direction was `ShortOnly` and the size was a negative infinity, any short position would be closed out and any long position would be enlarged.

## Valuation

Speaking about the valuation price, it's the latest available price at the time of decision-making, or the price used to calculate the portfolio value. In many simulation methods, valuation price defaults to the order price, but sometimes it makes more sense to use the open or previous close price for the conversion step. The separation of the valuation and order price enables us to introduce a time gap between order placement and its execution. This is important because, in reality, not always an order can be executed right away.

Let's order 100% of the portfolio value:

```
>>> order_result, new_exec_state = vbt.pf_nb.execute_order_nb(
...     exec_state=exec_state,
...     order=vbt.pf_nb.order_nb(
...         size=1.0,
...         size_type=vbt.pf_enums.SizeType.TargetPercent
...     ),
...     price_area=price_area
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=6.666666666666667,
  price=12.0,
  fees=0.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_exec_state)
ExecState(
  cash=20.0,
  position=6.666666666666667,
  debt=0.0,
  locked_cash=0.0,
  free_cash=20.0,
  val_price=15.0,
  value=100.0
)
```

Why haven't we spent the entire cash? Because to convert the target percentage into the target amount of shares, vectorbt used the provided order execution state with `val_price` of \$15 and `value` of \$100, which produced  $100 / 15 = 6.67$ . The closer the valuation price is to the order price, the closer the calculation result would be to the target requirement.

By default, if we want to place multiple orders within the same bar (for example, in pairs trading), vectorbt wouldn't adjust the portfolio value after each order. This is because it assumes that we made our trading decisions way before order execution and adjusting the value would affect those decisions. But also, an order has only a marginal immediate effect on the value, for example, because of a commission. To force vectorbt to update the valuation price and value itself, we can enable `update_value`:

```
>>> order_result, new_exec_state = vbt.pf_nb.execute_order_nb(
...     exec_state=exec_state,
...     order=vbt.pf_nb.order_nb(
...         size=1.0,
...         size_type=vbt.pf_enums.SizeType.TargetPercent,
...         fixed_fees=10,
...         slippage=0.01
...     ),
...     price_area=price_area,
...     update_value=True
... )
>>> vbt.pprint(order_result)
OrderResult(
  size=6.666666666666667,
  price=12.120000000000001,
  fees=10.0,
  side=0,
  status=0,
  status_info=-1
)
>>> vbt.pprint(new_exec_state)
ExecState(
  cash=9.199999999999999,
  position=6.666666666666667,
  debt=0.0,
  locked_cash=0.0,
  free_cash=9.199999999999999,
  val_price=12.120000000000001,
  value=90.0
)
```

Notice how the new valuation price has been set to the close price adjusted with the slippage while the value has decreased by the fixed commission. Any new order placed after this one would use the updated value and thus probably produce a different outcome.

#### Note

Use this feature only if you can control the order in which orders appear within a bar and when you have intra-bar data.

## Pipeline/2

Let's create another simplified pipeline that orders given a target percentage array. In particular, we'll keep 50% of the portfolio value in shares, and rebalance monthly. We'll calculate the portfolio value based on the open price at the beginning of each bar, and order at the end of each bar (to keep things realistic). Also, we'll fill asset value and portfolio value arrays to later plot the allocation at each bar.

```
>>> @njit
... def pipeline_2_nb(open, close, target_pct, init_cash=100):
...     asset_value_out = np.empty(close.shape, dtype=float_) ①
...     value_out = np.empty(close.shape, dtype=float_)
...     exec_state = vbt.pf_enums.ExecState( ②
...         cash=float(init_cash),
...         position=0.0,
...         debt=0.0,
...         locked_cash=0.0,
...         free_cash=float(init_cash),
...         val_price=np.nan,
...         value=np.nan
...     )
...     for i in range(close.shape[0]):
...         if not np.isnan(target_pct[i]): ③
...             val_price = open[i]
...             value = exec_state.cash + val_price * exec_state.position ④
...             exec_state = vbt.pf_enums.ExecState( ⑤
...                 cash=exec_state.cash,
...                 position=exec_state.position,
...                 debt=exec_state.debt,
...                 locked_cash=exec_state.locked_cash,
...                 free_cash=exec_state.free_cash,
...                 val_price=val_price,
...                 value=value
...             )
...             order = vbt.pf_nb.order_nb( ⑥
...                 size=target_pct[i],
...                 price=close[i],
...                 size_type=vbt.pf_enums.SizeType.TargetPercent
...             )
...             _, exec_state = vbt.pf_nb.execute_order_nb( ⑦
...                 exec_state=exec_state,
...                 order=order
...             )
...             asset_value_out[i] = exec_state.position * close[i] ⑧
...             value_out[i] = exec_state.cash + exec_state.position * close[i]
...     return asset_value_out, value_out
```

```
... return asset_value_out, value_out
```

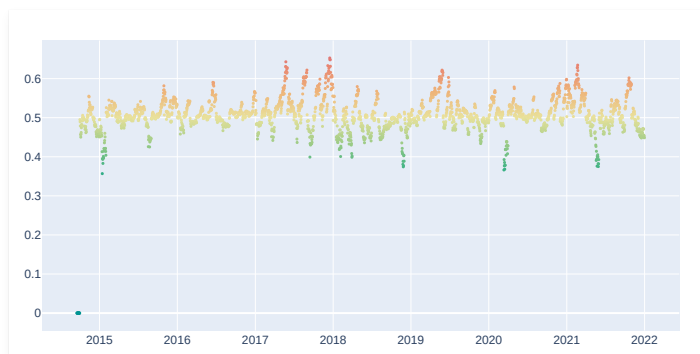
- 1 Create two empty arrays with a floating data type. Remember that creating an array with `np.empty` will produce an array with uninitialized (garbage) values that you should override.
- 2 Our initial order execution state
- 3 There is no need to run order execution when target percentage is `np.nan` (= do not rebalance)
- 4 Calculate the portfolio value at the beginning of the bar (= valuation)
- 5 Create a new existing order execution state after the valuation
- 6 Create a new order tuple using the close price as order price
- 7 Execute the order and return the order result and the new execution state
- 8 Fill the arrays (you should fill each single element!)

Let's run the pipeline on our Bitcoin data:

```
>>> symbol_wrapper = data.get_symbol_wrapper() 1
>>> target_pct = symbol_wrapper.fill()
>>> target_pct.vbt.set(0.5, every="M", inplace=True) 2

>>> asset_value, value = pipeline_2_nb(
...     data.get("Open").values,
...     data.get("Close").values,
...     target_pct.values
... )
>>> asset_value = symbol_wrapper.wrap(asset_value) 3
>>> value = symbol_wrapper.wrap(value)
>>> allocations = (asset_value / value).rename(None) 4
>>> allocations.vbt.scatterplot(trace_kwargs=dict(
...     marker=dict(
...         color=allocations,
...         colorscale="Temps",
...         size=3,
...         cmin=0.3,
...         cmid=0.5,
...         cmax=0.7
...     )
... ).show()
```

- 1 We cannot use `data.wrapper` because it contains OHLC as columns. What we need is a wrapper that has symbols as columns, to fill the array with target percentages.
- 2 Fill the array with NaNs and set all data points at the beginning of each month to `0.5`.
- 3 Use the same wrapper to convert the NumPy array into Pandas Series
- 4 Divide the asset value by the portfolio value to derive the allocation



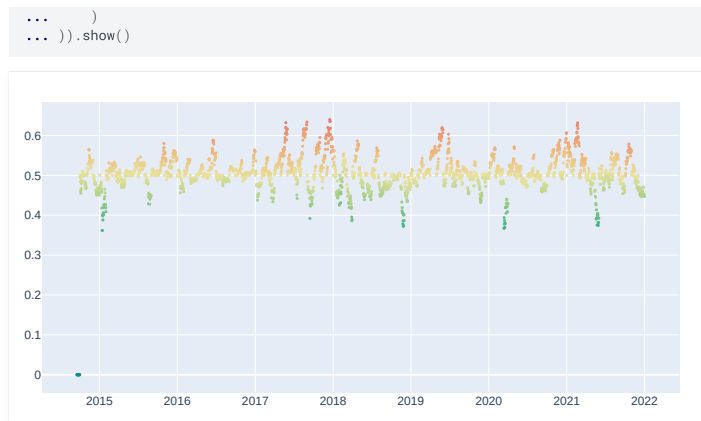
#### Hint

Each point represents a revaluation at the end of each bar.

We see that allocations are being regularly pulled back to the target level of 50%.

Let's validate the pipeline using `Portfolio.from_orders`:

```
>>> pf = vbt.Portfolio.from_orders(
...     data,
...     size=target_pct,
...     size_type="targetpercent"
... )
>>> pf.allocations.vbt.scatterplot(trace_kwargs=dict(
...     marker=dict(
...         color=allocations,
...         colorscale="Temps",
...         size=3,
...         cmin=0.3,
...         cmid=0.5,
...         cmax=0.7
...     )
... ).show())
```



One of the biggest advantages of using vectorbt is that you can run your minimalistic trading environment in any Python function, even inside objective functions of machine learning models! There is no need to trigger the entire backtesting pipeline as a script or any other complex process like most backtesting frameworks force us to do 😊

## Order processing

Order execution takes an order instruction and translates it into a buy or sell operation. The responsibility of the user is to do something with the returned order execution state and result; mostly, we want to post-process and append each successful order to some list for later analysis - that's where order and log records come into play. Furthermore, we may want to raise an error if an order has been rejected and a certain flag in the requirements is present. All of this is ensured by `process_order_nb`.

## Order records

Order records is a `structured` NumPy array of the data type `order_dt` containing information on each successful order. Each order in this array is assumed to be completed, that is, you should view an order as a trade in the vectorbt's world. Since we're dealing with Numba, we cannot and should not use lists and other inefficient data structures for storing such complex information. Given that orders have fields with variable data types, the best data structure is a record array, which is a regular NumPy array with a complex data type and that behaves similarly to a Pandas DataFrame.

Since any NumPy array is a non-appendable structure, we should initialize an empty array of a sufficient size, and gradually fill it with new information. For this, we need a counter - a simple integer - that points to an index of the next record to be written.

### Info

Actually, you can append to a NumPy array, but it will create a new array. Don't try this at home 😊

Let's create an array with two order records and a counter:

```
>>> order_records = np.empty(2, dtype=vbt.pf_enums.order_dt)
>>> order_count = 0
```

We shouldn't access this array just yet because it contains memory garbage, thus it requires the user to manually set all the values in the array, and should be used with caution.

```
>>> order_records
array([(4585679916398730403, ..., 4583100142070297783),
      (4582795628349012822, ..., 4576866499094039639)],
      dtype={'names': ['id', 'col', 'idx', 'size', 'price', 'fees', 'side'], ...})
```

Let's execute an order using `execute_order_nb` at the 678th bar, and fill the first record in the array:

```
>>> exec_state = vbt.pf_enums.ExecState(
...     cash=100.0,
...     position=0.0,
...     debt=0.0,
...     locked_cash=0.0,
...     free_cash=100.0,
...     val_price=15.0,
...     value=100.0
... )
>>> order_result, new_exec_state = vbt.pf_nb.execute_order_nb(
...     exec_state=exec_state,
...     order=vbt.pf_nb.order_nb(size=np.inf, price=15.0)
... )
>>> if order_result.status == vbt.pf_enums.OrderStatus.Filled: ❶
...     order_records["id"][order_count] = order_count ❷
...     order_records["col"][order_count] = 0
...     order_records["idx"][order_count] = 678 ❸
...     order_records["size"][order_count] = order_result.size
...     order_records["price"][order_count] = order_result.price
...     order_records["fees"][order_count] = order_result.fees
...     order_records["side"][order_count] = order_result.side
...     order_count += 1
```

```
>>> order_records[0]
(0, 0, 678, 6.66666667, 15., 0., 0)

>>> order_count
1
```

- ❶ Check that the order has been filled
- ❷ Order ids start with 0 and follow the counter
- ❸ Index of the current bar

#### Note

When writing to an element of a record field, first select the field, and then the index.

At the next bar, we'll reverse the position and fill the second record:

```
>>> order_result, new_exec_state2 = vbt.pf_nb.execute_order_nb(
...     exec_state=new_exec_state,
...     order=vbt.pf_nb.order_nb(size=-np.inf, price=16.0)
... )
>>> if order_result.status == vbt.pf_enums.OrderStatus.Filled:
...     order_records["id"][order_count] = order_count ❶
...     order_records["col"][order_count] = 0
...     order_records["idx"][order_count] = 679
...     order_records["size"][order_count] = order_result.size
...     order_records["price"][order_count] = order_result.price
...     order_records["fees"][order_count] = order_result.fees
...     order_records["side"][order_count] = order_result.side
...     order_count += 1

>>> order_records[1]
(1, 0, 679, 13.33333333, 16., 0., 1)

>>> order_count
2
```

- ❶ Don't forget to increment the order id

Here are the order records that we've populated:

```
>>> order_records
array([(0, 0, 678, 6.66666667, 15., 0., 0),
      (1, 0, 679, 13.33333333, 16., 0., 1)],
      dtype={'names': ['id', 'col', 'idx', 'size', 'price', 'fees', 'side'], ...})
```

But instead of setting each of these records manually, we can use `process_order_nb` to do it for us! We just need to do one little adjustment: both the order records and the counter must be provided per column since vectorbt primarily works on multi-columnar data. This means that the order records array must become a two-dimensional array and the counter constant must become a one-dimensional array (both with only one column in our example):

```
>>> order_records = np.empty((2, 1), dtype=vbt.pf_enums.order_dt)
>>> order_counts = np.full(1, 0, dtype=int_)

>>> order_result1, new_exec_state1 = vbt.pf_nb.process_order_nb(
...     0, 0, 678, ❶
...     exec_state=exec_state,
...     order=vbt.pf_nb.order_nb(size=np.inf, price=15.0),
...     order_records=order_records,
...     order_counts=order_counts
... )
>>> order_result2, new_exec_state2 = vbt.pf_nb.process_order_nb(
...     0, 0, 679,
...     exec_state=new_exec_state,
...     order=vbt.pf_nb.order_nb(size=-np.inf, price=16.0),
...     order_records=order_records,
...     order_counts=order_counts
... )

>>> order_records
array([(0, 0, 678, 6.66666667, 15., 0., 0),
      (1, 0, 679, 13.33333333, 16., 0., 1)],
      dtype={'names': ['id', 'col', 'idx', 'size', 'price', 'fees', 'side'], ...})

>>> order_counts
array([2])
```

- ❶ Current group, column, and index (row)

Such filled order records will become the backbone of the post-analysis phase.

## Log records

Log records have the data type `log_dt` and are similar to order records, but with a few key differences: they are saved irrespective of whether the order has been filled, and they also contain information on the current execution state, the order request, and the new execution state. This way, we can completely and post-factum track down issues related to order processing.

```

>>> order_records = np.empty((2, 1), dtype=vbt.pf_enums.order_dt)
>>> order_counts = np.full(1, 0, dtype=int_)
>>> log_records = np.empty((2, 1), dtype=vbt.pf_enums.log_dt)
>>> log_counts = np.full(1, 0, dtype=int_)

>>> order_result1, new_exec_state1 = vbt.pf_nb.process_order_nb(
...     0, 0, 678,
...     exec_state=exec_state,
...     order=vbt.pf_nb.order_nb(size=np.inf, price=15.0, log=True), ❶
...     order_records=order_records,
...     order_counts=order_counts,
...     log_records=log_records,
...     log_counts=log_counts
... )
>>> order_result2, new_exec_state2 = vbt.pf_nb.process_order_nb(
...     0, 0, 679,
...     exec_state=new_exec_state,
...     order=vbt.pf_nb.order_nb(size=np.inf, price=16.0, log=True),
...     order_records=order_records,
...     order_counts=order_counts,
...     log_records=log_records,
...     log_counts=log_counts
... )

>>> log_records
array([[0, 0, 0, 678, ..., 0., 15., 100., 0)],
      [(1, 0, 0, 679, ..., 0., 15., 100., 1)]],
      dtype={'names': ['id', 'group', ..., 'res_status_info', 'order_id'], ...})

```

❶ Logging of each order must be explicitly enabled

#### Note

Logging costs performance and memory. Use only when really needed.

### Pipeline/3

Let's extend the [last pipeline](#) to independently process an arbitrary number of columns, and gradually fill order records. This way, we can backtest multiple parameter combinations by taking advantage of multidimensionality!

```

>>> @njit
... def pipeline_3_nb(open, close, target_pct, init_cash=100):
...     order_records = np.empty(close.shape, dtype=vbt.pf_enums.order_dt) ❶
...     order_counts = np.full(close.shape[1], 0, dtype=int_)
...
...     for col in range(close.shape[1]): ❷
...         exec_state = vbt.pf_enums.ExecState(
...             cash=float(init_cash),
...             position=0.0,
...             debt=0.0,
...             locked_cash=0.0,
...             free_cash=float(init_cash),
...             val_price=np.nan,
...             value=np.nan
...         )
...
...         for i in range(close.shape[0]):
...             if not np.isnan(target_pct[i, col]): ❸
...                 val_price = open[i, col]
...                 value = exec_state.cash + val_price * exec_state.position
...
...                 exec_state = vbt.pf_enums.ExecState(
...                     cash=exec_state.cash,
...                     position=exec_state.position,
...                     debt=exec_state.debt,
...                     locked_cash=exec_state.locked_cash,
...                     free_cash=exec_state.free_cash,
...                     val_price=val_price,
...                     value=value
...                 )
...                 order = vbt.pf_nb.order_nb(
...                     size=target_pct[i, col],
...                     price=close[i, col],
...                     size_type=vbt.pf_enums.SizeType.TargetPercent
...                 )
...                 _, exec_state = vbt.pf_nb.process_order_nb(
...                     col, col, i, ❹
...                     exec_state=exec_state,
...                     order=order,
...                     order_records=order_records,
...                     order_counts=order_counts
...                 )
...
...     return vbt.nb.repartition_nb(order_records, order_counts) ❺

```

❶ Since we don't know the number of orders in advance, let's prepare for the worst-case scenario: one record at each bar. Remember that order records must be aligned column-wise.

❷ Iterate over columns in `close` and run our logic on each one

❸ Since every array passed to the pipeline now must be two-dimensional, don't forget to specify the column when accessing an array element. Also, in indexing, first comes the row and then the column 📌

- 4 Use `repartition_nb` to flatten the final order records array (= concatenate records of all columns into a one-dimensional array)
- 5 Since all columns represent independent backtests, groups become columns

#### Info

We are flattening (repartitioning) order records because most records are left unfilled, thus unnecessarily taking memory. By flattening, we're effectively compressing them without losing any information because each record already tracks the column it's supposed to be in.

Our pipeline now expects all arrays to be two-dimensional. Let's test three value combinations of the parameter `every`, which controls the re-allocation periodicity. For this, we need to expand all arrays to have the same number of columns as the parameter combinations.

```
>>> every = pd.Index(["M", "Q", "Y"], name="every")

>>> open = data.get("Open").vbt.tile(3, keys=every) ❶
>>> close = data.get("Close").vbt.tile(3, keys=every)
>>> close
every
Date
2014-09-17 00:00:00+00:00    457.334015    457.334015    457.334015
2014-09-18 00:00:00+00:00    424.440002    424.440002    424.440002
2014-09-19 00:00:00+00:00    394.795990    394.795990    394.795990
...
2021-12-29 00:00:00+00:00   46444.710938   46444.710938   46444.710938
2021-12-30 00:00:00+00:00   47178.125000   47178.125000   47178.125000
2021-12-31 00:00:00+00:00   46306.445312   46306.445312   46306.445312

[2663 rows x 3 columns]

>>> target_pct = symbol_wrapper.fill().vbt.tile(3, keys=every)
>>> target_pct.vbt.set(0.5, every="M", columns=["M"], inplace=True) ❷
>>> target_pct.vbt.set(0.5, every="Q", columns=["Q"], inplace=True)
>>> target_pct.vbt.set(0.5, every="Y", columns=["Y"], inplace=True)

>>> order_records = pipeline_3_nb(
...     open.values,
...     close.values,
...     target_pct.values
... )
>>> order_records
array([[ 0,  0,  14, 1.29056570e-01,  383.61499023,  0.,  0), << first column
       ( 1,  0,  45, 1.00206092e-02,  325.74899292,  0.,  0),
       ( 2,  0,  75, 7.10912824e-03,  379.24499512,  0.,  1),
       ...
       (84,  0, 2571, 7.79003416e-04, 48116.94140625,  0.,  0),
       (85,  0, 2602, 3.00678739e-03, 61004.40625,  0.,  1),
       (86,  0, 2632, 6.84410394e-04, 57229.828125,  0.,  0),
       ( 0,  1,  13, 1.32947604e-01,  386.94400024,  0.,  0), << second column
       ( 1,  1, 105, 1.16132613e-02,  320.19299316,  0.,  0),
       ( 2,  1, 195, 1.83187063e-02,  244.22399902,  0.,  0),
       ...
       (27,  1, 2478, 7.74416872e-03, 35040.8359375,  0.,  0),
       (28,  1, 2570, 2.08567037e-03, 43790.89453125,  0.,  1),
       (29,  1, 2662, 1.72637091e-03, 46306.4453125,  0.,  1),
       ( 0,  2, 105, 1.60816173e-01,  320.19299316,  0.,  0), << third column
       ( 1,  2, 470, 2.34573523e-02,  430.56698608,  0.,  1),
       ( 2,  2, 836, 3.81744650e-02,  963.74298096,  0.,  1),
       ...
       ( 5,  2, 1931, 2.83026812e-02,  7193.59912109,  0.,  1),
       ( 6,  2, 2297, 3.54188390e-02, 29001.72070312,  0.,  1),
       ( 7,  2, 2662, 1.14541249e-02, 46306.4453125,  0.,  1)],
      dtype={'names': ['id', 'col', 'idx', 'size', 'price', 'fees', 'side'], ...})
```

- 1 Use `BaseAccessor.tile` to populate columns and append a new column level for our parameter combinations
- 2 Change the corresponding column only

## Wrapping

This output is exactly what `Portfolio` requires as input: order records with a couple of other arguments can be used to reconstruct the simulation state, including the regular cash balance and the position size at each time step. The reconstructed state can be used to model the equity curve, then returns, and then the accompanying metrics such as the old but gold Sharpe ratio. So, what how do we construct a portfolio? Instead of using any class method, we'll pass the data directly to the class. For this, only three arguments are required: a wrapper, a close series, and order records. Ideally, we should also supply arguments that were used during the simulation, such as the initial cash:

```
>>> vbt.phelp(vbt.Portfolio)
Portfolio.__init__(
    self,
    wrapper,
    order_records,
    *,
    close,
    open=None,
    high=None,
    low=None,
    log_records=None,
    cash_sharing=False,
    init_cash='auto',
    ...
)
```

 Hint

Notice \*? It means any argument after `order_records` must be provided as a keyword argument.

Let's do the wrapping part:

```
>>> pf = vbt.Portfolio(
...     close.vbt.wrapper,
...     order_records,
...     open=open,
...     close=close,
...     init_cash=100 ❶
... )
```

❶ Make sure to use the same initial cash as during the simulation

We can now use the portfolio the same way as if we simulated it with any preset method:

```
>>> pf.sharpe_ratio
every
MS      1.267804
Q       1.309943
Y       1.393820
Name: sharpe_ratio, dtype: float64
```

## Flexible indexing

The issue of bringing all arrays to the same shape as we did above is that it unnecessarily consumes memory: even though the only array that has different data in each column is `target_pct`, we have almost tripled memory consumption by having to expand other arrays like `close`. Imagine how expensive would it be having to align dozens of such array-like arguments 😞

Flexible indexing allows us to overcome this alignment step and to access each element of an array solely based on its shape. For example, there is no need to tile `close` three times if each row stays the same for each column - we can simply return the same row element irrespective of the column being queried. The same goes for a one-dimensional array with elements per column - return the same column element for each row. The only requirement is that the array must have one dimension if it should broadcast against rows **or** columns, and two dimensions if it should broadcast against rows **and** columns. Any scalars should be transformed into one of the above formats, otherwise we'll be greeted with an ugly Numba error.

For the actual indexing, we can use the following Numba-compiled functions:

- One-dimensional array (generic): [flex\\_select\\_1d\\_nb](#)
- One-dimensional array (per row): [flex\\_select\\_1d\\_pr\\_nb](#)
- One-dimensional array (per column): [flex\\_select\\_1d\\_pc\\_nb](#)
- Two-dimensional array: [flex\\_select\\_nb](#)

Let's demonstrate their use in different scenarios:

```
>>> per_row_arr = np.array([1, 2, 3])
>>> per_col_arr = np.array([4, 5])
>>> per_elem_arr = np.array([
...     [6, 7],
...     [8, 9],
...     [10, 11]
... ])

>>> vbt.flex_select_1d_pr_nb(per_row_arr, 2) ❶
3

>>> vbt.flex_select_1d_pc_nb(per_col_arr, 1) ❷
5

>>> vbt.flex_select_nb(per_elem_arr, 2, 1) ❸
11
```

- ❶ Get the value under the third row
- ❷ Get the value under the second column
- ❸ Get the value under the third row and second column

One-dimensional indexing functions are suited only for arguments that are one-dimensional by design, such as initial capital, which makes only sense to be provided per column, not per element. But what if the user should also be able to pass `per_row_arr` or `per_col_arr` as fully-broadcast arrays? In this case, the user needs to expand both arrays to two dimensions according to the [NumPy's broadcasting rules](#) and use exclusively [flex\\_select\\_nb](#). The reason for this is that Numba isn't flexible enough to permit doing operations on both one-dimensional and two-dimensional arrays, so we must decide on the indexing function beforehand.

```
>>> per_row_arr_2d = per_row_arr[:, None] ❶
>>> per_row_arr_2d
array([[1],
       [2],
       [3]])

>>> vbt.flex_select_nb(per_row_arr_2d, 2, 1)
```



```

3

>>> per_col_arr_2d = per_col_arr[None] ❷
>>> per_col_arr_2d
array([[4, 5]])

>>> vbt.flex_select_nb(per_col_arr_2d, 2, 1)
5

```

❶ Create the second axis of length one: from (3,) to (3, 1)

❷ Create the first axis of length one: from (3,) to (1, 3)

This yields the same results as if we had aligned the arrays prior to indexing (= memory expensive):

```

>>> target_shape = (3, 2)

>>> vbt.broadcast_array_to(per_row_arr, target_shape[0])[2]
3

>>> vbt.broadcast_array_to(per_col_arr, target_shape[1])[1]
5

>>> vbt.broadcast_array_to(per_row_arr_2d, target_shape)[2, 1]
3

>>> vbt.broadcast_array_to(per_col_arr_2d, target_shape)[2, 1]
5

```

#### Hint

If you're not sure whether a flexible array will be indexed correctly, try broadcasting it with NumPy!

## Rotational indexing

But what happens if the index is out of bounds? Let's say we iterate over 6 columns but an array has data only for 3. In such a case, vectorbt can rotate the index and return the first element in the array for the fourth column, the second element for the fifth column, and so on:

```

>>> vbt.flex_select_1d_pr_nb(per_row_arr, 100, rotate_rows=True) ❶
2

>>> vbt.flex_select_1d_pc_nb(per_col_arr, 100, rotate_cols=True) ❷
4

>>> vbt.flex_select_nb(per_elem_arr, 100, 100, rotate_rows=True, rotate_cols=True)
8

```

❶ Resolves to index  $100 \% 3 == 1$  and element 2

❷ Resolves to index  $100 \% 2 == 0$  and element 4

If you think that this is crazy, and you would rather have an error shown: rotational indexing is very useful when it comes to testing multiple assets and parameter combinations. Without it (default), we would need to tile the asset DataFrame by the number of parameter combinations, but with it, we could have just passed the data without tiling and thus wasting memory. But also, in many places, vectorbt ensures that all arrays can broadcast against each other nicely anyway.

## Pipeline/4

Let's adapt the previous pipeline for flexible indexing. Since usually we don't know which one of the passed arrays has the full shape, and sometimes there is no array with the full shape at all, we need to introduce another argument - `target_shape` - to provide the full shape for our loops to iterate over. We'll also experiment with rotational indexing, which isn't supported by any of the preset simulation methods.

```

>>> @njit
... def pipeline_4_nb(
...     target_shape,
...     open,
...     close,
...     target_pct,
...     init_cash=100,
...     rotate_cols=False
... ):
...     init_cash_ = vbt.to_1d_array_nb(np.asarray(init_cash)) ❶
...     open_ = vbt.to_2d_array_nb(np.asarray(open)) ❷
...     close_ = vbt.to_2d_array_nb(np.asarray(close))
...     target_pct_ = vbt.to_2d_array_nb(np.asarray(target_pct))
...     order_records = np.empty(target_shape, dtype=vbt.pf_enums.order_dt)
...     order_counts = np.full(target_shape[1], 0, dtype=int_)
...
...     for col in range(target_shape[1]):
...         init_cash_elem = vbt.flex_select_1d_pc_nb(
...             init_cash_, col, rotate_cols=rotate_cols) ❸
...
...         exec_state = vbt.pf_enums.ExecState(
...             cash=float(init_cash_elem),
...             position=0.0,
...             debt=0.0,
...             locked_cash=0.0,
...             free_cash=float(init_cash_elem),
...             val_price=np.nan,

```

```

...     value=np.nan
... )
...
...     for i in range(target_shape[0]):
...         open_elem = vbt.flex_select_nb(
...             open_, i, col, rotate_cols=rotate_cols) ④
...         close_elem = vbt.flex_select_nb(
...             close_, i, col, rotate_cols=rotate_cols)
...         target_pct_elem = vbt.flex_select_nb(
...             target_pct_, i, col, rotate_cols=rotate_cols)
...
...         if not np.isnan(target_pct_elem):
...             value = exec_state.cash + open_elem * exec_state.position
...
...             exec_state = vbt.pf_enums.ExecState(
...                 cash=exec_state.cash,
...                 position=exec_state.position,
...                 debt=exec_state.debt,
...                 locked_cash=exec_state.locked_cash,
...                 free_cash=exec_state.free_cash,
...                 val_price=open_elem,
...                 value=value
...             )
...             order = vbt.pf_nb.order_nb(
...                 size=target_pct_elem,
...                 price=close_elem,
...                 size_type=vbt.pf_enums.SizeType.TargetPercent
...             )
...             _, exec_state = vbt.pf_nb.process_order_nb(
...                 col, col, i,
...                 exec_state=exec_state,
...                 order=order,
...                 order_records=order_records,
...                 order_counts=order_counts
...             )
...
...     return vbt.nb.repartition_nb(order_records, order_counts)

```

- ① This line allows us to pass a one-dimensional flexible array also as a scalar. Note how we write the result to a new variable (with a trailing underscore) and then use it in indexing.
- ② Same for two-dimensional flexible arrays
- ③ Select the current element of the initial cash array. Remember that indexing functions with the suffix `1d` are expecting one-dimensional arrays.
- ④ Since all three arrays are not guaranteed to have the full shape anymore, we must switch to flexible indexing instead of doing `open[i, col]`. Remember that indexing functions without the suffix `1d` are expecting two-dimensional arrays.

Thanks to flexible indexing, we can now use all arrays without tiling:

```

>>> target_shape = vbt.broadcast_shapes( ①
...     data.get("Open").values.shape,
...     data.get("Close").values.shape,
...     target_pct.values.shape
... )
>>> target_shape
(2663, 3)

>>> order_records = pipeline_4_nb(
...     target_shape,
...     data.get("Open").values,
...     data.get("Close").values,
...     target_pct.values
... )
>>> len(order_records)
125

```

- ① We need to build the target shape to iterate over. This also works as a broadcasting check.

This also allows us to provide target percentages as a constant to re-allocate at each bar! Since constants don't have any implications on the target shape, we only need to broadcast the price shapes:

```

>>> target_shape = vbt.broadcast_shapes(
...     data.get("Open").values.shape,
...     data.get("Close").values.shape
... )
>>> target_shape
(2663,)

>>> target_shape = vbt.to_2d_shape(target_shape) ①
>>> target_shape
(2663, 1)

>>> order_records = pipeline_4_nb(
...     target_shape,
...     data.get("Open").values,
...     data.get("Close").values,
...     0.5
... )
>>> len(order_records)
2663

```

- ① Target shape must always be two-dimensional

This operation has generated the same number of orders as we have elements in the data:

```
>>> np.product(symbol_wrapper.shape_2d)
2663
```

To demonstrate rotational indexing, let's pull multiple symbols and perform the simulation without having to tile or change them in any way:

```
>>> mult_data = vbt.YFData.pull(
...     ["BTC-USD", "ETH-USD"],
...     end="2022-01-01",
...     missing_index="drop"
... )
```

Symbol 2/2

```
>>> mult_symbol_wrapper = mult_data.get_symbol_wrapper()
>>> mult_target_pct = pd.concat([
...     mult_symbol_wrapper.fill().vbt.set(0.5, every=every[i])
...     for i in range(len(every))
... ], axis=1, keys=every) ❶

>>> target_shape = vbt.broadcast_shapes(
...     vbt.tile_shape(mult_data.get("Open").values.shape, len(every)), ❷
...     vbt.tile_shape(mult_data.get("Close").values.shape, len(every)),
...     mult_target_pct.values.shape
... )
>>> target_shape
(1514, 6)

>>> order_records = pipeline_4_nb(
...     target_shape,
...     mult_data.get("Open").values, ❸
...     mult_data.get("Close").values,
...     mult_target_pct.values,
...     rotate_cols=True
... )
>>> len(order_records)
142
```

- ❶ That's another way of constructing the target percentage array
- ❷ Since broadcasting doesn't support rotations, we can use `tile_shape` to tile the shape of `open` and `close` manually (don't tile the actual array!)
- ❸ There is no need to tile any array thanks to rotational indexing

Without rotation, we would have got an `"IndexError: index out of bounds"` error as the number of columns in the target shape is bigger than that in the price arrays.

## Grouping

Using groups, we can put multiple columns to the same backtesting basket 🍷

Generally, a group consists of a number of columns that are part of a single portfolio entity and should be backtested as a single whole. Very often, we use groups to share capital among multiple columns, but we can also use groups to bind columns on some logical level. During a simulation, it's our responsibility to make use of grouping. For example, even though `process_order_nb` requires a group index, it uses it just for filling log records and nothing else. But after the simulation, vectorbt has many tools at its disposal to enable us in aggregating and analyzing various information per group, such as portfolio value.

Groups can be constructed and provided in two ways: as group lengths and as a group map. The former is easier to handle, marginally faster, and requires the columns to be split into monolithic groups, while the latter allows the columns of a group to be distributed arbitrarily, and is generally a more flexible option. Group lengths is the format primarily used by simulation methods (since asset columns, in contrast to parameter columns, are usually located next to each other), while group maps are predominantly used by generic functions specialized in pre- and post-analysis. Both formats can be easily generated by a `Grouper` instance.

### Group lengths

Let's create a custom column index with 5 assets, and put them into 2 groups. Since group lengths work on monolithic groups only, assets in each group must be next to each other:

```
>>> columns = pd.Index(["BTC-USD", "ETH-USD", "BNB-USD", "SOL-USD", "XRP-USD"])
>>> mono_grouper = vbt.Grouper(columns, group_by=[0, 0, 0, 1, 1])
>>> mono_grouper.get_group_lens() ❶
array([3, 2])
```

#### ❶ Using `Grouper.get_group_lens`

The first element in the returned array is the number of columns with the label `0`, and the second element is the number of columns with the label `1`.

#### Hint

`Grouper` doesn't care if we pass a list of integers or a sequence of strings - it will convert everything into a Pandas Index and treat it as group labels. They **don't** have to be alphanumerically sorted.

If we create discrete groups, the generation will fail:

```
>>> dist_grouper = vbt.Grouper(columns, group_by=[0, 1, 0, 1, 1])
>>> dist_grouper.get_group_lens()
```

```
ValueError: group_by must form monolithic groups
```

Now, how do we define logic per group? Here's a template:

```
>>> group_lens = mono_grouper.get_group_lens()

>>> group_end_idxxs = np.cumsum(group_lens) ❶
>>> group_start_idxxs = group_end_idxxs - group_lens ❷

>>> for group in range(len(group_lens)): ❸
...     from_col = group_start_idxxs[group]
...     to_col = group_end_idxxs[group]
...     ❹
...     for col in range(from_col, to_col): ❺
...         pass ❻
```

- ❶ Get the end column index of each group (excluding)
- ❷ Get the start column index of each group (including)
- ❸ Iterate over all groups
- ❹ Define here your logic per group
- ❺ Iterate over all columns in the group
- ❻ Define here your logic per column in the group

## Group map

Group map is a tuple of two arrays:

1. One-dimensional array with column indices sorted by group
2. One-dimensional array with the length of each group in the first array

Thus, a group map makes distributed groups inherently monolithic, such that we can work with any possible group distribution:

```
>>> mono_grouper.get_group_map()
(array([0, 1, 2, 3, 4]), array([3, 2]))

>>> dist_grouper.get_group_map()
(array([0, 2, 1, 3, 4]), array([2, 3]))
```

In the second example, the first two (2) column indices in the first array belong to the first group, while the remaining three (3) column indices belong to the second group.

Here's a template for working with a group map:

```
>>> group_map = dist_grouper.get_group_map()

>>> group_idxxs, group_lens = group_map
>>> group_start_idxxs = np.cumsum(group_lens) - group_lens ❶

>>> for group in range(len(group_lens)):
...     group_len = group_lens[group]
...     start_idx = group_start_idxxs[group]
...     col_idxxs = group_idxxs[start_idx : start_idx + group_len] ❷
...     ❸
...     for k in range(len(col_idxxs)): ❹
...         col = col_idxxs[k]
...         ❺
```

- ❶ Get the start index of each group in the first array
- ❷ Get the column indices of the group in the first array
- ❸ Define here your logic per group
- ❹ Iterate over all column indices in the group
- ❺ Define here your logic per column in the group

## Call sequence

When sharing capital between two or more assets, we sometimes want to process one column before the others. This makes most sense, for example, in cases where we need to exit positions before opening new ones to release funds for them. If we look at the templates for both grouping formats above, we can precisely identify where the column processing order should be changed: in the for-loop that iterates over columns. But how do we programmatically change this order? Here comes a call sequence into play.

Call sequence is an array of column indices in the order of their processing. For example, if the third column should be processed first, the first column second, and the second column third, the call sequence would be `[2, 0, 1]`. That is, we are always moving from left to right in the call sequence and pick the current column index. Such a design has one immense benefit: we can use another array, such as with potential order values, to (arg-)sort the call sequence.

The sorting is done by the function `insert_argsort_nb`, which takes an array with values to sort by and an array of indices, and sorts the indices in-place using `insertion sort` in the order the values appear in the first array. This sorting algorithm is best suited for smaller arrays and does not require any additional memory

space - perfect for groups with assets!

Let's say we have three assets: one not in position, one in a short position, and one in a long position. We want to close all positions in the order such that assets that should be sold are processed first. Otherwise, we wouldn't have cash from exiting the long position to close out the short position. For this, we will first use `approx_order_value_nb` to approximate the order value of each operation:

```
>>> position = np.array([0.0, -10.0, 10.0])
>>> val_price = np.array([10.0, 25.0, 15.0])
>>> debt = np.array([0.0, 100.0, 0.0])
>>> locked_cash = np.array([0.0, 100.0, 0.0])
>>> order_value = np.empty(3, dtype=float_)

>>> for col in range(len(position)):
...     exec_state = vbt.pf_enums.ExecState(
...         cash=200.0, ❶
...         position=position[col], ❷
...         debt=debt[col],
...         locked_cash=locked_cash[col],
...         free_cash=0.0,
...         val_price=val_price[col],
...         value=100.0
...     )
...     order_value[col] = vbt.pf_nb.approx_order_value_nb(
...         exec_state=exec_state,
...         size=0.,
...         size_type=vbt.pf_enums.SizeType.TargetAmount,
...         direction=vbt.pf_enums.Direction.Both
...     )

>>> order_value ❸
array([0., 50., -150.]
```

- ❶ Cash-related information is defined per group using a constant
- ❷ Position-related information is defined per column using an array
- ❸ Positive number means outbound cash flow, negative number means inbound cash flow

We see that the second column would require approx. \$50 in cash and the third column would bring approx. \$150 in cash to close out the position. Let's create a call sequence and sort it by the order value:

```
>>> from vectorbtpro.utils.array_ import insert_argsort_nb

>>> call_seq = np.array([0, 1, 2]) ❶
>>> insert_argsort_nb(order_value, call_seq)
>>> call_seq
array([2, 0, 1])
```

- ❶ We should always start with a simple range

#### Note

Both the order value and the call sequence are sorted in-place!

We can then modify the for-loop to iterate over the call sequence instead:

```
>>> for k in range(len(call_seq)): ❶
...     c = call_seq[k]
...     col = from_col + c

>>> for k in range(len(call_seq)): ❷
...     c = call_seq[k]
...     col = col_idxs[c]
```

- ❶ When working with group lengths
- ❷ When working with a group map

#### Hint

A good practice is to keep a consistent naming of variables. Here, we're using `k` to denote an index in the call sequence, `c` to denote a column index within a group, and `col` to denote a global column index.

## Pipeline/5

Let's upgrade our previous pipeline to rebalance groups of assets. To better illustrate how important is sorting by order value when rebalancing multi-asset portfolios, we'll introduce another argument `auto_call_seq` to switch between sorting and not sorting. We will use group lengths as the grouping format of choice because of its simplicity. Also note that now we have to keep a lot of position-related information in arrays rather than constants since they exist in relation to columns rather than groups. In addition, as we already know how to fill order records, let's track the allocation at each bar instead.

```
>>> @njit
... def pipeline_5_nb(
...     target_shape, ❶
...     group_lens, ❷
...     open,
...     close,
```

```

... target_pct, ③
... init_cash=100,
... auto_call_seq=True,
... rotate_cols=False
... ):
...     init_cash_ = vbt.to_1d_array_nb(np.asarray(init_cash))
...     open_ = vbt.to_2d_array_nb(np.asarray(open))
...     close_ = vbt.to_2d_array_nb(np.asarray(close))
...     target_pct_ = vbt.to_2d_array_nb(np.asarray(target_pct))
...     alloc = np.empty(target_shape, dtype=float_) ④
...
...     group_end_idx = np.cumsum(group_lens)
...     group_start_idx = group_end_idx - group_lens
...
...     for group in range(len(group_lens)): ⑤
...         group_len = group_lens[group]
...         from_col = group_start_idx[group]
...         to_col = group_end_idx[group]
...
...         ⑥
...         init_cash_elem = vbt.flex_select_1d_pc_nb(
...             init_cash_, group, rotate_cols=rotate_cols)
...
...         last_position = np.full(group_len, 0.0, dtype=float_) ⑦
...         last_debt = np.full(group_len, 0.0, dtype=float_)
...         last_locked_cash = np.full(group_len, 0.0, dtype=float_)
...         cash_now = float(init_cash_elem)
...         free_cash_now = float(init_cash_elem)
...
...         order_value = np.empty(group_len, dtype=float_) ⑧
...         call_seq = np.empty(group_len, dtype=int_)
...
...         for i in range(target_shape[0]): ⑨
...             ⑩
...             value_now = cash_now
...             for c in range(group_len):
...                 col = from_col + c
...                 open_elem = vbt.flex_select_nb(
...                     open_, i, col, rotate_cols=rotate_cols)
...                 value_now += last_position[c] * open_elem ⑪
...
...             ⑫
...             for c in range(group_len):
...                 col = from_col + c
...                 open_elem = vbt.flex_select_nb(
...                     open_, i, col, rotate_cols=rotate_cols)
...                 target_pct_elem = vbt.flex_select_nb(
...                     target_pct_, i, col, rotate_cols=rotate_cols)
...                 exec_state = vbt.pf_enums.ExecState(
...                     cash=cash_now,
...                     position=last_position[c],
...                     locked_cash=last_locked_cash[c],
...                     debt=last_debt[c],
...                     free_cash=free_cash_now,
...                     val_price=open_elem,
...                     value=value_now,
...                 )
...                 order_value[c] = vbt.pf_nb.approx_order_value_nb( ⑬
...                     exec_state=exec_state,
...                     size=target_pct_elem,
...                     size_type=vbt.pf_enums.SizeType.TargetPercent,
...                     direction=vbt.pf_enums.Direction.Both
...                 )
...                 call_seq[c] = c ⑭
...
...         if auto_call_seq:
...             vbt.pf_nb.insert_argsort_nb(order_value, call_seq) ⑮
...
...         for k in range(len(call_seq)): ⑯
...             c = call_seq[k] ⑰
...             col = from_col + c ⑱
...
...             open_elem = vbt.flex_select_nb(
...                 open_, i, col, rotate_cols=rotate_cols)
...             close_elem = vbt.flex_select_nb(
...                 close_, i, col, rotate_cols=rotate_cols)
...             target_pct_elem = vbt.flex_select_nb(
...                 target_pct_, i, col, rotate_cols=rotate_cols)
...
...             if not np.isnan(target_pct_elem): ⑲
...                 order = vbt.pf_nb.order_nb(
...                     size=target_pct_elem,
...                     price=close_elem,
...                     size_type=vbt.pf_enums.SizeType.TargetPercent,
...                     direction=vbt.pf_enums.Direction.Both
...                 )
...                 exec_state = vbt.pf_enums.ExecState(
...                     cash=cash_now,
...                     position=last_position[c],
...                     debt=last_debt[c],
...                     locked_cash=last_locked_cash[c],
...                     free_cash=free_cash_now,
...                     val_price=open_elem,
...                     value=value_now,
...                 )
...                 _, new_exec_state = vbt.pf_nb.process_order_nb(
...                     group=group,
...                     col=col,

```

```

...         i=i,
...         exec_state=exec_state,
...         order=order
...     )
...     cash_now = new_exec_state.cash
...     free_cash_now = new_exec_state.free_cash
...     value_now = new_exec_state.value
...     last_position[c] = new_exec_state.position
...     last_debt[c] = new_exec_state.debt
...     last_locked_cash[c] = new_exec_state.locked_cash
...
...     20
...     value_now = cash_now
...     for c in range(group_len):
...         col = from_col + c
...         close_elem = vbt.flex_select_nb(
...             close_, i, col, rotate_cols=rotate_cols)
...         value_now += last_position[c] * close_elem
...
...     21
...     for c in range(group_len):
...         col = from_col + c
...         close_elem = vbt.flex_select_nb(
...             close_, i, col, rotate_cols=rotate_cols)
...         alloc[i, col] = last_position[c] * close_elem / value_now
...
...     return alloc

```

- 1 Second number in the target shape tracks assets (x parameter combinations) - it doesn't track groups!
- 2 The group lengths array must have the same number of elements as we have groups, while the sum of this array must yield the number of columns in `target_shape`
- 3 Target allocations must be provided per asset, thus the array should broadcast against `target_shape`
- 4 Allocations must be filled per asset, thus the same shape as `target_shape`
- 5 Iterate over groups
- 6 Here comes the creation of various arrays that should exist only per group, such as the cash balance, position size per asset, and other state information. Remember that different groups represent independent, isolated tests, and shouldn't be connected by any means!
- 7 We can't create a single instance of `ExecState` like we did before because an order execution state contains information per asset, thus we need to keep track of its fields using separate variables (constants for data per group, arrays for data per asset)
- 8 We could have also created those two arrays at each bar, but frequent array creation slows down the simulation. A better practice is to create an array only once and re-fill it as many times as we want.
- 9 Iterate over time steps (bars) as we did in previous pipelines
- 10 Calculate the value of each group (= portfolio value) by iterating over the assets of the group and adding their position value to the current cash balance
- 11 Last position array exists only for this group, thus we're using `c`, not `col`!
- 12 Prepare the order value and call sequence arrays
- 13 Approximate the value of a potential order. We're at the beginning of the bar, thus we use the open price.
- 14 Write the current column index within this group. This will connect order values to column indices.
- 15 Sort both arrays in-place such that column indices associated with a lower order value appear first in the call sequence
- 16 Next we want to execute orders, hence iterate over the newly sorted call sequence
- 17 Get the associated column index within the group
- 18 Get the associated global column index
- 19 Perform the same logic as in the previous pipeline
- 20 Calculate the group value once again, but now using the updated position and the close price. Note that we are running this outside of the `np.nan` check since we want to track the allocation at each single bar.
- 21 Calculate the current allocation of each asset, and write it to the output array

Wow, this went complex really fast! 🤖

But it's not that complex as it may appear. We took a bunch of columns and split them into groups. Then, for each group, we defined a mini-pipeline that applies our logic on the columns within this group only, acting as a single portfolio unit. At the beginning of each bar, we calculate the portfolio value, and build a call sequence that re-arranges columns by their order value. We then iterate over this sequence and execute an order in each column. Finally, at the end of each bar, we again re-calculate the portfolio value and write the real allocation of each asset to the output array. The best in this pipeline is that it closely mimics how preset simulation methods work in vectorbt, and it's one of the most flexible pieces of code you can actually write!

Let's allocate 70% to BTC and 30% to ETH, and rebalance on a monthly basis:

```

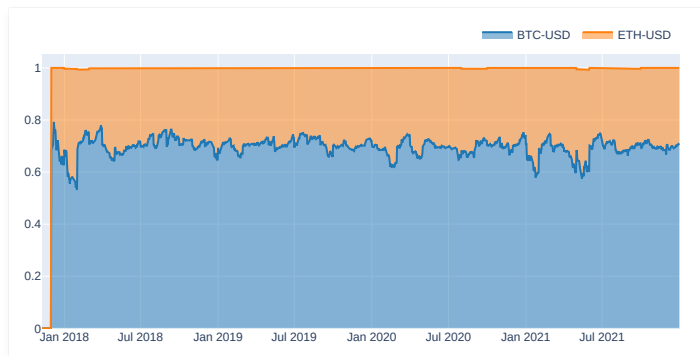
>>> mult_target_pct = mult_symbol_wrapper.fill()
>>> mult_target_pct.vbt.set([[0.7, 0.3]], every="M", inplace=True)
>>> grouper = vbt.Grouper(mult_symbol_wrapper.columns, group_by=True)
>>> group_lens = grouper.get_group_lens()

>>> target_shape = vbt.broadcast_shapes(
...     mult_data.get("Open").values.shape,
...     mult_data.get("Close").values.shape,
...     mult_target_pct.values.shape
... )
>>> target_shape

```

```
(1514, 2)

>>> alloc = pipeline_5_nb(
...     target_shape,
...     group_lens,
...     mult_data.get("Open").values,
...     mult_data.get("Close").values,
...     mult_target_pct.values
... )
>>> alloc = mult_symbol_wrapper.wrap(alloc)
>>> alloc.vbt.plot(
...     trace_kwargs=dict(stackgroup="one"),
...     use_gl=False
... ).show()
```

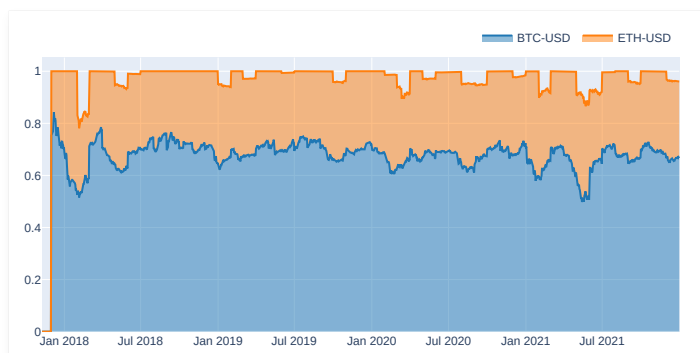


#### Info

As you might have noticed, some allocations do not quite sum to 100%. This is because we used the open price for group valuation and decision-making, while the actual orders were executed using the close price. By the way, it's a bad sign when everything aligns perfectly - this could mean that your simulation is too ideal for the real world.

And here's the same procedure but without sorting the call sequence array:

```
>>> alloc = pipeline_5_nb(
...     target_shape,
...     group_lens,
...     mult_data.get("Open").values,
...     mult_data.get("Close").values,
...     mult_target_pct.values,
...     auto_call_seq=False
... )
>>> alloc = mult_symbol_wrapper.wrap(alloc)
>>> alloc.vbt.plot(
...     trace_kwargs=dict(stackgroup="one"),
...     use_gl=False
... ).show()
```



As we see, some rebalancing steps couldn't be completed at all because long operations were executed before short operations, leaving them without the required funds.

The biggest advantage of this pipeline is in its flexibility: we can turn off grouping via `group_by=False` to run the entire logic per column (each group will contain only one column). We can also test multiple weight combinations via multiple groups, without having to tile the pricing data thanks to rotational indexing. This, for example, cannot be done even with [Portfolio.from\\_orders](#) 😞

```
>>> groups = pd.Index([0, 0, 1, 1], name="group")
>>> target_alloc = pd.Index([0.7, 0.3, 0.5, 0.5], name="target_alloc")

>>> final_columns = vbt.stack_indexes((
...     groups,
...     target_alloc,
...     vbt.tile_index(mult_symbol_wrapper.columns, 2)
... ))
>>> final_wrapper = mult_symbol_wrapper.replace(
...     columns=final_columns,
```



```

...     group_by="group"
... )
>>> mult_target_pct = final_wrapper.fill(group_by=False) ❸
>>> mult_target_pct.vbt.set(target_alloc.values[None], every="M", inplace=True)
>>> group_lens = final_wrapper.grouper.get_group_lens()

>>> n_groups = final_wrapper.grouper.get_group_count()
>>> target_shape = vbt.broadcast_shapes(
...     vbt.tile_shape(mult_data.get("Open").values.shape, n_groups), ❹
...     vbt.tile_shape(mult_data.get("Close").values.shape, n_groups),
...     mult_target_pct.values.shape
... )
>>> target_shape
(1514, 4)

>>> alloc = pipeline_5_nb(
...     target_shape,
...     group_lens,
...     mult_data.get("Open").values,
...     mult_data.get("Close").values,
...     mult_target_pct.values,
...     rotate_cols=True
... )
>>> alloc = mult_target_pct.vbt.wrapper.wrap(alloc)
>>> alloc
group
target_alloc      0.7      0.3      0.5      0.5
symbol
Date
2017-11-09 00:00:00+00:00  0.000000  0.000000  0.000000  0.000000
2017-11-10 00:00:00+00:00  0.000000  0.000000  0.000000  0.000000
2017-11-11 00:00:00+00:00  0.000000  0.000000  0.000000  0.000000
2017-11-12 00:00:00+00:00  0.000000  0.000000  0.000000  0.000000
2017-11-13 00:00:00+00:00  0.000000  0.000000  0.000000  0.000000
...
2021-12-27 00:00:00+00:00  0.703817  0.296183  0.504464  0.495536
2021-12-28 00:00:00+00:00  0.703452  0.296548  0.504026  0.495974
2021-12-29 00:00:00+00:00  0.708035  0.291965  0.509543  0.490457
2021-12-30 00:00:00+00:00  0.706467  0.293533  0.507650  0.492350
2021-12-31 00:00:00+00:00  0.704346  0.295654  0.505099  0.494901

[1514 rows x 4 columns]

```

- ❶ Build a new column hierarchy with three levels: groups, weights, and assets. Each level must have the same length.
- ❷ Create a new (grouped) wrapper with the new column hierarchy
- ❸ Create the target percentage array of the same shape as the new wrapper and fill with NaN
- ❹ We're using rotational indexing - tile the shapes of `open` and `close` by the number of groups for the shapes to broadcast nicely and build the target shape

## Contexts

Sometimes, there is a need to create a simulation method that takes a user-defined function and calls it to make some trading decision. Such a UDF would require access to the simulation's state (such as the current position size and direction) and other information, which could quickly involve dozens of variables.

Remember that we cannot do full-scale OOP in Numba, thus we have to pass data using primitive containers such as tuples. But usage of variable positional arguments or a regular tuple would be quite cumbersome for the user because accessing each field can only be done using an integer index or tuple unpacking. To ease this burden, we usually pass such information in form of a named tuple, often referred to as a (simulation) "context".

### Pipeline/6

Let's create a very basic pipeline that iterates over rows and columns, and, at each element, calls a UDF to get an order and execute it!

First, we need to answer the following question: "What information would a UDF need?" Mostly, we just include everything we have:

```

>>> SimContext = namedtuple("SimContext", [
...     "open", ❶
...     "high",
...     "low",
...     "close",
...     "init_cash",
...     "col", ❷
...     "i",
...     "price_area", ❸
...     "exec_state"
... ])

```

- ❶ Arguments passed to the simulator
- ❷ Loop variables
- ❸ State information, either unpacked (marginally faster) or in form of named tuples (more convenient)

And here's our pipeline that takes and calls an order function:

```

>>> @njit
... def pipeline_6_nb(
...     open, high, low, close,
...     order_func_nb, order_args=(),
...     init_cash=100
... )

```

```

... ):
...     order_records = np.empty(close.shape, dtype=vbt.pf_enums.order_dt)
...     order_counts = np.full(close.shape[1], 0, dtype=int_)
...
...     for col in range(close.shape[1]):
...         exec_state = vbt.pf_enums.ExecState(
...             cash=float(init_cash),
...             position=0.0,
...             debt=0.0,
...             locked_cash=0.0,
...             free_cash=float(init_cash),
...             val_price=np.nan,
...             value=np.nan
...         )
...
...         for i in range(close.shape[0]):
...             val_price = open[i, col]
...             value = exec_state.cash + val_price * exec_state.position
...
...             price_area = vbt.pf_enums.PriceArea(
...                 open[i, col],
...                 high[i, col],
...                 low[i, col],
...                 close[i, col]
...             )
...             exec_state = vbt.pf_enums.ExecState(
...                 cash=exec_state.cash,
...                 position=exec_state.position,
...                 debt=exec_state.debt,
...                 locked_cash=exec_state.locked_cash,
...                 free_cash=exec_state.free_cash,
...                 val_price=val_price,
...                 value=value
...             )
...             sim_ctx = SimContext( ❶
...                 open=open,
...                 high=high,
...                 low=low,
...                 close=close,
...                 init_cash=init_cash,
...                 col=col,
...                 i=i,
...                 price_area=price_area,
...                 exec_state=exec_state
...             )
...             order = order_func_nb(sim_ctx, *order_args) ❷
...             _, exec_state = vbt.pf_nb.process_order_nb(
...                 col, col, i,
...                 exec_state=exec_state,
...                 order=order,
...                 price_area=price_area,
...                 order_records=order_records,
...                 order_counts=order_counts
...             )
...
...     return vbt.nb.repartition_nb(order_records, order_counts)

```

- ❶ Initialize the simulation context (= creates a named tuple)
- ❷ Call the UDF by first passing the context and then any user-defined arguments

Let's write our own order function that generates orders based on signals:

```

>>> @njit ❶
... def signal_order_func_nb(c, entries, exits):
...     if entries[c.i, c.col] and c.exec_state.position == 0: ❷
...         return vbt.pf_nb.order_nb()
...     if exits[c.i, c.col] and c.exec_state.position > 0:
...         return vbt.pf_nb.close_position_nb()
...     return vbt.pf_nb.order_nothing_nb()

>>> broadcasted_args = vbt.broadcast( ❸
...     dict(
...         open=data.get("Open").values,
...         high=data.get("High").values,
...         low=data.get("Low").values,
...         close=data.get("Close").values,
...         entries=entries.values, ❹
...         exits=exits.values
...     ),
...     min_ndim=2
... )

>>> pipeline_6_nb(
...     broadcasted_args["open"],
...     broadcasted_args["high"],
...     broadcasted_args["low"],
...     broadcasted_args["close"],
...     signal_order_func_nb,
...     order_args=(
...         broadcasted_args["entries"],
...         broadcasted_args["exits"]
...     )
... )
array([( 0, 0, 300, 0.34786966, 287.46398926, 0., 0),
       ( 1, 0, 362, 0.34786966, 230.64399719, 0., 1),

```

```
( 2, 0, 406, 0.26339233, 304.61801147, 0., 0),
( 3, 0, 1290, 0.26339233, 6890.52001953, 0., 1),
( 4, 0, 1680, 0.33210511, 5464.86669922, 0., 0),
( 5, 0, 1865, 0.33210511, 9244.97265625, 0., 1),
( 6, 0, 1981, 0.31871477, 9633.38671875, 0., 0),
( 7, 0, 2016, 0.31871477, 6681.06298828, 0., 1),
( 8, 0, 2073, 0.2344648 , 9081.76171875, 0., 0),
( 9, 0, 2467, 0.2344648 , 35615.87109375, 0., 1),
(10, 0, 2555, 0.17333543, 48176.34765625, 0., 0)],
dtype={'names': ['id', 'col', 'idx', 'size', 'price', 'fees', 'side'], ...})
```

- ❶ Don't forget to decorate your order function with `@njit` as well!
- ❷ We can access any information similarly to accessing attributes of any Python object
- ❸ Neither prices nor signals utilize flexible indexing, thus we need to broadcast them to the full shape
- ❹ These two arrays were generated for the [first pipeline](#)

We just created our own shallow `Portfolio.from_order_func` functionality, neat! 🌟

Your homework is to extend this pipeline with flexible indexing 🤔

## Performance

In terms of performance, Numba code is often a roller coaster 🎢

Numba is a just-in-time (JIT) compiler that analyzes and optimizes code, and finally uses the [LLVM compiler library](#) to generate a machine code version of a Python function to be compiled. But sometimes, even if the function looks efficient on paper, Numba may generate a suboptimal machine code because of some variables or their types not interacting optimally. In such a case, the code may still run very fast compared to a similar implementation with Python or even to another JIT compiler, but there is a lot of space for improvement that may be hard to discover, even for experienced users. There are even cases where switching the lines in which variables are defined suddenly and unexpectedly has a negative/positive effect on performance.

Apart from [official tips](#), there are some of the best practices you should always keep in mind when designing and optimizing Numba-compiled functions:

1. Numba is perfectly happy with loops, and often even more happy than with vectorized operations. That's why 90% of vectorbt's functionality is enabled by loops.
2. Numba hates repeated creation of new arrays and allocating (even small) chunks of memory in loops. A much better idea is to create a bunch of bigger arrays prior to the iteration, and use them as a buffer for storing temporary information. Be aware that operations with NumPy that yield a new array, such as `np.cumsum`, create a new array!
3. Reading and writing one array element at a time is more efficient than in chunks
4. Basic math operations such as `-`, `+`, `*`, and `/` should be preferred to NumPy operations
5. When using a function as an argument to another function, arguments to this function should be accepted in a packed format (`*args`) instead of an unpacked format (`*args`). This rule is often violated by vectorbt itself, but such cases are usually benchmarked to ensure that performance stays the same.
6. Packing named tuples inside other named tuples (as we did above) is not encouraged, but sometimes there is no negative effect at all
7. NumPy arrays are almost always a better deal than lists and dictionaries!
8. Even if `fastmath=True` option has a positive impact on performance, be aware that it's associated with various [compromises](#) when doing numeric operations
9. Do not iterate over elements of an array, iterate over a range with the same length instead and use the loop variable to select the respective element
10. When overwriting a variable, make sure that it has the same type

### Hint

As a rule of thumb: the simpler is the code, the easier it becomes for Numba to analyze and optimize it.

## Benchmarking

To benchmark a simulator, we can use the [timeit](#) module. If possible, create some sample data of a sufficient size, and prepare for the worst-case scenario where orders are issued and executed at each single time step to benchmark the full load. Also, make sure to run tests all the way during the simulator's development to track the evolution of its execution time and stability.

### Note

Generation of sample data and preparation of other inputs must be done prior to benchmarking.

Let's generate 1-minute random OHLC data for one year using [RandomOHLCData](#):

```
>>> test_data = vbt.RandomOHLCData.pull(
...     start="2020-01-01",
...     end="2021-01-01",
...     timeframe="1min",
...     std=0.0001,
...     symmetric=True
... )
>>> test_data.resample("1d").plot().show()
```

- ❶ Set tick frequency to 1 minute

- ② Set tick volatility to 0.01%
- ③ Use symmetric returns (50% negative return == 100% positive return)
- ④ Plot to ensure that the generated data is realistic. Here we're resampling to daily frequency for faster plotting.



Then, we need to prepare all the data, which includes filling signals such that there is at least one order at each bar (our worst-case scenario for performance and memory):

```
>>> test_open = test_data.get("Open").values[:, None] ②
>>> test_high = test_data.get("High").values[:, None]
>>> test_low = test_data.get("Low").values[:, None]
>>> test_close = test_data.get("Close").values[:, None]
>>> test_entries = np.full(test_data.get_symbol_wrapper().shape, False)[: , None]
>>> test_exits = np.full(test_data.get_symbol_wrapper().shape, False)[: , None]
>>> test_entries[0::2] = True ③
>>> test_exits[1::2] = True ④
>>> del test_data ⑤

>>> test_entries.shape
(527041, 1)
```

- ① Generate random OHLC data with symmetric returns
- ② Get a column, extract the NumPy array, and expand the array to two dimensions (omit the last step if your data is already two-dimensional!)
- ③ Place an entry at each second bar starting from the first bar
- ④ Place an exit at each second bar starting from the second bar
- ⑤ Delete the data object to release memory

Each of the arrays is 527,041 data points long.

So, how is our simulator performing on this data?

```
>>> %timeit ①
>>> pipeline_6_nb(
...     test_open,
...     test_high,
...     test_low,
...     test_close,
...     signal_order_func_nb,
...     order_args=(
...         test_entries,
...         test_exits
...     )
... )
79.4 ms ± 290 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

- ① This magic command works only in a Jupyter environment and only if you place this command at the beginning of the cell. If you're running the code as a script, use the `timeit` module.

80 milliseconds to generate half a million orders on Apple M1, not bad! 🔥

To better illustrate how only a minor change can impact performance, we will create a new order function that also creates a zero-sized empty array:

```
>>> @njit
... def subopt_signal_order_func_nb(c, entries, exits):
...     _ = np.empty(0) ①
...
...     if entries[c.i, c.col] and c.exec_state.position == 0:
...         return vbt.pf_nb.order_nb()
...     if exits[c.i, c.col] and c.exec_state.position > 0:
...         return vbt.pf_nb.close_position_nb()
...     return vbt.pf_nb.order_nothing_nb()

>>> %timeit
>>> pipeline_6_nb(
...     test_open,
...     test_high,
...     test_low,
...     test_close,
...     subopt_signal_order_func_nb,
```

```
...     order_args=(
...         test_entries,
...         test_exits
...     )
... )
130 ms ± 675 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

#### 1 Here

As we see, creating an empty array at each bar has slowed down the execution by more than 50%. And this is a very important lesson to learn: create arrays outside of loops and only once!

### Auto-parallelization

Because of path dependencies (= the current state depends on the previous one), we cannot parallelize the loop that iterates over rows (= time). But here's the deal: since vectorbt allows us to define a multi-columnar backtesting logic, we can parallelize the loop that iterates over columns or groups of columns, given that those columns or groups of columns are independent of each other - all using Numba alone. By the way, this is one of the primary reasons why vectorbt loves two-dimensional data layouts so much.

Automatic parallelization with Numba cannot be simpler: just replace `range` that you want to parallelize with `numba.prange`, and instruct Numba to parallelize the function by passing `parallel=True` to the `@njit` decorator. This will (try to) execute the code in the loop simultaneously by multiple parallel threads. You can read more about automatic parallelization with Numba [here](#) and about the available threading layers [here](#). On MacBook Air (M1, 2020), turning on parallelization reduces the processing time by 2-3 times on average. Usually, a simple arithmetic-heavy code without creating any arrays can be better parallelized than a complex vectorization-heavy code.

#### Important

You can modify the same array from multiple threads, as done by countless functions in vectorbt. Just make sure that multiple threads (columns, in our case) aren't modifying the same elements and data in general!

Here's a small example of a function that computes the expanding maximum on two-dimensional data, without and with automatic parallelization:

```
>>> arr = np.random.uniform(size=(1000000, 10))

>>> @njit
... def expanding_max_nb(arr):
...     out = np.empty_like(arr, dtype=float_)
...     for col in range(arr.shape[1]):
...         maxv = -np.inf
...         for i in range(arr.shape[0]):
...             if arr[i, col] > maxv:
...                 maxv = arr[i, col]
...             out[i, col] = maxv
...     return out

>>> %timeit expanding_max_nb(arr)
40.7 ms ± 558 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

>>> @njit(parallel=True) 1
... def parallel_expanding_max_nb(arr):
...     out = np.empty_like(arr, dtype=float_)
...     for col in prange(arr.shape[1]): 2
...         maxv = -np.inf
...         for i in range(arr.shape[0]):
...             if arr[i, col] > maxv:
...                 maxv = arr[i, col]
...             out[i, col] = maxv
...     return out

>>> %timeit parallel_expanding_max_nb(arr)
26.6 ms ± 437 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

#### 1 Here's the first change

#### 2 Here's the second change

It's your turn: enable automatic parallelization of columns in the [sixth pipeline](#) and benchmark it! Just don't forget to reduce the number of rows and increase the number of columns.

### Caching

Even if we had optimized the simulation pipeline for the best-possible performance, the actual compilation step would take a huge chunk of that time savings away. However, the good news is that Numba doesn't have to re-compile the function the second time it's executed, given that we passed the same argument **types** (not data!). This means that we need to wait only once if we want to test the same function on many parameter combinations, at the same Python runtime. Sadly, if only one argument differs in type, or we've restarted the Python runtime, Numba has to compile again.

But luckily, Numba gives us a mechanism to avoid re-compilation even if we've restarted the runtime, called **caching**. To enable caching, just pass `cache=True` to the `@njit` decorator.

**Important**

Avoid turning on caching for functions that take complex, user-defined data, such as (named) tuples and other functions. This may lead to some hidden bugs and kernel crashes if the data changes during the next runtime. Also make sure that your function doesn't use global variables. For example, the [fifth pipeline](#) is perfectly cacheable, while the [sixth pipeline](#) is not cacheable, or maybe could be if `order_func_nb` was cacheable as well.

Make sure to define any cached function inside a Python file rather than in a notebook cell since it must have a clear filepath such that it can be introspected by Numba. To invalidate the cache, go to the directory where the function resides and remove the `__pycache__` directory. You can do that by running `rm -rf __pycache__` from your terminal.

**Hint**

A good practice is to invalidate the cache every time you change the code of a cached function to avoid potential side effects. Also, disable caching for the entire time of developing a function and turn it only on once the function has been fully implemented.

## AOT compilation

Using [ahead-of-time compilation](#), we can compile a function only once and get no compilation overhead at runtime. Although this feature of Numba isn't widely used in vectorbt because it would restrict us from passing input data flexibly, we can make use of it in cases where we know the argument types in advance. Let's pre-compile our [fifth pipeline](#)!

For this, we have to specify the signature of a function explicitly. You can read more about it in the [types](#) reference.

```
>>> from numba.pycc import CC
>>> cc = CC('pipeline_5') ①

>>> sig = "f8[:, :](" \ ②
...       "UniTuple(i8, 2), " \ ③
...       "i8[:, :], " \ ④
...       "f8[:, :], " \ ⑤
...       "f8[:, :], " \
...       "f8[:, :], " \
...       "f8[:, :], " \
...       "b1, " \ ⑦
...       "b1" \
...       ")"

>>> cc.export('pipeline_5_nb', sig)(pipeline_5_nb) ⑧
>>> cc.compile() ⑨
```

- ① Initialize a new module
- ② Function should return a two-dimensional array of 64-bit floating-point data type (allocations)
- ③ Tuple with two 64-bit integers ( `target_shape` )
- ④ One-dimensional array of 64-bit integer data type ( `group_lens` )
- ⑤ Two-dimensional array of 64-bit floating-point data type ( `open`, `close`, and `target_pct` )
- ⑥ One-dimensional array of 64-bit floating-point data type ( `init_cash` )
- ⑦ Boolean value ( `auto_call_seq` and `rotate_cols` )
- ⑧ Register the function with the provided signature. You can bind multiple signatures to the same function.
- ⑨ Compile the module

This has generated an extension module named `pipeline_5`. On macOS, the actual filename is `pipeline_5.cpython-37m-darwin.so`. We can then import the module like a regular Python module and run the function `pipeline_5_nb` of that module:

```
>>> import pipeline_5

>>> mult_alloc = pipeline_5.pipeline_5_nb(
...     target_shape,
...     group_lens,
...     vbt.to_2d_array(mult_data.get("Open")).astype("f8", copy=False), ①
...     vbt.to_2d_array(mult_data.get("Close")).astype("f8", copy=False),
...     vbt.to_2d_array(mult_target_pct).astype("f8", copy=False),
...     vbt.to_1d_array(100).astype("f8", copy=False),
...     auto_call_seq=True, ②
...     rotate_cols=True
... )
```

- ① This makes sure that the array has a proper dimensionality and data type
- ② Keyword arguments must be provided as positional ones. Also, default values must be provided explicitly.

That was lightning fast! ⚡

**Important**

You should ensure that the provided arguments exactly match the registered signature, otherwise you may get errors that may be very difficult to debug. For example, while setting `init_cash` to `100` would yield an `"index is out of bounds"` error, casting the array to integer would make all allocations zero!

## Summary

We've covered in detail a lot of components of a typical simulator in vectorbt. Simulation is the primary step in backtesting of a trading strategy, and by mastering it you'll gain some hard-core skills that can be applied just in any place of the vectorbt's rich Numba ecosystem.

One of the most important takeaways from this documentation piece is that implementing a custom simulator is as easy (or as difficult) as any other Numba-compiled function, and there is no point in using the preset simulation methods such as [Portfolio.from\\_signals](#) if you can produce the same results, achieve a multifold performance gain, be able to use rotational indexing, caching, and AOT compilation, by designing your own pipeline from scratch. After all, it's just a bunch of loops that gradually move over the shape of a matrix, execute orders, update the state of the simulation, and write some output data. Everything else is up to your imagination 🚀

 [Python code](#)

## From orders

Instead of building our custom simulator from scratch, we can make use of one of the preset simulation methods offered by vectorbt. There are three predominantly used methods: `Portfolio.from_orders`, `Portfolio.from_signals`, and `Portfolio.from_order_func`. Each one has its own benefits and drawbacks, but all of them follow the same iteration schema that we discussed previously: iterate over the rows and columns, and at each step, convert the current element of all the input data passed by the user into an order request, and process it by updating the current simulation state and by appending the filled order record to an array. This array along with other information can later be used during the reconstruction phase to analyze the simulation.

`Portfolio.from_orders` is the most basic method out of three: it doesn't take any UDFs, and allows us to provide every bit of information on orders as separate, broadcastable arrays. Literally **each** element across all the passed arrays will be converted into an instance of `Order` and processed as usual. Since the number of orders is limited by the number of elements in the passed arrays, we can issue **and execute** only one order per timestamp and asset.

For example, passing `[0.1, -0.1, np.nan]` as the order size array and `[11, 10, 12]` as the order price array will generate three orders at three subsequent timestamps:

1. Buy 0.1 shares for \$11
2. Sell 0.1 shares for \$10
3. Do nothing (size of NaN gets ignored)

### Hint

This method should be used when you know exactly what to order at each timestamp. And, as practice shows, many types of signals and other inputs can be successfully converted into an order format for a nice speedup. For example, if your entries and exits are cleaned (i.e., one exit comes exactly after one entry and vice versa), you can convert them to order size using `entries.astype(int) - exits.astype(int)`, which will order 1 share once a signal is encountered, in both directions.

## Numba

The backbone of this method is the Numba-compiled function `from_orders_nb`, which is the fastest simulation function out of three, but also the only one that can be (and is) safely cached in Numba because it doesn't depend on any complex or global data. It's also using flexible indexing, it's registered as a chunkable function, and it can be parallelized by Numba with a single command.

The arguments of this simulation function include some arguments we've encountered in the documentation on simulation: target shape, group lengths, initial cash, and some others. The arguments we haven't seen before include the call sequence array, initial state arrays (such as `init_position`), continuous state change arrays (such as `cash_deposits`), order information arrays (such as `size`), and various flags for controlling the simulation process (such as `save_returns`). Also, most arguments have sensitive default values that are consistent across most Numba-compiled functions defined in `portfolio.nb`.

### Order fields

Every single field in the named tuple `Order` is present as an argument in the signature of this simulation function, and can be provided in a format suitable for flexible indexing.

Let's simulate the three orders we mentioned above:

```
>>> from vectorbtpro import *

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(3, 1), ①
...     group_lens=np.array([1]), ②
...     size=np.array([[0.1], [-0.1], [np.nan]]), ③
...     price=np.array([[11], [10], [12]])
... )
>>> sim_out.order_records
array([(0, 0, 0, 0.1, 11., 0., 0), (1, 0, 1, 0.1, 10., 0., 1)],
      dtype={'names': ['id', 'col', 'idx', 'size', 'price', 'fees', 'side'], ...})
```

- ① Target shape must be two-dimensional, 3 and (3,) are not allowed
- ② We must provide a one-dimensional group lengths array, which without column grouping is just an array of ones (one group per column)
- ③ Make size and price arrays of the same shape as `target_shape` for now

The result of the simulation is an instance of the named tuple `SimulationOutput`, which consists of the filled order and log records, and other supporting information for post-analysis:



```
>>> print(vbt.prettify(sim_out))
SimulationOutput(
  order_records=<numpy.ndarray object at 0x7f88606d5710 of shape (2,)>,
  log_records=<numpy.ndarray object at 0x7f8860907fa8 of shape (0,)>,
  cash_deposits=<numpy.ndarray object at 0x7f8860907f50 of shape (1, 1)>,
  cash_earnings=<numpy.ndarray object at 0x7f8860a355b0 of shape (1, 1)>,
  call_seq=None,
  in_outputs=FSInOutputs(
    returns=<numpy.ndarray object at 0x7f88a1976fa8 of shape (0, 0)>
  )
)
```

#### Info

Despite the fact that we didn't instruct vectorbt to create an array with log records, cash deposits, and cash earnings, we still see those arrays in the simulation output. Because Numba has difficulties in processing optional writable arrays, we cannot make those outputs `None`, but instead, we create empty arrays with ultra-small shapes to denote that they should be ignored during post-analysis.

Here's a basic helper function to pretty-print order records:

```
>>> def print_orders(target_shape, order_records):
...     wrapper = vbt.ArrayWrapper.from_shape(target_shape)
...     print(vbt.Orders(wrapper, order_records).readable)

>>> print_orders((3, 1), sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0      0      0          0  0.1  11.0  0.0  Buy
1      1      0          1  0.1  10.0  0.0  Sell
```

To apply any information on **every** element, we can provide a scalar. This works because most simulation methods convert any scalar into a two-dimensional array suitable for **flexible indexing** automatically! We can also provide any information that should be applied per row, such as price, as a one-dimensional array, like this:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(3, 1),
...     group_lens=np.array([1]),
...     size=np.array([0.1, -0.1, np.nan]),
...     price=np.array([11, 10, 12]),
...     fees=0.01
... )
>>> print_orders((3, 1), sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0      0      0          0  0.1  11.0  0.011 Buy
1      1      0          1  0.1  10.0  0.010 Sell
```

1. Apply 1% commission on all orders

#### Important

Default broadcasting rules of vectorbt are slightly different from the NumPy broadcasting rules: in NumPy, `(3,)` will broadcast along columns in the shape `(3, 1)`, but vectorbt assumes that flexible two-dimensional arrays are always time series and thus are provided per row.

If we want to provide any information per column instead, we always need wrap it into a two-dimensional array. We can additionally make this array have only one row to apply information to all rows. Let's test multiple size values by providing them per column, while the price is provided per row:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(3, 3), ①
...     group_lens=np.array([1, 1, 1]), ②
...     size=np.array([[np.inf, np.nan, -np.inf]]), ③
...     price=np.array([11, 10, 12]), ④
...     fees=0.01 ⑤
... )
>>> print_orders((3, 3), sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0      0      0          0  9.0009 11.0 0.990099 Buy
1      0      2          0  9.0009 11.0 0.990099 Sell
```

- ① Shape `(n, m)` resulting from imaginary broadcasting all the passed arrays
- ② Three groups, one per column
- ③ Arrays with the shape `(1, m)` broadcast along columns. Size of `np.inf` will buy as much as possible at every timestamp, size of `np.nan` will be ignored at every timestamp, and size of `-np.inf` will short sell as much as possible at every timestamp.
- ④ Arrays with the shape `(n,)` or `(n, 1)` broadcast along rows
- ⑤ Scalars and arrays with the shape `(1,)` or `(1, 1)` broadcast along columns and rows

This has the same effect as if we broadcasted all the arrays prior to passing.

```
>>> size, price, fees = vbt.broadcast_arrays( ①
...     np.array([[np.inf, np.nan, -np.inf]]),
...     np.array([11, 10, 12]),
...     0.01
... )
>>> size
array([[ inf,  nan, -inf],
       [ inf,  nan, -inf],
```

```
[ inf, nan, -inf]])

>>> price
array([[11, 11, 11],
       [10, 10, 10],
       [12, 12, 12]])

>>> fees
array([[0.01, 0.01, 0.01],
       [0.01, 0.01, 0.01],
       [0.01, 0.01, 0.01]])

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=size.shape, ❶
...     group_lens=np.full(size.shape[1], 1),
...     size=size,
...     price=price,
...     fees=fees
... )
>>> print_orders(size.shape, sim_out.order_records)
   Order Id  Column  Timestamp  Size  Price  Fees  Side
0         0         0           0  9.0009  11.0  0.990099  Buy
1         0         2           0  9.0009  11.0  0.990099  Sell
```

❶ Use `broadcast_arrays`

❷ We can use the shape of any broadcasted array since they all now have the same size

Notice how all the arrays have the same shape `(3, 3)`, which becomes our target shape.

#### Important

We cannot use `np.broadcast_arrays` since one-dimensional arrays will broadcast along columns.

#### Info

Visit the API documentation of `Order` to learn more about different arguments and their meanings.

## Grouping

If any element in the array `group_lens` is greater than 1, it's assumed that the columns are grouped and cash sharing is enabled by default. Let's simulate a portfolio with two assets where we try to order as many shares as possible in both assets:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(1, 2), ❶
...     group_lens=np.array([2]), ❷
...     size=np.array([[np.inf, np.inf]]), ❸
...     price=np.array([[10, 5]])
... )
>>> print_orders((1, 2), sim_out.order_records)
   Order Id  Column  Timestamp  Size  Price  Fees  Side
0         0         0           0  10.0  10.0  0.0  Buy
```

❶ One row (bar) and two columns (assets)

❷ Build a group with cash sharing consisting of two columns

❸ Provide order information per element by creating an array of the same shape as `target_shape`

We see that the first asset used up the entire cash and left the second one without funds.

## Call sequence

One practical feature of `from_orders_nb` is that we can provide our own call sequence to change the order in which columns are processed inside their groups. Without providing a call sequence, the default processing order is *from left to right*. For example, let's perform two rebalancing steps with the default call sequence: allocate 100% to the second asset at the first timestamp, and close out the position and allocate 100% to the first asset at the second timestamp.

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(2, 2),
...     group_lens=np.array([2]),
...     size=np.array([[0, 1], [1, 0]]),
...     size_type=vbt.pf_enums.SizeType.TargetPercent,
...     price=np.array([[10, 5], [11, 4]])
... )
>>> print_orders((2, 2), sim_out.order_records)
   Order Id  Column  Timestamp  Size  Price  Fees  Side
0         0         1           0  20.0  5.0  0.0  Buy
1         1         1           1  20.0  4.0  0.0  Sell
```

We see that only the column `1` has been processed. This is because we attempted to allocate 100% to the first asset without first closing out the position of the second asset at the second timestamp. To account for this order, we can pass our own call sequence:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(2, 2),
...     group_lens=np.array([2]),
...     size=np.array([[0, 1], [1, 0]]),
```

```
...     size_type=vbt.pf_enums.SizeType.TargetPercent,
...     price=np.array([[10, 5], [11, 4]]),
...     call_seq=np.array([[0, 1], [1, 0]]) ❶
... )
>>> print_orders((2, 2), sim_out.order_records)
Order Id Column Timestamp      Size Price Fees Side
0         0      0          1  7.272727 11.0  0.0 Buy
1         0      1          0 20.000000  5.0  0.0 Buy
2         1      1          1 20.000000  4.0  0.0 Sell
```

- ❶ First timestamp: first process the first asset then the second. Second timestamp: first process the second asset then the first. Note that a call sequence must provide exactly one position per timestamp and asset, thus its shape must follow `target_shape`.

#### Info

Order records are partitioned by column and are guaranteed to come in the order they were filled in each column. Also, order ids (the first field) are generated per column, not globally!

The first asset at the second timestamp now has the required funds to go long.

To avoid providing the call sequence manually, we can leave it at `None` and enable `auto_call_seq` instead. In such a case, vectorbt will automatically sort the columns by their approximated order value:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(2, 2),
...     group_lens=np.array([2]),
...     size=np.array([[0, 1], [1, 0]]),
...     size_type=vbt.pf_enums.SizeType.TargetPercent,
...     price=np.array([[10, 5], [11, 4]]),
...     auto_call_seq=True ❶
... )
>>> print_orders((2, 2), sim_out.order_records)
Order Id Column Timestamp      Size Price Fees Side
0         0      0          1  7.272727 11.0  0.0 Buy
1         0      1          0 20.000000  5.0  0.0 Buy
2         1      1          1 20.000000  4.0  0.0 Sell
```

#### Here

Sometimes though, we would like to throw a look at the sequence in which orders were processed, which isn't reflected in the order records. For this, we can provide our own call sequence for vectorbt to change it in-place and return it as a field of the **SimulationOutput** named tuple. The call sequence array must be pre-filled with indices in the strict order from 0 to `n`, where `n` is the length of the particular group of columns. We can easily build such an array using the function `build_call_seq`, which takes the target shape, the group lengths, and the call sequence type from **CallSeqType**:

```
>>> from vectorbtpro.portfolio.call_seq import build_call_seq

>>> call_seq = build_call_seq(
...     target_shape=(2, 2),
...     group_lens=np.array([2]),
...     call_seq_type=vbt.pf_enums.CallSeqType.Default ❶
... )

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(2, 2),
...     group_lens=np.array([2]),
...     size=np.array([[0, 1], [1, 0]]),
...     size_type=vbt.pf_enums.SizeType.TargetPercent,
...     price=np.array([[10, 5], [11, 4]]),
...     call_seq=call_seq,
...     auto_call_seq=True
... )
>>> sim_out.call_seq
array([[0, 1],
       [1, 0]])
```

- ❶ If we want for vectorbt to sort the call sequence using `auto_call_seq`, it must be of the type `Default`.

The generated and then sorted call sequence exactly follows the correct one we constructed manually before.

#### Hint

There is usually no big reason in providing your own call sequence. Setting it to `None` (default) and turning on the flag `auto_call_seq` will derive the best call sequence in the most resource-friendly manner.

## Filling returns

In addition to returning order records, we can instruct vectorbt to also fill the return based on the portfolio value at the end of each bar. The constructed series will be available under the field `returns` of **SimulationOutput.in\_outputs**, and can be used to calculate a range of metrics, such as the Sharpe ratio. Let's do the following: pull the BTC price and calculate the return of a simple buy-and-hold strategy. We'll see that the filled returns will closely match the returns calculated from the price directly:

```
>>> data = vbt.YFData.pull("BTC-USD", end="2022-01-01")
>>> symbol_wrapper = data.get_symbol_wrapper()

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=symbol_wrapper.shape_2d, ❶
```

```

...     group_lens=np.array([1]),
...     open=data.get("Open").values, ❷
...     high=data.get("High").values,
...     low=data.get("Low").values,
...     close=data.get("Close").values,
...     save_returns=True ❸
... )
>>> returns = symbol_wrapper.wrap(sim_out.in_outputs.returns) ❹
>>> returns
Date
2014-09-17 00:00:00+00:00    0.000000
2014-09-18 00:00:00+00:00   -0.071926
2014-09-19 00:00:00+00:00   -0.069843
...
2021-12-29 00:00:00+00:00   -0.024042
2021-12-30 00:00:00+00:00    0.015791
2021-12-31 00:00:00+00:00   -0.018476
Freq: D, Name: BTC-USD, Length: 2663, dtype: float64

>>> data.get("Close").vbt.to_returns() ❺
Date
2014-09-17 00:00:00+00:00    0.000000
2014-09-18 00:00:00+00:00   -0.071926
2014-09-19 00:00:00+00:00   -0.069843
...
2021-12-29 00:00:00+00:00   -0.024042
2021-12-30 00:00:00+00:00    0.015791
2021-12-31 00:00:00+00:00   -0.018476
Freq: D, Name: Close, Length: 2663, dtype: float64

```

- ❶ Use the shape of the data as the target shape. Make sure that the columns are symbols and the shape is two-dimensional.
- ❷ Pass OHLC data as separate arrays. Passing the open, high, and low price is optional.
- ❸ Tell vectorbt to calculate the portfolio value and return at each timestamp
- ❹ Extract the return series and convert it into a Pandas object
- ❺ Buy-and-hold produces the same returns as the close price itself

Returns are calculated from the value of each group, thus the number of columns in the returned time series matches the number of groups in `group_lens`:

```

>>> mult_data = vbt.YFData.pull( ❶
...     ["BTC-USD", "ETH-USD"],
...     end="2022-01-01",
...     missing_index="drop"
... )
>>> mult_symbol_wrapper = mult_data.get_symbol_wrapper()

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=mult_symbol_wrapper.shape_2d,
...     group_lens=np.array([2]), ❷
...     close=mult_data.get("Close").values,
...     size=np.array([[0.5, 0.5]]), ❸
...     size_type=vbt.pf_enums.SizeType.TargetPercent,
...     save_returns=True
... )
>>> returns = mult_symbol_wrapper\
...     .replace(columns=["group"], ndim=1)\
...     .wrap(sim_out.in_outputs.returns) ❹
>>> returns
Date
2017-11-09 00:00:00+00:00    0.000000
2017-11-10 00:00:00+00:00   -0.070482
2017-11-11 00:00:00+00:00    0.006159
...
2021-12-29 00:00:00+00:00   -0.034684
2021-12-30 00:00:00+00:00    0.019652
2021-12-31 00:00:00+00:00   -0.013406
Freq: D, Name: group, Length: 1514, dtype: float64

```

- ❶ Fetch two columns (assets) of data
- ❷ Build a single group out of two columns
- ❸ Define an allocation per column, such that the array nicely broadcasts against the target shape
- ❹ Our wrapper holds metadata per column while the returned array is per group - replace the columns with the single group we created above

## Initial state

Initial state defines the starting conditions of the entire trading environment. It mainly consists of three variables: the initial cash, the initial position, and the average entry price of the initial position. Each variable has at most one dimension and is either defined per column or per group.

The initial cash is the most important variable out of three, and can be provided as a flexible array `init_cash` with values per column or per group with cash sharing. As a rule of thumb: it must be either a scalar or one-dimensional array with the same number of elements as there are in `group_lens`.

Let's create a group of two columns and allocate 50% to each column, and another two groups with one column and 100% allocation each. This way, we can perform two independent backtests: with grouping and without. By supplying the initial cash as a single number (\$100 by default), the first group will split it among two assets, and thus the starting condition of the grouped columns will differ from that of the columns without grouping, which isn't ideal if you want to set up fair statistical experiments.

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(1, 4),
...     group_lens=np.array([2, 1, 1]),
...     init_cash=100, ①
...     size=np.array([[0.5, 0.5, 1.0, 1.0]]), ②
...     size_type=vbt.pf_enums.SizeType.TargetPercent,
...     price=np.array([[10, 11, 10, 11]]),
... )
>>> print_orders((1, 4), sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0 0 0 0 5.000000 10.0 0.0 Buy
1 0 1 0 4.545455 11.0 0.0 Buy
2 0 2 0 10.000000 10.0 0.0 Buy
3 0 3 0 9.090909 11.0 0.0 Buy
```

- ① Make \$100 the starting cash of all three groups
- ② Evenly split the cash among the two columns in the first group. Remember that size and other order information must be provided per column (asset), not per group!

Let's fix this and provide the first group with twice as much capital:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=(1, 4),
...     group_lens=np.array([2, 1, 1]),
...     init_cash=np.array([200, 100, 100]), ①
...     size=np.array([[0.5, 0.5, 1.0, 1.0]]),
...     size_type=vbt.pf_enums.SizeType.TargetPercent,
...     price=np.array([[10, 11, 10, 11]]),
... )
>>> print_orders((1, 4), sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0 0 0 0 10.000000 10.0 0.0 Buy
1 0 1 0 9.090909 11.0 0.0 Buy
2 0 2 0 10.000000 10.0 0.0 Buy
3 0 3 0 9.090909 11.0 0.0 Buy
```

- ① Provide the initial cash as an array with values per group

Apart from the initial cash, we can also specify the initial position of each asset to start with. Let's start the simulation with 1 BTC and 1 ETH, and calculate the returns:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=mult_symbol_wrapper.shape_2d,
...     group_lens=np.array([1, 1]),
...     init_position=np.array([1, 1]), ①
...     close=mult_data.get("Close").values,
...     save_returns=True
... )
>>> returns = mult_symbol_wrapper.wrap(sim_out.in_outputs.returns)
>>> returns
symbol          BTC-USD   ETH-USD
Date
2017-11-09 00:00:00+00:00      NaN      NaN
2017-11-10 00:00:00+00:00 -0.073554 -0.067411
2017-11-11 00:00:00+00:00 -0.039368 0.051555
...
2021-12-29 00:00:00+00:00 -0.024042 -0.045348
2021-12-30 00:00:00+00:00 0.015791 0.023514
2021-12-31 00:00:00+00:00 -0.018476 -0.008406

[1514 rows x 2 columns]
```

- ① Initial position must be provided per column (asset)

The first data point is NaN because vectorbt cannot calculate the initial value of each portfolio instance without knowing the entry price of each initial position.

Let's fix this by setting the entry price to the first open price:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=mult_symbol_wrapper.shape_2d,
...     group_lens=np.array([1, 1]),
...     init_position=np.array([1, 1]),
...     init_price=mult_data.get("Open").values[0], ①
...     close=mult_data.get("Close").values,
...     save_returns=True
... )
>>> returns = mult_symbol_wrapper.wrap(sim_out.in_outputs.returns)
>>> returns
symbol          BTC-USD   ETH-USD
Date
2017-11-09 00:00:00+00:00 -0.040182 0.029950
2017-11-10 00:00:00+00:00 -0.073554 -0.067411
2017-11-11 00:00:00+00:00 -0.039368 0.051555
...
2021-12-29 00:00:00+00:00 -0.024042 -0.045348
2021-12-30 00:00:00+00:00 0.015791 0.023514
2021-12-31 00:00:00+00:00 -0.018476 -0.008406

[1514 rows x 2 columns]
```

- ① Entry price of the initial position must also be provided per column (asset)

**Important**

Make sure to distinguish between a column and a group!

Columns represent individual assets, and most information in vectorbt must be supplied at this (lowest) level of granularity. Such arrays must broadcast against `target_shape`. Groups, on the other hand, represent collections of assets that share the same cash, and only some variables related to cash must be provided per group. Such arrays must broadcast against `group_lens.shape`.

**Hint**

If you're not sure whether an argument is expected as a one-dimensional or two-dimensional array, take a look into the source code of the function: one-dimensional arrays are annotated with `FlexArray1dLike` and two-dimensional with `FlexArray2dLike`. If an argument is expected as strictly one-dimensional or two-dimensional array (i.e., a scalar is not allowed), it's annotated with `FlexArray1d` and `FlexArray2d` respectively. If an argument is not flexible at all (i.e., it must have the same shape as `target_shape`), it will be just `Array1d` and `Array2d` respectively.

## Cash deposits

In addition to providing the initial cash, we can also deposit or withdraw arbitrary cash amounts as the simulation progresses. Similar to `init_cash`, the argument `cash_deposits` must be specified per group. The only difference is that the array now should broadcast along rows and columns, and specify the deposited or withdrawn amount at each timestamp. Thanks to flexible indexing, we can specify the amount to be applied per each element, row, group, or to the entire frame. The actual operation then takes place at the beginning of each bar.

Let's simulate a simple **DCA** strategy where we deposit \$100 at the beginning of each year and spend it right away:

```
>>> cash_deposits = symbol_wrapper.fill(0) ❶
>>> cash_deposits.vbt.set(100, every="Y", inplace=True)

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=symbol_wrapper.shape_2d,
...     group_lens=np.array([1]),
...     cash_deposits=cash_deposits.values,
...     close=data.get("Close").values
... )
>>> print_orders(symbol_wrapper.shape_2d, sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0 0 0 0.218659 457.334015 0.0 Buy
1 1 0 106 0.318219 314.248993 0.0 Buy
2 2 0 471 0.230238 434.334015 0.0 Buy
3 3 0 837 0.100168 998.325012 0.0 Buy
4 4 0 1202 0.007322 13657.200195 0.0 Buy
5 5 0 1567 0.026018 3843.520020 0.0 Buy
6 6 0 1932 0.013889 7200.174316 0.0 Buy
7 7 0 2298 0.003404 29374.152344 0.0 Buy
```

- ❶ Create an array with the same shape as our data, fill it with zeros, and set the elements at an annual frequency to 100

Below, we're doing the same but on a group with two assets and equal allocations. Since the array `cash_deposits` changes the cash balance, it must have the same number of columns as we have groups (with cash sharing).

```
>>> cash_deposits = mult_symbol_wrapper\
...     .replace(columns=["group"], ndim=1)\
...     .fill(0) ❶
>>> cash_deposits.vbt.set(100, every="Y", inplace=True)
>>> size = mult_symbol_wrapper.fill(np.nan) ❷
>>> size.vbt.set(0.5, every="Y", inplace=True)
>>> size.iloc[0] = 0.5 ❸

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=mult_symbol_wrapper.shape_2d,
...     group_lens=np.array([2]),
...     cash_deposits=cash_deposits.values,
...     close=mult_data.get("Close").values,
...     size=size.values,
...     size_type=vbt.pf_enums.SizeType.TargetPercent
... )
>>> print_orders(mult_symbol_wrapper.shape_2d, sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0 0 0 0.006999 7143.580078 0.0 Buy
1 1 0 53 0.004569 13657.200195 0.0 Buy
2 2 0 418 0.010971 3843.520020 0.0 Buy
3 3 0 783 0.001263 7200.174316 0.0 Buy
4 4 0 1149 0.003404 29374.152344 0.0 Buy
5 0 1 0 0.155820 320.884003 0.0 Buy
6 1 1 53 0.048663 772.640991 0.0 Buy
7 2 1 418 0.410697 140.819412 0.0 Buy
8 3 1 783 0.695013 130.802002 0.0 Buy
9 4 1 1149 0.108007 730.367554 0.0 Sell
```

- ❶ Create an array with one column for our group
- ❷ Create a size array to allocate 50% of the deposited cash to each asset
- ❸ We have also initial cash, thus allocate at the first timestamp too

To withdraw cash, we need to provide a negative amount. If the amount of cash to be withdrawn exceeds the amount of cash we have in our account, only the available cash will be withdrawn. Let's start with 1 BTC, sell 10% each year, and continuously withdraw the entire cash balance:

```
>>> size = symbol_wrapper.fill(np.nan) ❶
>>> size.vbt.set(-0.1, every="Y", inplace=True)
```

```

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=symbol_wrapper.shape_2d,
...     group_lens=np.array([1]),
...     init_position=1,
...     cash_deposits=-np.inf, ❶
...     close=data.get("Close").values,
...     size=size.values,
...     size_type=vbt.pf_enums.SizeType.Percent, ❷
...     direction=vbt.pf_enums.Direction.LongOnly ❸
... )
>>> print_orders(symbol_wrapper.shape_2d, sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0 0 0 106 0.100000 314.248993 0.0 Sell
1 1 0 471 0.090000 434.334015 0.0 Sell
2 2 0 837 0.081000 998.325012 0.0 Sell
3 3 0 1202 0.072900 13657.200195 0.0 Sell
4 4 0 1567 0.065610 3843.520020 0.0 Sell
5 5 0 1932 0.059049 7200.174316 0.0 Sell
6 6 0 2298 0.053144 29374.152344 0.0 Sell

>>> cash_deposits = symbol_wrapper.wrap(sim_out.cash_deposits) ❹
>>> print(cash_deposits[cash_deposits != 0]) ❺
Date
2014-09-17 00:00:00+00:00 -100.000000
2015-01-02 00:00:00+00:00 -31.424899
2016-01-02 00:00:00+00:00 -39.090061
2017-01-02 00:00:00+00:00 -80.864326
2018-01-02 00:00:00+00:00 -995.609894
2019-01-02 00:00:00+00:00 -252.173348
2020-01-02 00:00:00+00:00 -425.163093
2021-01-02 00:00:00+00:00 -1561.062890
Name: BTC-USD, dtype: float64

```

- ❶ Create a size array to sell 10% of the current position at an annual frequency
- ❷ Withdraw as much as possible at each timestamp. Remember that most arrays can be provided as a scalar to be applied on each element using flexible indexing.
- ❸ Our size means the percentage of the current position
- ❹ Make each order long-only to avoid going into the short direction
- ❺ Access and wrap the cash deposits
- ❻ Positive cash deposits mean money inflow, negative cash deposits mean money outflow

As we see, on the first date of each year, we sold 10% of our position, and since any changes related to cash are only applied at the beginning of each bar, the cash resulting from the transaction can only be withdrawn on the date that follows. Also, whenever we specify cash deposits, vectorbt will create a full-scale array `SimulationOutput.cash_deposits` to account for values that couldn't be satisfied, such as `-np.inf`, which was replaced by the cash balance at each timestamp. Without having this array, we wouldn't be able to properly reconstruct the simulation during the post-analysis phase.

## Cash earnings

In contrast to cash deposits, cash earnings (`cash_earnings`) mark cash that is either inflowing to or outflowing away from the user, and so they have a direct effect on the profitability. Also, they are applied at the end of each bar. For example, we can use (negative) cash earnings to charge some fixed commission during a pre-determined period of time, or to simulate profit taking from stacking cryptocurrency. One of the most useful applications of cash earnings are cash dividends, which are expressed by a separate argument `cash_dividends`. Cash dividends are multiplied by the current position size and added to the cash earnings. Similar to cash deposits, vectorbt creates a separate array for cash earnings once it registers any non-zero value in either cash earnings or cash dividends provided by the user, and writes final operations to this array, available under `SimulationOutput.cash_earnings`.

Let's simulate investing in Apple and keeping the dividends:

```

>>> aapl_data = vbt.YFData.pull("AAPL", end="2022-01-01")
>>> aapl_wrapper = aapl_data.get_symbol_wrapper()
>>> size = aapl_wrapper.fill() ❶
>>> size.iloc[0] = np.inf

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=aapl_wrapper.shape_2d,
...     group_lens=np.array([1]),
...     close=aapl_data.get("Close").values,
...     cash_dividends=aapl_data.get("Dividends").values,
...     size=size.values
... )
>>> print_orders(aapl_wrapper.shape_2d, sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0 0 0 996.754204 0.100326 0.0 Buy

>>> cash_earnings = aapl_wrapper.wrap(sim_out.cash_earnings)
>>> print(cash_earnings[cash_earnings != 0]) ❷
Date
1987-05-11 00:00:00+00:00 0.534260
1987-08-10 00:00:00+00:00 0.534260
1987-11-17 00:00:00+00:00 0.711683
...
2021-05-07 00:00:00+00:00 219.285925
2021-08-06 00:00:00+00:00 219.285925
2021-11-05 00:00:00+00:00 219.285925
Name: AAPL, Length: 73, dtype: float64

```

- ❶ Create a size array for buying the maximum amount at the first bar (`np.inf`) and then holding (`np.nan`)

- ② Positive cash earnings mean money inflow, negative cash earnings mean money outflow

### Max record count

By default, since vectorbt doesn't know the number of order records to be filled in advance, it creates an empty array of the same shape as `target_shape` and gradually "appends" new records. As soon as there are hundreds of thousands or even millions of elements in the target shape, we may run out of memory by trying to create such a huge empty array with such a complex data type. To avoid stressing our RAM too much, we can specify the maximum number of potential records to be filled in each column using `max_order_records` for order records and `max_log_records` for log records.

For instance, if we have tick data with one million of data points, and we want to simulate a simple strategy where we buy at the first timestamp and then sell at the latest timestamp, it makes sense to limit the number of potential orders in each column to just 2:

```
>>> target_shape = (1000000, 1)
>>> np.random.seed(42) ①
>>> rand_price = np.random.randint(8, 12, size=target_shape) ②
>>> size = np.full(target_shape, np.nan)
>>> size[0] = np.inf
>>> size[-1] = -np.inf

>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=target_shape,
...     group_lens=np.array([1]),
...     price=rand_price,
...     size=size,
...     max_order_records=2 ③
... )
>>> print_orders(target_shape, sim_out.order_records)
  Order Id  Column  Timestamp  Size  Price  Fees  Side
0         0       0          0   10.0   10.0   0.0  Buy
1         1       0  999999   20.0    8.0   0.0  Sell
```

- ① Set a seed to make the results reproducible
- ② Quickly generate a random time series acting as a price
- ③ We expect to fill just two orders

Exceeding this limit will throw an error:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=target_shape,
...     group_lens=np.array([1]),
...     price=rand_price,
...     size=size,
...     max_order_records=1 ①
... )
IndexError: order_records index out of range. Set a higher max_order_records.
```

- ① Reduce the maximum count of order records to 1

We can also completely disable filling order records by setting `max_order_records` to zero:

```
>>> sim_out = vbt.pf_nb.from_orders_nb(
...     target_shape=target_shape,
...     group_lens=np.array([1]),
...     price=rand_price,
...     size=size,
...     max_order_records=0
... )
>>> print_orders(target_shape, sim_out.order_records)
Empty DataFrame
Columns: [Order Id, Column, Timestamp, Size, Price, Fees, Side]
Index: []
```

#### Note

`max_order_records` and `max_log_records` are effective for each column. If one column expects to generate 2 records and another one to generate 1000 records, you must use the value of 1000. Also, don't reduce the maximum number of log records (apart from setting it to zero) since logs are generated at each single timestamp.

### Jitting

Every simulation function, including `from_orders_nb`, is registered as a jittable function in `JITRegistry` once vectorbt is imported. The so-called "jitable setup" resulting from the registration contains various information on compilation and decoration, such as what arguments were passed to the Numba's `@njit` decorator. At any point in time, we can instruct the registry to redecorate the function by keeping other decoration arguments at their defaults. For example, to disable Numba and run the simulator as a regular Python function:

```
>>> f_py = vbt.jit_reg.resolve_option(
...     task_id=vbt.pf_nb.from_orders_nb,
...     option=False ①
... )
```

- ① Option gets resolved using `resolve_jitted_option`

To disable caching:



```
>>> f_no_cache = vbt.jit_reg.resolve_option(
...     task_id=vbt.pf_nb.from_orders_nb,
...     option=dict(cache=False)
... )
```

To enable automatic parallelization:

```
>>> f_parallel = vbt.jit_reg.resolve_option(
...     task_id=vbt.pf_nb.from_orders_nb,
...     option=dict(parallel=True)
... )
```

#### Hint

All the returned functions can be used exactly the same way as `from_orders_nb`. The two latter functions can also be called from within Numba.

## Chunking

The same goes for chunking: each simulation function is registered as a chunkable function in [ChunkableRegistry](#) and all arguments in `from_orders_nb` are perfectly chunkable across **groups** (not rows or columns!). But since Numba-compiled functions are meant to be used by other Numba-compiled functions and Numba can't take regular Python functions, they haven't been decorated with the **chunked** decorator just yet. To decorate, we need to explicitly tell the registry to do so:

```
>>> f_chunked = vbt.ch_reg.resolve_option(
...     setup_id_or_func=vbt.pf_nb.from_orders_nb,
...     option=True ❶
... )
>>> print(vbt.prettify(f_chunked.options))
Config(
  n_chunks=None,
  size=ArraySizer(
    arg_query='group_lens',
    single_type=None,
    axis=0
  ),
  min_size=None,
  chunk_len=None,
  chunk_meta=None,
  skip_single_chunk=None,
  arg_take_spec={},
  template_context=None,
  engine=None,
  engine_config={},
  merge_func=<function merge_sim_outs at 0x7f88873f1ea0>,
  merge_kwargs={},
  return_raw_chunks=False,
  silence_warnings=None,
  disable=None
)
```

❶ Option gets resolved using [resolve\\_chunked\\_option](#)

#### Hint

The returned function can be used the same way as `from_orders_nb`, but not within Numba since it's now wrapped with a regular Python function that takes care of chunking.

Let's go all into BTC and ETH while utilizing chunking, which will produce two totally-isolated simulations. Internally, this will split `group_lens` into two arrays (`[1]` and `[1]`), then split each argument value into chunks such that each chunk contains information only about one of the groups, execute the same function but on different chunks using [Dask](#), and finally, merge the results of both simulations as if they were produced by a single, monolithic simulation ✂

```
>>> sim_out = f_chunked(
...     target_shape=mult_symbol_wrapper.shape_2d,
...     group_lens=np.array([1, 1]),
...     close=mult_data.get("Close").values,
...     _nchunks=2, ❶
...     _execute_kwargs=dict(engine="dask")
... )
>>> print_orders(mult_symbol_wrapper.shape_2d, sim_out.order_records)
Order Id Column Timestamp Size Price Fees Side
0      0      0      0.013999 7143.580078 0.0 Buy
1      0      1      0.311639 320.884003 0.0 Buy
```

❶ Add an underscore to an argument to override a setting with the same name in **chunked**

Chunking even knows how to split flexible arrays!

## Class method

Often, using the Numba-compiled simulation function directly is quite cumbersome, given that many of our inputs are either scalars or Pandas objects that would have to be converted to NumPy arrays. Nevertheless, knowing how inputs are processed at the most fundamental level is very important for understanding how vectorbt works under the hood. To add another level of abstraction and to assist the user, vectorbt enhances each simulation function by wrapping it with a class method, such as [Portfolio.from\\_orders](#) for `from_orders_nb`, that automates some input pre-processing and output post-processing.

For example, here's how easy it is to test the three orders we mentioned at the beginning:

```
>>> pf = vbt.Portfolio.from_orders(
...     close=[11, 10, 12],
...     size=[0.1, -0.1, np.nan]
... )
>>> pf.orders.readable
```

|   | Order | Id | Column | Timestamp | Size | Price | Fees | Side |
|---|-------|----|--------|-----------|------|-------|------|------|
| 0 | 0     | 0  | 0      | 0         | 0.1  | 11.0  | 0.0  | Buy  |
| 1 | 1     | 0  | 1      | 0.1       | 10.0 | 0.0   | Sell |      |

But probably the simplest example involves just a single order (for \$10, buy 1 share):

```
>>> pf = vbt.Portfolio.from_orders(10, 1) ❶
>>> pf.orders.readable
```

|   | Order | Id | Column | Timestamp | Size | Price | Fees | Side |
|---|-------|----|--------|-----------|------|-------|------|------|
| 0 | 0     | 0  | 0      | 1.0       | 10.0 | 0.0   | Buy  |      |

❶ Any array-like argument gets converted into a NumPy array

Each class method is basically a small pipeline that pulls default argument values from global settings, broadcasts arguments, checks whether arguments have correct data types, redecorates the simulation function by resolving jitting and chunking options, runs the simulation, and, finally, creates a new [Portfolio](#) instance based on the result of the simulation and ready to be analyzed. But the most important: it makes use of Pandas, including datetime indexes and column hierarchies - nobody wants to work with NumPy arrays alone!

## Close price

In contrast to their Numba-compiled functions, the class methods require the close price to be provided. This requirement is associated with the post-analysis phase: many metrics and time series such as the equity curve can only be reliably calculated using the latest price at each bar since they use information that happens somewhere during that bar, and we should avoid looking into the future by using the open price or any other intermediate data.

## Defaults

If we look at the signature of [Portfolio.from\\_orders](#), we would notice an exceptionally high amount of `None` values. The value `None` has a special meaning and usually instructs vectorbt to replace it with the respective default value from the global settings.

```
>>> vbt.phelp(vbt.Portfolio.from_orders, incl_doc=False)
Portfolio.from_orders(
    close,
    size=None,
    size_type=None,
    direction=None,
    price=None,
    fees=None,
    ...
    jitted=None,
    chunked=None,
    wrapper_kwargs=None,
    freq=None,
    bm_close=None,
    **kwargs
)
```

The global settings for [Portfolio](#) are stored in the config [settings.portfolio](#). For example, by default, vectorbt uses the close price as the order price (remember how the negative infinity means the open price and the positive infinity means the close price?):

```
>>> vbt.settings.portfolio["price"]
inf
```

To change a default, we can override it directly in the settings. For example, let's introduce a fixed commission of \$1 to every order:

```
>>> vbt.settings.portfolio["fixed_fees"] = 1

>>> pf = vbt.Portfolio.from_orders(
...     close=pd.Series([11, 10, 12]),
...     size=pd.Series([0.1, -0.1, np.nan])
... )
>>> pf.orders.readable
```

|   | Order | Id | Column | Timestamp | Size | Price | Fees | Side |
|---|-------|----|--------|-----------|------|-------|------|------|
| 0 | 0     | 0  | 0      | 0.1       | 11.0 | 1.0   | Buy  |      |
| 1 | 1     | 0  | 1      | 0.1       | 10.0 | 1.0   | Sell |      |

^  
here

The settings can be reset as any other [config](#):

```
>>> vbt.settings.portfolio.reset()

>>> vbt.settings.portfolio["fixed_fees"]
0.0
```

Generally, global defaults follow the same schema as keyword arguments across most simulation functions. For example, `price` defaults to `np.array(np.inf)` just about everywhere in Numba. If you cannot find an argument in the global settings, then there is no default value for that argument and `None` is a legitimate

value.

## Enums

In the vectorbt's ecosystem, we can find a lot of arguments of an enumerated type. Enums are regular integers that act as categorical variables. Similarly to contexts, they are also represented by a named tuple, but this tuple is already initialized with values usually ranging from 0 to the total number of categories in the enum. Let's take the [SizeType](#) as an example:

```
>>> print(vbt.prettify(vbt.pf_enums.SizeType))
.SizeType(
  Amount=0,
  Value=1,
  Percent=2,
  TargetAmount=3,
  TargetValue=4,
  TargetPercent=5
)
```

Instead of requiring the user to pass any value as an integer, we can take the name of its field instead and convert it to an integer using [map\\_enum\\_fields](#), which takes the field name and the enumerated type, and returns the value of that field in that type. In addition, it allows us to convert entire collections of fields, such as lists and Pandas objects.

```
>>> vbt.map_enum_fields("targetamount", vbt.pf_enums.SizeType)
3

>>> vbt.map_enum_fields([
...     "amount",
...     "targetamount",
...     "targetpercent"
... ], vbt.pf_enums.SizeType)
[0, 3, 5]
```

Internally, this function uses [apply\\_mapping](#), which enables many useful options, such as ignoring the input if it's already of an integer data type:

```
>>> vbt.map_enum_fields(3, vbt.pf_enums.SizeType)
3
```

Also, by default, it ignores the case and removes all non-alphanumeric characters:

```
>>> vbt.map_enum_fields("Target Amount", vbt.pf_enums.SizeType)
3
```

And that's all the magic behind [Portfolio.from\\_orders](#) and other simulation methods knowing how to handle string options! Let's enter a position of one share at one bar and then sell 50% of it at the next bar:

```
>>> pf = vbt.Portfolio.from_orders(
...     close=pd.Series([10, 11]),
...     size=pd.Series([1, -0.5]),
...     size_type=pd.Series(["amount", "percent"]),
...     direction="longonly"
... )
>>> pf.orders.readable
```

|   | Order | Id | Column | Timestamp | Size | Price | Fees | Side |
|---|-------|----|--------|-----------|------|-------|------|------|
| 0 | 0     | 0  | 0      | 1.0       | 10.0 | 0.0   | Buy  |      |
| 1 | 1     | 0  | 1      | 0.5       | 11.0 | 0.0   | Sell |      |

### Note

Conversion isn't vectorized - it's advisable to provide actual integers when bigger arrays are involved to avoid performance penalties.

## Broadcasting

Once the argument values have been resolved, vectorbt takes all the arguments that should broadcast against the target shape, and passes them to the function [broadcast](#).

Broadcasting is one of the most essential building blocks in vectorbt because it allows us to provide arrays with various shapes and types, including NumPy arrays, Pandas Series and DataFrames, and even regular lists. Whenever we iterate over rows (timestamps) and columns (assets), the simulator needs to know which element in each array currently to pick. Instead of providing huge arrays where each element is set, we can pass some arrays with elements per row, some with elements per column, and some as scalars, and the vectorbt's broadcaster will make sure that they perfectly align with the target shape over which the simulator iterates.

Let's broadcast some arrays manually. Here, we have one column in the price array and want to test multiple combinations of order size by making it a DataFrame with one row and multiple columns:

```
>>> close = pd.Series(
...     [11, 10, 12],
...     index=vbt.date_range("2020-01-01", periods=3)
... )
>>> size = pd.DataFrame(
...     [[-np.inf, np.nan, np.inf]],
...     columns=pd.Index(["short", "nan", "long"], name="size")
... )
```

```
>>> fees = 0.01

>>> broadcasted = vbt.broadcast(dict(
...     close=close,
...     size=size,
...     fees=0.01
... ))
>>> broadcasted["close"]
size      short  nan  long
2020-01-01    11    11    11
2020-01-02    10    10    10
2020-01-03    12    12    12

>>> broadcasted["size"]
size      short  nan  long
2020-01-01   -inf NaN   inf
2020-01-02   -inf NaN   inf
2020-01-03   -inf NaN   inf

>>> broadcasted["fees"]
size      short  nan  long
2020-01-01    0.01 0.01 0.01
2020-01-02    0.01 0.01 0.01
2020-01-03    0.01 0.01 0.01
```

But thanks to flexible indexing, we don't have to bring each argument to the full shape and materialize it. That's why, to avoid high memory consumption, each simulation method also passes `keep_flex=True` to the broadcaster to keep all arguments in their original form suitable for flexible indexing. For them, the broadcaster will only check whether they **can broadcast** to the final shape. Since we not only broadcast NumPy arrays but also Pandas objects as well, we need to return the wrapper resulting from the broadcasting operation, which will contain the final shape and Pandas metadata including the index and columns:

```
>>> broadcasted, wrapper = vbt.broadcast(dict(
...     close=close,
...     size=size,
...     fees=0.01
... ), keep_flex=True, return_wrapper=True)
>>> broadcasted["close"]
[[11]
 [10]
 [12]]

>>> broadcasted["size"]
[[-inf nan inf]]

>>> broadcasted["fees"]
[0.01]

>>> wrapper.fill() ❶
size      short  nan  long
2020-01-01   NaN NaN   NaN
2020-01-02   NaN NaN   NaN
2020-01-03   NaN NaN   NaN
```

❶ Create a dummy array to reveal the broadcasted Pandas metadata

#### Hint

Even though we passed `fees` as a scalar, the broadcaster automatically wrapped it with a NumPy array and expanded to two dimensions for Numba. For the same reason, it also converted Pandas to NumPy.

Some arrays, such as `init_cash` and `cash_deposits`, cannot broadcast together with `close` because their final shape depends on the number of groups, or they are one-dimensional by nature. Thus, after all the regular arrays were aligned, the wrapper was created, and the target shape was established, vectorbt will first create the group lengths array, and then individually broadcast all the arrays that aren't broadcastable against `target_shape`. For example, to broadcast the initial position array:

```
>>> init_position = 1
>>> new_init_position = vbt.broadcast_array_to(init_position, wrapper.shape_2d[1])
>>> new_init_position
array([1, 1, 1])
```

All of this is conveniently done by **Portfolio.from\_orders!**

```
>>> pf = vbt.Portfolio.from_orders(
...     close=close,
...     size=size,
...     fees=fees,
...     init_position=init_position
... )
>>> pf.orders.readable
  Order Id Column Timestamp      Size Price      Fees Side
0      0  short 2020-01-01 10.981098  11.0  1.207921  Sell
1      0   long 2020-01-01  9.000900  11.0  0.990099   Buy

>>> pf.value
size      short  nan  long
2020-01-01 109.792079 111.0 110.009901
2020-01-02 119.773177 110.0 100.009001
2020-01-03  99.810981 112.0 120.010001
```

To control the broadcasting process, we can pass additional arguments to **broadcast** via `broadcast_kwargs`. For example, let's replace the final columns:

```
>>> pf = vbt.Portfolio.from_orders(
...     close=close,
...     size=size,
...     fees=fees,
...     init_position=init_position,
...     broadcast_kwargs=dict(columns_from=["a", "b", "c"])
... )
>>> pf.value
```

|            | a          | b     | c          |
|------------|------------|-------|------------|
| 2020-01-01 | 109.792079 | 111.0 | 110.009901 |
| 2020-01-02 | 119.773177 | 110.0 | 100.009001 |
| 2020-01-03 | 99.810981  | 112.0 | 120.010801 |

We can also wrap any argument with the class `BCO` to provide broadcasting-related keyword arguments just for that particular object, or with the class `Param` to mark the object as a parameter. Below, we are testing the Cartesian product of two parameters: size and fees.

```
>>> pf = vbt.Portfolio.from_orders(
...     close=close,
...     size=vbt.Param([-np.inf, np.inf]), ❶
...     fees=vbt.Param([0, 0.01]),
...     init_position=init_position
... )
>>> pf.value
```

| size       |            | -inf       |            | inf        |
|------------|------------|------------|------------|------------|
| fees       | 0.00       | 0.01       | 0.00       | 0.01       |
| 2020-01-01 | 111.000000 | 109.792079 | 111.000000 | 110.009901 |
| 2020-01-02 | 121.090909 | 119.773177 | 100.909091 | 100.009001 |
| 2020-01-03 | 100.909091 | 99.810981  | 121.090909 | 120.010801 |

❶ We could have also passed a regular `pd.Index` for the same effect

Furthermore, we can broadcast differently-shaped Pandas DataFrames if their column levels overlap. Let's say we have two assets, and we want to test the open and close price as the order price. For this, we need to create a Pandas DataFrame for the order price that has 4 columns, each price type per each asset:

```
>>> mult_close = mult_data.get("Close")
>>> mult_close
```

| symbol                    |  | BTC-USD      | ETH-USD     |
|---------------------------|--|--------------|-------------|
| Date                      |  |              |             |
| 2017-11-09 00:00:00+00:00 |  | 7143.580078  | 320.884003  |
| 2017-11-10 00:00:00+00:00 |  | 6618.140137  | 299.252991  |
| 2017-11-11 00:00:00+00:00 |  | 6357.600098  | 314.681000  |
| ...                       |  | ...          | ...         |
| 2021-12-29 00:00:00+00:00 |  | 46444.710938 | 3628.531738 |
| 2021-12-30 00:00:00+00:00 |  | 47178.125000 | 3713.852051 |
| 2021-12-31 00:00:00+00:00 |  | 46306.445312 | 3682.632812 |

[1514 rows x 2 columns]

```
>>> mult_price = pd.concat((
...     mult_data.get("Open"),
...     mult_data.get("Close")
... ), axis=1, keys=pd.Index(["open", "close"], name="price"))
>>> mult_price
```

| price                     |  | open         |             | close        |
|---------------------------|--|--------------|-------------|--------------|
| symbol                    |  | BTC-USD      | ETH-USD     | BTC-USD      |
| Date                      |  |              |             |              |
| 2017-11-09 00:00:00+00:00 |  | 7446.830078  | 308.644989  | 7143.580078  |
| 2017-11-10 00:00:00+00:00 |  | 7173.729980  | 320.670990  | 6618.140137  |
| 2017-11-11 00:00:00+00:00 |  | 6618.609863  | 298.585999  | 6357.600098  |
| ...                       |  | ...          | ...         | ...          |
| 2021-12-29 00:00:00+00:00 |  | 47623.871094 | 3797.436279 | 46444.710938 |
| 2021-12-30 00:00:00+00:00 |  | 46490.605469 | 3632.219727 | 47178.125000 |
| 2021-12-31 00:00:00+00:00 |  | 47169.371094 | 3713.430176 | 46306.445312 |

[1514 rows x 4 columns]

```
price
symbol
Date
2017-11-09 00:00:00+00:00 320.884003
2017-11-10 00:00:00+00:00 299.252991
2017-11-11 00:00:00+00:00 314.681000
...
2021-12-29 00:00:00+00:00 3628.531738
2021-12-30 00:00:00+00:00 3713.852051
2021-12-31 00:00:00+00:00 3682.632812
```

[1514 rows x 4 columns]

Even though both shapes (1524, 2) and (1524, 4) are **not broadcastable** (!) in NumPy, vectorbt identifies that both DataFrames share the same column level `symbol`, and aligns them based on that level using `align_pd_arrays`, which then yields a successful simulation:

```
>>> pf = vbt.Portfolio.from_orders(close=mult_close, price=mult_price)
>>> pf.value
```

| price                     |  | open       |             | close      |             |
|---------------------------|--|------------|-------------|------------|-------------|
| symbol                    |  | BTC-USD    | ETH-USD     | BTC-USD    | ETH-USD     |
| Date                      |  |            |             |            |             |
| 2017-11-09 00:00:00+00:00 |  | 95.927798  | 103.965402  | 100.000000 | 100.000000  |
| 2017-11-10 00:00:00+00:00 |  | 88.871910  | 96.957022   | 92.644585  | 93.258931   |
| 2017-11-11 00:00:00+00:00 |  | 85.373240  | 101.955648  | 88.997394  | 98.066902   |
| ...                       |  | ...        | ...         | ...        | ...         |
| 2021-12-29 00:00:00+00:00 |  | 623.684312 | 1175.632804 | 650.160150 | 1130.792345 |
| 2021-12-30 00:00:00+00:00 |  | 633.532987 | 1203.276315 | 660.426908 | 1157.381490 |
| 2021-12-31 00:00:00+00:00 |  | 621.827608 | 1193.161381 | 648.224627 | 1147.652355 |

[1514 rows x 4 columns]

#### Info

See the API documentation of [broadcast](#) for more examples on broadcasting.

Even though [Portfolio.from\\_orders](#) is the most basic simulation method, it already takes dozens of broadcastable array-like arguments. So, how do we know exactly which argument is broadcastable?

To see which arguments can broadcast, take a look either at the API documentation of the argument, or at the annotation of the argument in the source code, which has the type `ArrayLike` when it can be provided both as a scalar and as an array. You can also look at the argument annotations of the Numba-compiled simulation function (here [from\\_orders\\_nb](#)) and search for the prefix `FlexArray`. Finally, the last and probably the least underrated method is to look at the argument taking specification of the chunking decorator: arguments that are chunked using [FlexArraySlicer](#) or have `flex` in the specification name are broadcastable by nature. The specification also reveals against what shape the argument should broadcast.

```
>>> print(vbt.prettify(f_chunked.options["arg_take_spec"]["close"]))
FlexArraySlicer(
  single_type=None,
  ignore_none=True,
  mapper={
    'should_cache': True,
    'chunk_meta_cache': {...},
    'arg_query': {
      'pattern': '(group_lens|group_map)',
      'flags': 0
    }
  },
  axis=1
)
```

Here, the argument `close` is expected as a flexible array that is chunked along the column axis using the group lengths mapper. Since the simulation function is always chunked by its groups and columns of this argument are mapped to those groups, it should broadcast along columns in `target_shape` as opposed to groups in `group_lens`. Arguments that have no mapper, such as `cash_deposits`, always broadcast along groups:

```
>>> print(vbt.prettify(f_chunked.options["arg_take_spec"]["cash_deposits"]))
FlexArraySlicer(
  single_type=None,
  ignore_none=True,
  mapper=None,
  axis=1
)
```

#### Hint

As a rule of thumb: if you take a look at the source code of [from\\_orders\\_nb](#), only the arguments that have `portfolio.ch.flex_array_gl_slicer` as their specification broadcast together and build the target shape. All other flexible arguments broadcast individually once the target shape has been built.

## Grouping

Since the target shape is now being generated from broadcasting instead of being passed manually by the user, there is no possibility for the user to provide the group lengths either. Instead, vectorbt will take a grouping instruction and create this array for us. Grouping is performed by constructing a [Grouper](#) instance, which takes the broadcasted columns and a group-by object (`group_by`). It then uses the group-by object to distribute the columns into groups and generate the group lengths array. See the API documentation of [Grouper](#) to learn about various group-by options. For example, we can pass `group_by=True` to put all columns into a single group, or specify the column level by which the columns should be grouped.

And even though [from\\_orders\\_nb](#) automatically turns on cash sharing whenever it discovers multiple columns in a group, we must explicitly enable cash sharing with `cash_sharing` in the class method, otherwise no grouping during the simulation will be performed! This is because a [Portfolio](#) instance can also group its columns during the post-analysis phase, and `cash_sharing` is a special flag that tells vectorbt to perform the grouping during the simulation as well.

Let's demonstrate what it's like to invest all the initial cash into two assets without grouping, with grouping, and with grouping and cash sharing:

```
>>> pf = vbt.Portfolio.from_orders(
...     close=mult_data.get("Close")
... )
>>> pf.value
symbol      BTC-USD      ETH-USD
Date
2017-11-09 00:00:00+00:00  100.000000  100.000000
2017-11-10 00:00:00+00:00   92.644585   93.258931
2017-11-11 00:00:00+00:00   88.997394   98.066902
...
2021-12-29 00:00:00+00:00  650.160150  1130.792345
2021-12-30 00:00:00+00:00  660.426908  1157.381490
2021-12-31 00:00:00+00:00  648.224627  1147.652355

[1514 rows x 2 columns]

>>> pf = vbt.Portfolio.from_orders(
...     close=mult_data.get("Close"),
...     group_by=True
... )
>>> pf.value
Date
```

```

2017-11-09 00:00:00+00:00    200.000000
2017-11-10 00:00:00+00:00    185.903516
2017-11-11 00:00:00+00:00    187.064296
...
2021-12-29 00:00:00+00:00    1780.952495
2021-12-30 00:00:00+00:00    1817.808397
2021-12-31 00:00:00+00:00    1795.876982
Freq: D, Name: group, Length: 1514, dtype: float64

```

```

>>> pf = vbt.Portfolio.from_orders(
...     close=mult_data.get("Close"),
...     group_by=True,
...     cash_sharing=True
... )
>>> pf.value ③
Date
2017-11-09 00:00:00+00:00    100.000000
2017-11-10 00:00:00+00:00    92.644585
2017-11-11 00:00:00+00:00    88.997394
...
2021-12-29 00:00:00+00:00    650.160150
2021-12-30 00:00:00+00:00    660.426908
2021-12-31 00:00:00+00:00    648.224627
Freq: D, Name: group, Length: 1514, dtype: float64

```

- ① Without grouping, each column manages its own initial cash
- ② With grouping but without cash sharing, each column still manages its own initial cash, but during the post-analysis phase vectorbt concatenates both columns to analyze them as a single group, thus we see the first value being \$200 instead of \$100
- ③ With grouping and with cash sharing, both columns manage the same initial cash. The first column went all in and left the second column without the required funds, such that the value effectively reflects only the first asset

Passing `group_by=True` only works when all columns are different assets and there are no parameter combinations. But how about multiple assets and parameter combinations?

Let's play with different group-by instructions on the `mult_close` and `mult_price` arrays that we constructed earlier. The final broadcasted shape has 4 columns: each price type per each asset. What we want to do is to create two groups by putting the assets of each parameter combination into a single basket. We cannot pass `group_by=True` because it will combine all 4 columns. We also cannot pass the column level `symbol` as `group_by` because it will group by asset, that is, it will put the columns with `BTC-USD`, such as `(Open, BTC-USD)` and `(Close, BTC-USD)`, into one group and the columns with `ETH-USD` into another. What we need though is to put `(Open, BTC-USD)` and `(Open, ETH-USD)` into one group and `(Close, BTC-USD)` and `(Close, ETH-USD)` into another; that is, we need to pass all the column levels **apart from symbols** as `group_by`, which can be done in multiple ways:

```

>>> pf = vbt.Portfolio.from_orders(
...     close=mult_close,
...     price=mult_price,
...     group_by=pd.Index(["group1", "group1", "group2", "group2"]) ①
... )
>>> pf.value

```

| Date                      | group1      | group2      |
|---------------------------|-------------|-------------|
| 2017-11-09 00:00:00+00:00 | 199.893199  | 200.000000  |
| 2017-11-10 00:00:00+00:00 | 185.828932  | 185.903516  |
| 2017-11-11 00:00:00+00:00 | 187.328888  | 187.064296  |
| ...                       | ...         | ...         |
| 2021-12-29 00:00:00+00:00 | 1799.317116 | 1780.952495 |
| 2021-12-30 00:00:00+00:00 | 1836.809302 | 1817.808397 |
| 2021-12-31 00:00:00+00:00 | 1814.988988 | 1795.876982 |

[1514 rows x 2 columns]

```

>>> pf = vbt.Portfolio.from_orders(
...     close=mult_close,
...     price=mult_price,
...     group_by=["price"] ②
... )
>>> pf.value

```

| price                     | open        | close       |
|---------------------------|-------------|-------------|
| Date                      |             |             |
| 2017-11-09 00:00:00+00:00 | 199.893199  | 200.000000  |
| 2017-11-10 00:00:00+00:00 | 185.828932  | 185.903516  |
| 2017-11-11 00:00:00+00:00 | 187.328888  | 187.064296  |
| ...                       | ...         | ...         |
| 2021-12-29 00:00:00+00:00 | 1799.317116 | 1780.952495 |
| 2021-12-30 00:00:00+00:00 | 1836.809302 | 1817.808397 |
| 2021-12-31 00:00:00+00:00 | 1814.988988 | 1795.876982 |

[1514 rows x 2 columns]

```

>>> pf = vbt.Portfolio.from_orders(
...     close=mult_close,
...     price=mult_price,
...     group_by=vbt.ExceptLevel("symbol") ③
... )
>>> pf.value

```

| price                     | open        | close       |
|---------------------------|-------------|-------------|
| Date                      |             |             |
| 2017-11-09 00:00:00+00:00 | 199.893199  | 200.000000  |
| 2017-11-10 00:00:00+00:00 | 185.828932  | 185.903516  |
| 2017-11-11 00:00:00+00:00 | 187.328888  | 187.064296  |
| ...                       | ...         | ...         |
| 2021-12-29 00:00:00+00:00 | 1799.317116 | 1780.952495 |
| 2021-12-30 00:00:00+00:00 | 1836.809302 | 1817.808397 |

2021-12-31 00:00:00+00:00 1814.988988 1795.876982

- 1 Pass any collection of group labels manually, preferably a Pandas Index. This collection will be converted into a column level. Must have the same length as the number of columns in the target shape.
- 2 Pass the names of all column levels except the one with the assets. Only works when there are multiple levels, that is, the column index is a `pd.MultiIndex`. The to-be-grouped column levels will be visible in the final column hierarchy.
- 3 Pass the column level with the assets to `ExceptLevel` to group all column levels except this one. This will work with any column index - the best option!

#### Important

To make sure that the grouping operation on assets was successful, the final column hierarchy should include all columns levels except the one with asset symbols. For example, passing `group_by=True` will hide all columns levels, while passing `group_by='symbol'` will show only the column level with asset symbols ✗

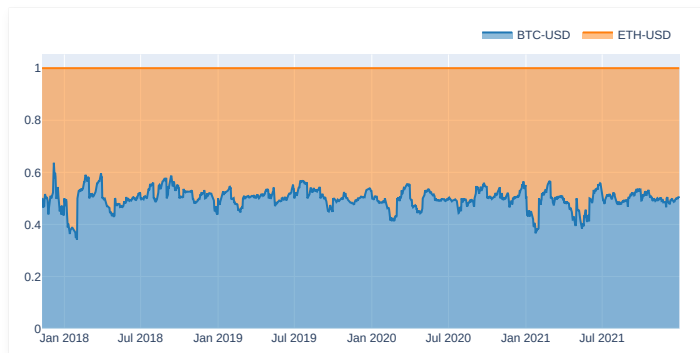
## Call sequence

Similarly to grouping, the class method also simplifies handling of call sequences. There is an argument `call_seq` that not only accepts a (broadcastable) array, but can also be provided as a value of the enum `CallSeqType`. For example, we can pass "auto" to sort the call sequence automatically and to first execute the assets that should be sold before executing the assets that should be bought, which is an important requirement for rebalancing. Let's create an equally-weighted portfolio that is being rebalanced every month:

```
>>> size = mult_symbol_wrapper.fill(np.nan)
>>> size.vbt.set(0.5, every="M", inplace=True)
>>> size.iloc[0] = 0.5

>>> pf = vbt.Portfolio.from_orders(
...     close=mult_data.get("Close"),
...     size=size,
...     size_type="targetpercent",
...     group_by=vbt.ExceptLevel("symbol"),
...     cash_sharing=True,
...     call_seq="auto"
... )
>>> allocations = pf.get_asset_value(group_by=False).vbt / pf.value
>>> allocations.vbt.plot(
...     trace_kwargs=dict(stackgroup="one"),
...     use_gl=False
... ).show()
```

- 1 Plot how much each asset contributes to the portfolio value



#### Hint

By the way, this is exactly the way `Portfolio.from_optimizer` uses `Portfolio.from_orders` for rebalancing!

To access the sorted call sequence after the simulation, we can pass `attach_call_seq` and then read the property `Portfolio.call_seq`:



```
>>> pf = vbt.Portfolio.from_orders(
...     close=mult_data.get("Close"),
...     size=size,
...     size_type="targetpercent",
...     group_by=vbt.ExceptLevel("symbol"),
...     cash_sharing=True,
...     call_seq="auto",
...     attach_call_seq=True ❶
... )
>>> pf.call_seq
symbol          BTC-USD  ETH-USD
Date
2017-11-09 00:00:00+00:00      0      1
2017-11-10 00:00:00+00:00      1      0
2017-11-11 00:00:00+00:00      1      0
...
2021-12-29 00:00:00+00:00      1      0
2021-12-30 00:00:00+00:00      1      0
2021-12-31 00:00:00+00:00      1      0

[1514 rows x 2 columns]
```

❶ Here

## Unlimited cash

When testing many trading strategies and parameter combinations, cash may quickly become a bottleneck. Also, determining the right amount of starting capital is often a challenge by itself. Gladly, there is a range of options that help us in backtesting a trading strategy without having to think about cash limits. One common approach is to pass `np.inf` as `init_cash` to simulate an unlimited cash balance. Welcome to the billionaires club 🏖️

But this would drive the post-analysis havoc because the portfolio value at each timestamp would be infinite as well. To account for this, we can instruct vectorbt to simulate an unlimited cash balance during the simulation, and then post-analyze the expenditures to determine the optimal starting capital. If wanted, we can then re-run the simulation but with the calculated optimal amount. To make this possible, we can pass one of the options from `InitCashMode` as `init_cash`. As opposed to other enums, this enum contains only negative values such that they cannot be confused with zero or positive amounts.

### Important

During the simulation, each group value will be infinity. Thus, we cannot use the size of (+/-) infinity, but also percentages, target percentages, and other size types that depend on the group value. We also cannot fill returns during the simulation.

Let's **DCA** into BTC and ETH by buying one unit each year, and then, retrospectively, find out how much capital this activity would require:

```
>>> size = mult_symbol_wrapper.fill(np.nan)
>>> size.vbt.set(1, every="Y", inplace=True)
>>> size.iloc[0] = 1

>>> pf = vbt.Portfolio.from_orders(
...     close=mult_data.get("Close"),
...     size=size,
...     init_cash="auto" ❶
... )
>>> pf.init_cash ❷
symbol
BTC-USD      61218.626953
ETH-USD      2095.513962
Name: init_cash, dtype: float64
```

❶ Use `InitCashMode.Auto`

❷ Calculated after the simulation

We can then pass these amounts to a new simulation if desired:

```
>>> pf2 = vbt.Portfolio.from_orders(
...     close=mult_data.get("Close"),
...     size=size,
...     init_cash=pf.init_cash
... )
>>> pf2.cash.loc[~size.isnull().all(axis=1)] ❶
symbol          BTC-USD  ETH-USD
Date
2017-11-09 00:00:00+00:00  54075.046875  1774.629959
2018-01-01 00:00:00+00:00  40417.846680  1001.988968
2019-01-01 00:00:00+00:00  36574.326660  861.169556
2020-01-01 00:00:00+00:00  29374.152344  730.367554
2021-01-01 00:00:00+00:00    0.000000    0.000000
```

❶ Print out the cash balance after each investment

We can see how each investment gradually reduces the cash balance and how the final investment drains it completely, while still enabling us to order exactly one unit of each cryptocurrency:

```
>>> pf2.orders.readable
Order Id  Column          Timestamp      Size      Price      Fees Side
0         0  BTC-USD  2017-11-09 00:00:00+00:00  1.0    7143.580078  0.0  Buy
1         1  BTC-USD  2018-01-01 00:00:00+00:00  1.0   13657.200195  0.0  Buy
2         2  BTC-USD  2019-01-01 00:00:00+00:00  1.0   3843.520020  0.0  Buy
```

|   |   |         |                           |     |              |     |     |
|---|---|---------|---------------------------|-----|--------------|-----|-----|
| 3 | 3 | BTC-USD | 2020-01-01 00:00:00+00:00 | 1.0 | 7200.174316  | 0.0 | Buy |
| 4 | 4 | BTC-USD | 2021-01-01 00:00:00+00:00 | 1.0 | 29374.152344 | 0.0 | Buy |
| 5 | 0 | ETH-USD | 2017-11-09 00:00:00+00:00 | 1.0 | 320.884003   | 0.0 | Buy |
| 6 | 1 | ETH-USD | 2018-01-01 00:00:00+00:00 | 1.0 | 772.640991   | 0.0 | Buy |
| 7 | 2 | ETH-USD | 2019-01-01 00:00:00+00:00 | 1.0 | 140.819412   | 0.0 | Buy |
| 8 | 3 | ETH-USD | 2020-01-01 00:00:00+00:00 | 1.0 | 130.802002   | 0.0 | Buy |
| 9 | 4 | ETH-USD | 2021-01-01 00:00:00+00:00 | 1.0 | 730.367554   | 0.0 | Buy |

### Output arrays

All the arrays that are returned as part of `SimulationOutput`, such as `cash_deposits`, `cash_earnings`, and `in_outputs.returns`, can be accessed as an attribute with the same name of a `Portfolio` instance. Let's do the same example as we did in [Cash deposits](#):

```
>>> size = symbol_wrapper.fill(np.nan)
>>> size.vbt.set(-0.1, every="Y", inplace=True)

>>> pf = vbt.Portfolio.from_orders(
...     close=data.get("Close"),
...     size=size,
...     size_type="percent",
...     direction="longonly",
...     init_position=1,
...     cash_deposits=-np.inf
... )
>>> pf.cash_deposits[pf.cash_deposits != 0]
Date
2014-09-17 00:00:00+00:00    -100.000000
2015-01-02 00:00:00+00:00    -31.424899
2016-01-02 00:00:00+00:00    -39.090061
2017-01-02 00:00:00+00:00    -80.864326
2018-01-02 00:00:00+00:00   -995.609894
2019-01-02 00:00:00+00:00   -252.173348
2020-01-02 00:00:00+00:00   -425.163093
2021-01-02 00:00:00+00:00  -1561.062890
dtype: float64
```

### Max record count

Another automation brought by `Portfolio.from_orders` is touching the maximum number of order and log records. Whenever we pass `None`, `from_orders_nb` will pick the maximum possible number of records. In the class method though, vectorbt will check how many non-NaN values there are in the size array, and will pick the highest number across all columns. The same goes for logs, where it determines the number of `True` values. This has almost no impact on performance because vectorbt doesn't need to fully broadcast both arrays to get these numbers. Thus, we don't have to provide `max_order_records` and `max_log_records` if we decide to represent inactive data points as NaN in a huge close array.

### Data type checks

Numba is the closest thing to a statically-typed code in vectorbt, and directly passing wrong data types to Numba usually leads to an ugly-formatted exception. To provide us with a bit of help in debugging, vectorbt checks all data types prior to calling `from_orders_nb`.

```
>>> vbt.Portfolio.from_orders(True)
AssertionError: Data type of 'close' must be <class 'numpy.number'>, not bool
```

Here, the argument `close` seems to require a numeric data type 😊

#### Note

In vectorbt, almost no function will change the data type of an array in a hidden manner because casting may be quite expensive in performance terms, especially when huge arrays are involved. It's the responsibility of the user to supply properly typed and sized data!

### Jitting

Remember how we resolved various jitting options to redecorate `from_orders_nb`? Just like everywhere in vectorbt, in the class method a jitting option can be provided using the argument `jitted`. Let's test a random simulation without and with automatic parallelization with Numba:

```
>>> big_target_shape = (1000, 1000)
>>> big_rand_price = np.random.randint(8, 12, size=big_target_shape)
>>> big_size = np.full(big_target_shape, 1)
>>> big_size[1::2] = -1
```

```
>>> %timeit
>>> vbt.Portfolio.from_orders(
...     close=big_rand_price,
...     size=big_size
... )
113 ms ± 2.34 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

```
>>> %timeit
>>> vbt.Portfolio.from_orders(
...     close=big_rand_price,
...     size=big_size,
...     jitted=dict(parallel=True)
... )
```

97 ms ± 3.47 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

#### Info

Preset simulators like `from_orders_nb` can only be parallelized along groups and columns, thus it would make no difference in enabling the parallel mode when there is only one group or column present. But also, it's usually a better idea to use chunking for parallelization as Numba may yield no performance benefit at all. Only certain user-crafted pipelines that make heavy use of math can be parallelized well with Numba.

## Chunking

Similarly to a jitting option, chunking can be enabled by providing an option via the argument `chunked`. Below, we do the same benchmark as above but now using Dask:

```
>>> %%timeit
>>> vbt.Portfolio.from_orders(
...     close=big_rand_price,
...     size=big_size,
...     chunked=dict(engine="dask", n_chunks=4)
... )
67.2 ms ± 3.63 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

#### Info

Multithreading with Dask is better suited for this job than multiprocessing with Ray because, by default, all Numba-compiled functions in vectorbt release the [GIL](#), and there is a much smaller overhead when starting multiple threads than processes. Consider using multiprocessing when the function takes a considerable amount of time to run.

## Use cases

This method is best suited for backtesting jobs where order information is known beforehand and there is no order that changes its parameters depending on changes in the environment. That is, we cannot implement SL, TP, limit, or any other complex order type using this method. It's really just a smart way of representing multiple instances of `Order` in a vectorized and resource-cheap way - imagine how difficult it would be to supply a list of named tuples instead of arrays! There are two particular use cases where `Portfolio.from_orders` becomes interesting: portfolio rebalancing with predefined weights, and "what-if" analysis. While we've already touched rebalancing more than enough times, the latter use case is about simulating and analyzing various hypothetical scenarios of a real-world trading activity.

Let's say we have made 3 trades on SOL/BTC on Binance and want to analyze them in depth. Even if Binance has made some improvements in its trade analysis capabilities, doing it with vectorbt opens an entirely new dimension. First, we need to pull the close price of a granularity with which we would like the trades to be analyzed. Then, we need to convert trade information into orders. Finally, we can use `Portfolio` to see how the portfolio evolved over time!

```
>>> trade1 = dict(
...     timestamp="2022-01-22 12:39:26",
...     price=0.0027702,
...     size=4.99,
...     fixed_fees=1.01571e-05 ❶
... )
>>> trade2 = dict(
...     timestamp="2022-01-29 02:12:50",
...     price=0.00243,
...     size=-1.72,
...     fixed_fees=3.0549e-06
... )
>>> trade3 = dict(
...     timestamp="2022-01-29 02:52:54",
...     price=0.0024299,
...     size=-3.27,
...     fixed_fees=5.8102e-06
... )

>>> trades = pd.DataFrame([trade1, trade2, trade3]) ❷
>>> trades["timestamp"] = pd.to_datetime(trades["timestamp"], utc=True) ❸
>>> trades.set_index("timestamp", inplace=True)
>>> trades
           price  size  fixed_fees
timestamp
2022-01-22 12:39:26+00:00  0.00277  4.99  0.000010
2022-01-29 02:52:54+00:00  0.00243 -1.72  0.000003
2022-01-29 02:52:54+00:00  0.00243 -3.27  0.000006

>>> solbtc_data = vbt.BinanceData.pull( ❹
...     "SOLBTC",
...     start=trades.index[0] - pd.Timedelta(days=1),
...     end=trades.index[-1] + pd.Timedelta(days=1),
...     timeframe="1h"
... )

>>> resampler = vbt.Resampler( ❺
...     source_index=trades.index,
...     target_index=solbtc_data.wrapper.index,
...     source_freq=None,
...     target_freq="1h"
... )

>>> ❻

>>> @njit
... def avg_price_reduce_meta_nb(from_i, to_i, col, size, price): ❼
```

```

...     _size = size[from_i:to_i, col]
...     _price = price[from_i:to_i, col]
...     return np.sum(_price * _size) / np.sum(_size)

>>> price = pd.Series.vbt.resample_to_index( ❸
...     resampler,
...     avg_price_reduce_meta_nb,
...     vbt.to_2d_array(trades["size"]),
...     vbt.to_2d_array(trades["price"]),
...     wrapper=trades["price"].vbt.wrapper,
... )
>>> price.loc[~price.isnull()] ❹
Open time
2022-01-22 12:00:00+00:00    0.00277
2022-01-29 02:00:00+00:00    0.00243
Freq: 158H, Name: price, dtype: float64

>>> size = trades["size"].vbt.resample_to_index( ❿
...     resampler,
...     vbt.nb.sum_reduce_nb
... )
>>> size.loc[~size.isnull()]
Open time
2022-01-22 12:00:00+00:00    4.99
2022-01-29 02:00:00+00:00   -4.99
Freq: 158H, Name: size, dtype: float64

>>> fixed_fees = trades["fixed_fees"].vbt.resample_to_index(
...     resampler,
...     vbt.nb.sum_reduce_nb
... )
>>> fixed_fees.loc[~fixed_fees.isnull()]
Open time
2022-01-22 12:00:00+00:00    0.000010
2022-01-29 02:00:00+00:00    0.000009
Freq: 158H, Name: fixed_fees, dtype: float64

>>> pf = vbt.Portfolio.from_orders( ❾
...     open=solbtc_data.get("Open"),
...     high=solbtc_data.get("High"),
...     low=solbtc_data.get("Low"),
...     close=solbtc_data.get("Close"),
...     price=price,
...     size=size,
...     fixed_fees=fixed_fees,
...     init_cash=0.1,
...     ffill_val_price=False,
...     skipna=True
... )
>>> pf.orders.readable ❿
   Order Id  Column      Timestamp  Size  Price    Fees  Side
0         0      0  2022-01-22 12:00:00+00:00  4.99  0.00277  0.000010  Buy
1         1      1  2022-01-29 02:00:00+00:00  4.99  0.00243  0.000009  Sell

>>> pf.plot().show() ⓫

```

- ❶ Commission paid for each trade is provided in absolute terms, thus name it "fixed fees" to avoid confusion with "fees" that are usually given in %. Also, make sure that this number is provided in the quote currency (BTC in our example).
- ❷ Put the trades into a DataFrame for easier handling
- ❸ Timezone must be the same as coming from Binance - UTC
- ❹ Pull the hourly SOL/BTC data ranging from the day before the first trade to the day after the last trade (this time buffer is optional)
- ❺ Datetime index of the pulled data acts as a backbone to our time-series analysis, thus we need to put our trades to timestamps present in this index. For this, we need to create an instance of [Resampler](#) mapping the trade timestamps to the pulled bars.
- ❻ Since some trades (second and third in our example) may fall into the same hour and [Portfolio.from\\_orders](#) cannot execute multiple orders at the same bar, we need to aggregate their information
- ❼ Implement a meta function that reduces the price of all the trades that fall into the same bar using the size-weighted average
- ❽ Pass the resampler and the meta function to [GenericAccessor.resample\\_to\\_index](#) to resample the trade price
- ❾ Display the aggregated price points for validation
- ❿ Size and fixed fees can be easily aggregated using the sum operation
- ⓫ Backtest the trades by setting the initial "cash" to 0.1 BTC and skipping missing data points
- ❿ Filled orders have the same information as our trades
- ⓫ Plot the portfolio



We can now change the `size`, `price`, and `fixed_fees` arrays to our liking and re-run the simulation to see how the performance of our trading strategy has been affected 🧙

## Summary

As we just learned, `vectorbt` deploys a range of preset simulators, each consisting of a Numba-compiled core, and a class method on top of `Portfolio` that wraps that core and supports it with diverse enhancements that make it easier and user-friendlier to operate. In particular, we looked at the most primitive simulator - "from orders", which takes a shape (timestamps x assets + parameter combinations), and at each single element of that shape, puts different information pieces like puzzles together to create an order instance. For visual thinkers: you can imagine it taking all arrays together, broadcasting them on the fly against a common shape, and overlaying them on top of each other to form a cube where each element viewed from top is a vector with order information from `Order`.

Because of flexible indexing, we don't have to actually broadcast and materialize all those arrays - `vectorbt` is smart enough to project (i.e., extrapolate) smaller arrays to bigger shapes, such that we can supply incomplete information per timestamp, per asset, or per the entire matrix, as if we supplied that information per each element. This way, there is almost no additional memory footprint, which allows us to conveniently work on big data and perform hyperparameter optimization without leaving Numba, as long as all input arrays fit into RAM, of course 😊

Finally, this documentation piece gave us a glimpse into how `vectorbt` gradually stacks various levels of abstraction to automate tasks. We started with just two commands - buy and sell, added tons of features on the way, and ended up with a Python method that makes it almost ridiculously easy to backtest things. And still, this method lies at the bottom of the food chain: it cannot backtest trading strategies where orders depend on the current simulation state, that is, we have to know all the order information prior to starting the simulation. But get ready, that's where signals and order functions come into play!

[Python code](#)

Documentation Portfolio

## From signals

The method `Portfolio.from_orders` (FO), which was discussed previously, is the most primitive simulation method: it takes order information in form of multiple array-like arguments and broadcasts them to a single shape, such that we know exactly what has to be ordered of each asset at each bar. This method requires us to have that information in advance, regardless of any events during the simulation. But what if we wanted to create an order only given that we're not currently in the market, or in general, to make an order dependable on the current simulation state? Such a conditional logic cannot be represented using orders alone - we either need to use a callback, or define more arrays. The former and the latter are both implemented by `Portfolio.from_signals` (FS).

Before we dive deep into this method, make sure to learn more about signals [here](#). In a nutshell: signals are an abstraction layer over orders. Each signal consists of four boolean values: **1** long entry, **2** long exit, **3** short entry, and **4** short exit. A combination of these values enables us to control the direction of an order relative to the current position. For example, a short entry flag will reverse the current long position, or open a new short one if we're not in the market. This way, position management can be abstracted away from order management, such that we can lean back and only express our decision of whether we're currently bullish or bearish - a perfect playground for ML models, by the way.

And there's another reason to love signals: statistically, in an entire universe of signal [permutations](#), there is at least one permutation that beats the market all the time. This means that we can design a perfect trading algorithm using the above signal schema alone - we just need to guess the right timing and direction of each signal, which reduces the number of factors we need to care about to just two (in a perfect world, of course, because in the real world we also need to take into account the risk, execution modalities, etc.). For example, if the price of any security is \$21 at day 1, \$20 at day 2, and \$22 at day 3, we can short entry at day 1 and long entry at day 2 to get positive-only returns. That's why trading systems and their backtesting components as well shouldn't necessarily scream from complexity in order to be profitable - they just need a robust signal generator as their algorithmic backbone and a trading infrastructure that closely matches the backtesting one.

### Example

Here are all the signal permutations for a price series with 4 points and their total return:

```
>>> from vectorbtpro import *
>>> price = np.array([21, 20, 22, 21])
>>> returns = (price[1:] - price[:-1]) / price[:-1]
>>> permutations = list(product([False, True], repeat=len(returns)))
>>> total_return = np.prod(1 + np.where(permutations, returns, -returns), axis=1) - 1
>>> pd.Series(total_return, index=permutations).sort_values(ascending=False)
(False, True, False)    0.204762
(False, True, True)     0.100000
(True, True, False)     0.095238
(True, True, True)      0.000000
(False, False, False)   -0.014286
(False, False, True)    -0.100000
(True, False, False)    -0.103896
(True, False, True)     -0.181818
dtype: float64
```

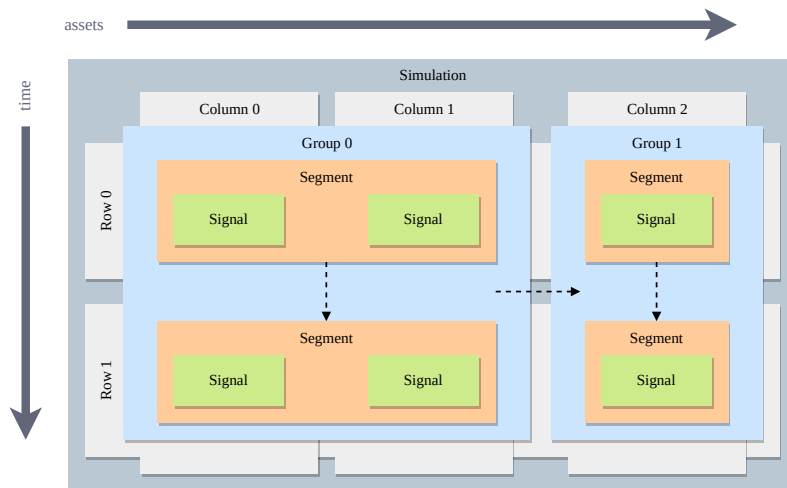
Don't run this on longer price series since the number of permutations grows exponentially with the number of data points -  $2^n$ . That is, a year of daily history would require checking  $2^{365}$  or  $7.515336 \times 10^9$  permutations.

## Mechanics

Similarly to FO, this method is also a class method of `Portfolio` and has two Numba-compiled core functions: `from_signals_nb` and `from_signal_func_nb`. In fact, FS shares many arguments with FO, especially those used to set up the simulation, such as `init_cash`, and those carrying order information, such as `size`. For instance, if we look at the API documentation of the argument `size` under `Portfolio.from_signals`, we'll see "See `Portfolio.from_orders`". But also the simulation procedure of FS itself is very similar to that of FO: while looping over all columns and rows, at each iteration, it resolves the current order and executes it by appending information on the filled order to the order records and updating the current simulation state. But that's where the similarities end.

## Framework

Here's an abstract visualization of the framework of FS run on three rows and two groups with two columns and one column respectively:

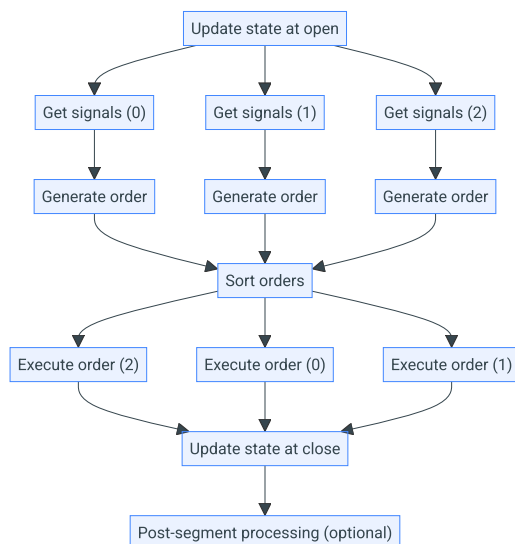


If you worked with vectorbt long enough, you have likely noticed that the framework of `Portfolio.from_signals` follows that of `Portfolio.from_orders` and `Portfolio.from_order_func`. Like most things in the vectorbt's universe, the simulation with `FS` is done by iterating over a so-called "target shape". This shape consists of two dimensions: rows representing the time axis and columns representing the asset axis (or, generally, configurations). Columns are further divided into groups: if multiple columns share the same cash, they are put into the same group (blue rectangle on the left above), while columns without cash sharing or grouping are isolated and appear as a group with exactly one column (blue rectangle on the right above). Groups are considered to be separate, atomic backtesting instances that aren't connected by any means, that is, splitting the shape by target groups shouldn't affect the final result. This, by the way, is why chunking is generally performed on groups rather than columns 💡

The actual iteration over the rows and groups happens in the **column-major order**: the simulator starts moving over the rows in the first group, and once finished, continues with the second group. Every time it hits a new row when processing a group, all the assets at this row are called a "segment" because they together compete for the same resources at the same time, or are connected by any user-defined means. For example, an account with `BTC-USD` and `ETH-USD` on the date `2020-01-01` is considered a segment because the value of both assets adds to the total value of the group at this date. Each asset within a segment is called an "element", which is the smallest simulation unit. An element in `FS` can host only one order, such that the number of filled orders is effectively capped by the number of rows times the number of columns. For example, a year of the daily `BTC-USD` and `ETH-USD` history can generate at most  $365 * 2 = 730$  orders, or one order per bar and asset.

#### Segment workflow

Segment is where the major part of the simulation takes place:



`FS` first updates the current simulation state using the opening price. This is done to get the group value in case some order has the size defined as a (target) percentage, to be able to convert the size into an absolute amount of units. Then, it iterates through the columns in the current group and determines the four signals under each column. Those signals then get converted into an order specification that is quite similar to the one `FQ` takes.

After having resolved all the order specifications, and if the automatic call sequence is enabled, the simulator attempts to sort the orders by their potential value such that sell orders are executed first - which is needed for rebalancing. This is only possible if all the (limit, order, and user-defined) orders within the current group happen either at the beginning or at the end of the current bar. If some orders happen in the middle of the bar, or some happen at the beginning and some at the end of a bar, an error will be thrown because we can only sort orders if they are guaranteed to happen at the same time! If the dynamic call sequence is disabled and there are orders in multiple bar zones, the simulator will sort them by the bar zone.

Finally, `FS` goes through the columns in a newly established call sequence to execute the orders, and updates the simulation state once again using the closing price. The final update is required to optionally generate the portfolio returns and for the segment post-processing function.

## Signal generation

`FS` supports two signal generation modes: fixed (cached), and dynamic (non-cached). The first mode is implemented by the function `from_signals_nb`, which takes the signals as four pre-defined arrays and doesn't allow defining any callbacks, thus it's perfectly cacheable and doesn't have to be recompiled with each new runtime, unless it discovers a new set of data types. The second mode is implemented by the function `from_signal_func_nb`, which doesn't accept any signal arrays, but defines a signal function - a special callback meant to generate the four signals for each asset and at each bar dynamically. It's most suited for use cases where the signal depends on the current simulation state. Furthermore, it defines a callback that is getting called after processing the current segment, which can be used to pre-compute various metrics, such as the Sharpe ratio. The main downside is that it cannot be cached (yet), thus it has to be re-compiled in each new runtime (🍌 to those running vectorbt as a script).

The convenience of the method `Portfolio.from_signals`, which wraps those two modes, is in its ability to choose the mode automatically: whenever we override any default callback, it runs the second mode over the first one.

### Signal function

Remember how in `FO` we had to provide everything as arrays and could neither dynamically change the provided information nor affect the execution in any way? `FS` is much more flexible than that: it expects most information to be defined beforehand (acting as a facade), while signals can be generated both statically and dynamically. Let's play with the dynamic signal generation a bit.

The second mode is implemented by accepting a user-defined callback function, `signal_func_nb`. Whenever the main simulation loop hits a new row (bar), it asks each asset in the current group to generate signals using this callback function. For this, it packs all the information that might be useful to the user, such as the current cash balance and the group value, into a named tuple of the type `SignalContext`. In return, it expects the function to return four signals, which will be used to create an order for that asset.

Here's how a dead-simple signal function that orders nothing looks like:

```
>>> from vectorbtpro import *

>>> @njit
... def signal_func_nb(c):
...     return False, False, False, False

>>> close = pd.DataFrame({
...     "BTC-USD": [20594.29, 20719.41, 19986.60, 21084.64],
...     "ETH-USD": [1127.51, 1125.37, 1051.32, 1143.20],
...     "DOT-USD": [7.88, 7.74, 7.41, 7.78],
...     "BNB-USD": [216.90, 219.67, 214.23, 228.92]
... })

>>> pf = vbt.Portfolio.from_signals(
...     close=close,
...     signal_func_nb=signal_func_nb
... )
>>> pf.order_records
array([], dtype={...})
```

#### Hint

To avoid waiting for the function to compile, remove the `@njit` decorator from `signal_func_nb` and pass `jitted=False` to `from_signals` in order to disable Numba for this method completely. Do this only if the amount of input data is small (< 1000).

To better understand when the function is called, let's expand our data to two assets and print out the current column and row:

```
>>> @njit
... def signal_func_nb(c):
...     print(c.col, c.i)
...     return False, False, False, False

>>> pf = vbt.Portfolio.from_signals(
...     close=close[["BTC-USD", "ETH-USD"]],
...     signal_func_nb=signal_func_nb
... )
0 0
0 1
0 2
0 3
1 0
1 1
1 2
1 3
```

We see that the function was called at each row, first in the column `BTC-USD`, then in the column `ETH-USD`. Here, both assets are acting as isolated tests, thus the simulator processes one column after another. But once we introduce a grouping with or without cash sharing, which binds columns semantically, the simulator will process the columns group-wise, that is, it will move over the groups, then over the rows, and finally over the columns in the current group and at the current row. Let's demonstrate this by introducing two groups with two assets sharing the same cash:

```
>>> @njit
... def signal_func_nb(c):
...     print(c.group, c.col, c.i)
...     return False, False, False, False
```



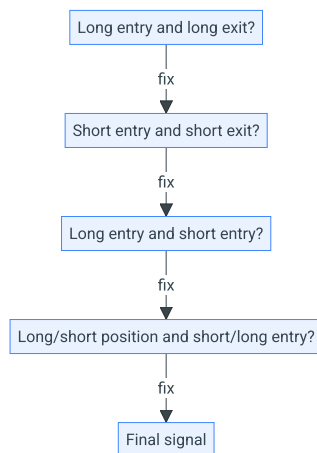
```
>>> pf = vbt.Portfolio.from_signals(
...     close=close,
...     signal_func_nb=signal_func_nb,
...     group_by=[0, 0, 1, 1],
...     cash_sharing=True
... )
0 0 0
0 1 0
0 0 1
0 1 1
0 0 2
0 1 2
0 0 3
0 1 3
1 2 0
1 3 0
1 2 1
1 3 1
1 2 2
1 3 2
1 2 3
1 3 3
```

The context tuple passed to the signal function contains all the necessary information to identify the position of the call in the simulation. For example, we can use `c.index[c.i]` with `SignalContext.index` and `SignalContext.i` to get the timestamp of the current bar. We can also change the current state of any pending limit or stop order before it's processed since the signal function is conceptually executed right before the beginning of the bar.

Thanks to the strict processing of groups from left to right and storing the state of each group globally, we can access the order records and, generally, the latest simulation state of all the groups that were processed earlier. For example, `SignalContext.last_cash` has the same number of elements as there are groups. This is powerful and dangerous at the same time: we can introduce complex intergroup relationships if wanted, or accidentally access the wrong group if not paying enough attention.

### Signal resolution

Signals are just one additional level of abstraction over orders, meaning there needs to be some logic in place that translates them into order specifications. Indeed, whenever the simulator receives a new set of four signals at each row and column, it first resolves them into a single signal, which then gets converted into an order. The resolution step checks whether the provided signals have conflicts. Mostly, the signals are expected to have only one `True` value and three `False` values, but sometimes multiple signals are `True`, especially when the signal function is forwarding data from multiple boolean arrays. In such a case, the simulator goes through the following procedure consisting of multiple checks and fixes:



It first checks whether there are multiple `True` values within the same direction, for example, when the long entry and long exit are both set. To decide which one in the long direction to keep, it looks at the argument `upon_long_conflict` of the type `ConflictMode` provided by the user. For example, the option "adjacent" will pick the signal adjacent to the position we are currently in such that only the long entry will remain active if we're in a long position. This is done by calling the function `resolve_signal_conflict_nb`:

```
>>> vbt.pf_nb.resolve_signal_conflict_nb(
...     position_now=20,
...     is_entry=True,
...     is_exit=True,
...     direction=vbt.pf_enums.Direction.LongOnly,
...     conflict_mode=vbt.pf_enums.ConflictMode.Adjacent
... )
(True, False)
```

After deciding for at most one signal in both directions, the simulator checks whether both the long and short entry are active and uses the function `resolve_dir_conflict_nb` based on the argument `upon_dir_conflict` of the type `DirectionConflictMode` to select the winner. For example, we can choose to always go short when there is any uncertainty:

```
>>> vbt.pf_nb.resolve_dir_conflict_nb(
...     position_now=20,
...     is_long_entry=True,
...     is_short_entry=True,
...     upon_dir_conflict=vbt.pf_enums.DirectionConflictMode.Short,
```

```
... )
(False, True)
```

Finally, it calls the function [resolve\\_opposite\\_entry\\_nb](#) if there is an entry signal that is opposite to the direction of the current position. For example, if we're in a long position and the short entry signal is set, the simulator will use the argument `upon_opposite_entry` of the type [OppositeEntryMode](#) to decide whether to reduce, close, or completely reverse the current long position. Let's make the short entry signal behave like the long exit signal:

```
>>> vbt.pf_nb.resolve_opposite_entry_nb(
...     position_now=20,
...     is_long_entry=False,
...     is_long_exit=False,
...     is_short_entry=True,
...     is_short_exit=False,
...     upon_opposite_entry=vbt.pf_enums.OppositeEntryMode.Close,
...     accumulate=False, ❶
... )
(False, True, False, False, 0)
```

❶ This argument can be `True`, `False`, or any of [AccumulationMode](#)

In the end, only one active signal out of four will remain 📊

## Signal conversion

We have our one signal, now what? It's time to convert it into an order! And this is the easiest step in the pipeline, done by the function [signal\\_to\\_size\\_nb](#), which takes the four signals (three of which are now deactivated) and the size requirement under this row and column (i.e., for this element), and returns the order size, size type, and direction to be requested. For example, being in a position of 20 shares and receiving the long exit signal, the size becomes minus 20 shares, the size type becomes [SizeType.Amount](#), and the direction becomes [Direction.LongOnly](#):

```
>>> vbt.pf_nb.signal_to_size_nb(
...     position_now=20,
...     val_price_now=20594.29, ❶
...     value_now=411885.80, ❷
...     is_long_entry=False,
...     is_long_exit=True,
...     is_short_entry=False,
...     is_short_exit=False,
...     size=0.1, ❸
...     size_type=vbt.pf_enums.SizeType.ValuePercent, ❹
...     accumulate=False ❺
... )
(-20.0, 0, 0)
```

- ❶ The latest asset price known
- ❷ The latest group value known
- ❸ Default value for this element
- ❹ Default value for this element
- ❺ Default value for this element

Even though we provided the function with the default order specification for the current element, such as `size`, the function didn't use it because it isn't required for closing the current position. On the other hand, if we wanted to reverse the current position (that is, close it and then order using the default specification), those inputs would suddenly become effective:

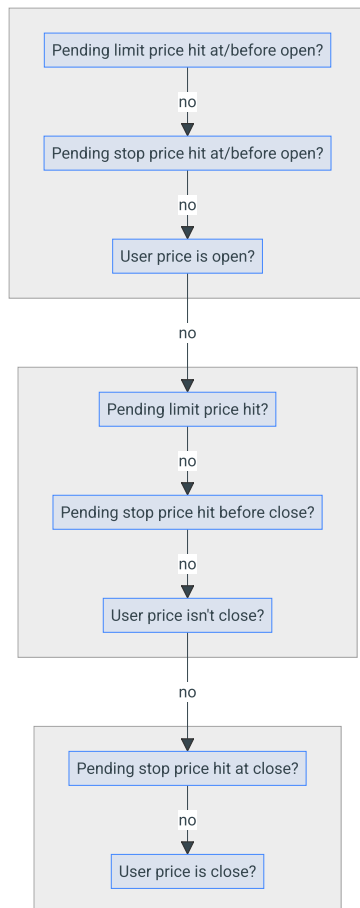
```
>>> vbt.pf_nb.signal_to_size_nb(
...     position_now=20,
...     val_price_now=20594.29,
...     value_now=411885.80,
...     is_long_entry=False,
...     is_long_exit=False,
...     is_short_entry=True, ❶
...     is_short_exit=False,
...     size=0.1,
...     size_type=vbt.pf_enums.SizeType.ValuePercent,
...     accumulate=False
... )
(-22.0, 0, 2)
```

- ❶ Reverse the long position

The size is calculated as follows: reduce the number of shares by 20 to close out the long position, and, given that we're operating with a percentage of the current group value, open a new short position of  $\text{size} * \text{value\_now} / \text{val\_price\_now} = 2.0$  shares. The size type is [SizeType.Amount](#) while the direction is [Direction.Both](#) because the operation now involves two directions.

## Main order resolution

The simulator called the signal function, resolved the incoming signals, and converted them into an order specification. But this is not the only order that competes for the current bar: there may be also pending limit and stop orders. Since the FS simulation function can process at most one order at each bar, it has to decide for a winner, which should always be an order that executes first. But how do we know which one comes first if we don't have any intra-bar data? We can still divide each bar into three "zones": opening (first rectangle below), somewhere in-between (second rectangle below), and closing (third rectangle below). For example, if a stop order was hit at or before the opening of the current bar and the user order should execute at the close price, then clearly the stop order should be first on the line. Here's the full decision chain:



As we see, limit orders have priority over stop orders, while stop orders have priority over user orders, but only if they are triggered within the same zone of a bar.

### Limit management

While a market order is a transaction meant to execute as quickly as possible at the current market price, a limit order is an order to buy or sell an asset with a restriction on the maximum price to be paid or the minimum price to be received (the "limit price"). The price of a limit order is getting compared against a pre-defined price level. Until this level is reached, the order is marked as pending, and it will not be filled if the price does not reach this level.

#### Creation

Whenever a stop or user-defined order goes through and its order type provided via `order_type` is `OrderType.Limit`, the simulator first determines what kind of limit price the order should be executed at: open, close, or something else? And this is a very important concept to grasp: the argument `price` gives vectorbt hints on where in the bar the operation should take place. If the limit price is the open price (provided as either `PriceType.Open` or `-np.inf`), the simulator is allowed to use the entire candle for its checks and execute the order right upon detecting the price being hit, at the same bar. If the limit price is not the close price but something in-between, the simulator is allowed to use the close price only. If the limit price is the close price (provided as either `PriceType.Close` or `np.inf`), the simulator is not allowed to execute the limit order right away - it's forced to postpone its very first check to the next bar.

If the limit order couldn't be executed at the same bar as it was created, the order is marked as pending and all the relevant information becomes stored in a record array of the type `limit_info_dt`, which is structured by asset. This array can keep only one instance per asset, thus `FS` allows only one limit order to be active at a time. In a signal function, this array can be accessed via `c.limit_info_dt`, allowing us to change any information before the new bar. For example, to change the price: `c.limit_info_dt["init_price"][c.col] = new_price`.

#### Expiration

Once the simulator hits the next bar, it first uses `check_limit_expired_nb` to check whether the pending limit order expired at the beginning or somewhere in the middle of the bar. If the former, the order is thrown away. If the latter, the simulator also checks whether the order was hit at the beginning of the bar (and execute it), and if not, throw it away because there is no guarantee that the order was hit before the deadline. For example, let's assume that the order can be at most 36 hours in force, it was issued at the day `2020-01-01`, and now is the day `2020-01-02`:

```

>>> vbt.pf_nb.check_limit_expired_nb(
...     creation_idx=0,
...     i=1,
...     tif=vbt.dt.to_ns(vbt.timedelta("36h")), ❶
...     index=vbt.dt.to_ns(vbt.date_range("2020-01-01", periods=3)),
...     freq=vbt.dt.to_ns(vbt.timedelta("1d"))
... )
(False, True)
  
```

❶ Index and time deltas must be converted into nanoseconds

We see that the function marked the order as expired, but not at the beginning of the bar such that it can still be executed using the open price. But if the lifespan of the order was 24 hours, the function would also raise the first flag and disallow any execution:

```
>>> vbt.pf_nb.check_limit_expired_nb(
...     creation_idx=0,
...     i=1,
...     tif=vbt.dt.to_ns(vbt.timedelta("24h")),
...     index=vbt.dt.to_ns(vbt.date_range("2020-01-01", periods=3)),
...     freq=vbt.dt.to_ns(vbt.timedelta("1d"))
... )
(True, True)
```

#### Info

The lifespan is calculated by subtracting any time from the opening time of the creation bar, even if the order was placed at the very end of the creation bar.

#### Activation

Once we're sure that the order **can** be executed at this bar (i.e., it won't expire), the simulator uses the function `check_limit_hit_nb` to check whether the order **should** be executed by determining whether its target price has been hit. This is implemented through comparison of the price against the current candle. For example, if we have a pending buy limit order with a target price of `9.5`, the function will check whether the low price went below the target price:

```
>>> vbt.pf_nb.check_limit_hit_nb(
...     open=10.0,
...     high=11.0,
...     low=9.0,
...     close=10.5,
...     price=9.5,
...     size=2.0
... )
(9.5, False, True)
```

If the target price was `11`, the function would notify us about the price being hit already at the beginning of the bar; in such a case, the order will be executed right away using the open price:

```
>>> vbt.pf_nb.check_limit_hit_nb(
...     open=10.0,
...     high=11.0,
...     low=9.0,
...     close=10.5,
...     price=11.0,
...     size=2.0
... )
(10.0, True, True)
```

#### Cancellation

If the target price hasn't been hit, the limit order remains pending. It can still be cancelled manually in the signal function called before all the checks above, or in the post-segment function called after processing the entire segment. The pending order will also be cancelled automatically once any stop order gets executed since the latter may change the simulation state and potentially pull the resources required to execute the former in the future.

Finally, the four signals returned by the signal function and resolved into a single signal also can affect the pending order, regardless of whether the final signal gets executed or not. Consider the example where we have a pending buy limit order and the user decides to issue the long exit or short entry signal; in this case, the most intuitive reaction would be cancelling the pending order since the user have changed their mind. Exactly this is happening by default. Such "pending conflicts" are resolved using the function `resolve_pending_conflict_nb`, which uses the arguments `upon_adj_limit_conflict` and `upon_opp_limit_conflict`, both of the type `PendingConflictMode`, to decide what to do if the direction of the pending order is adjacent and opposite respectively to the direction of the resolved user-defined signal.

```
>>> vbt.pf_nb.resolve_pending_conflict_nb(
...     is_pending_long=True,
...     is_user_long=False,
...     upon_adj_conflict=vbt.pf_enums.PendingConflictMode.KeepIgnore,
...     upon_opp_conflict=vbt.pf_enums.PendingConflictMode.CancelIgnore,
... )
(False, False)
```

Here, the function decided to cancel the limit order and to ignore the user-defined signal.

#### Stop management

Stop orders help to ensure a higher probability of achieving a predetermined entry or exit price, limiting the investor's loss, or locking in a profit. They remain dormant until a certain price is passed, at which time they are activated as a market or limit order. Execution of a stop order usually closes out the position.

#### Types

There are four types of stop orders:

1. Stop loss ([SL](#))
2. Trailing stop loss ([TSL](#))
3. Trailing take profit ([TTP](#))
4. Take profit ([TP](#))

By using a stop-loss order, we're limiting our risk in the trade to a set amount in the event that the market moves against us. For instance, if a stop-loss sell order were placed at \$45 per unit, the order would be inactive until the price reached or dropped below \$45. The order would then be transformed into a market or limit order, and the units would be sold at the best available price. A take profit is pretty much the exact opposite. It expresses how much we're willing to make as a profit with one trade and close it once we're happy with the amount. A combination of an SL and TP order creates a certain risk-to-reward ratio, which can be further tuned to respect the odds of reaching each certain breakout scenario.

A different bread are trailing orders: when the price increases, it drags the trailing stop along with it. Then, when the price finally stops rising, the new stop-loss price remains at the level it was dragged to, thus automatically protecting our downside, while locking in profits as the price reaches new highs. Also, TTP is a version of TSL that gets activated after a certain threshold. These two orders are usually viewed and represented as a single order.

### Creation

In contrast to limit orders, stop orders are created after an entry order has been filled, and behave identically to user-defined exit signals that are triggered once a stop condition has been met. An entry order is any successfully-filled order that has opened a new position or increased an existing one. Once the simulator detects such an order, it first resolves the stop entry price provided via the argument `stop_entry_price` of the type **StopEntryPrice**. This is the initial price to which all the stop values and thresholds are applied.

#### Note

By default, the stop entry price is the closing price, not the order price, to avoid a scenario where the stop is getting hit at the very first bar but cannot be executed since we don't have intra-bar data and cannot execute two orders at the same bar. By using the order price, the earliest time the stop can execute at is the opening of the next bar.

Based on this price, the simulator can also determine where exactly in the bar the stop order should be triggered. Why is this important? Because the simulator needs to know whether it can already use the current candle to update the price of any TSL and TTP order. Internally, the information on stop orders is stored inside three arrays of the data types `sl_info_dt`, `tsl_info_dt`, and `tp_info_dt`, laid out per asset. Each data type has a similar schema: the initial row and price, the current stop value (either in absolute or percentage terms), and the limit delta and its format if the stop order should ultimately trigger a limit order. The trailing stops also include the newly updated price and the row of the update.

### Activation

Stop orders cannot be activated at the same bar as they were issued, even if the entry price is the opening price. This is because FS cannot handle two orders at the same bar (if there is a need for this, use a flexible order function with **Portfolio.from\_order\_func**). After hitting a new bar, the price of any pending SL and TP order then gets checked against the low and high price of the current candle respectively (the other way round for short positions), using the function **check\_stop\_hit\_nb**, which returns the stop price, whether it happened on open, and whether it happened at all. For example, let's imagine that the initial price is \$10 per unit and the stop loss sits at 10%. For the stop to be marked as hit, the lowest price of this candle must be under  $10 * (1 - 0.1) = 9$  per unit:

```
>>> vbt.pf_nb.check_stop_hit_nb(
...     open=10.0,
...     high=11.0,
...     low=9.0,
...     close=10.5,
...     is_position_long=True,
...     init_price=10.0,
...     stop=0.1
... )
(9.0, False, True)
```

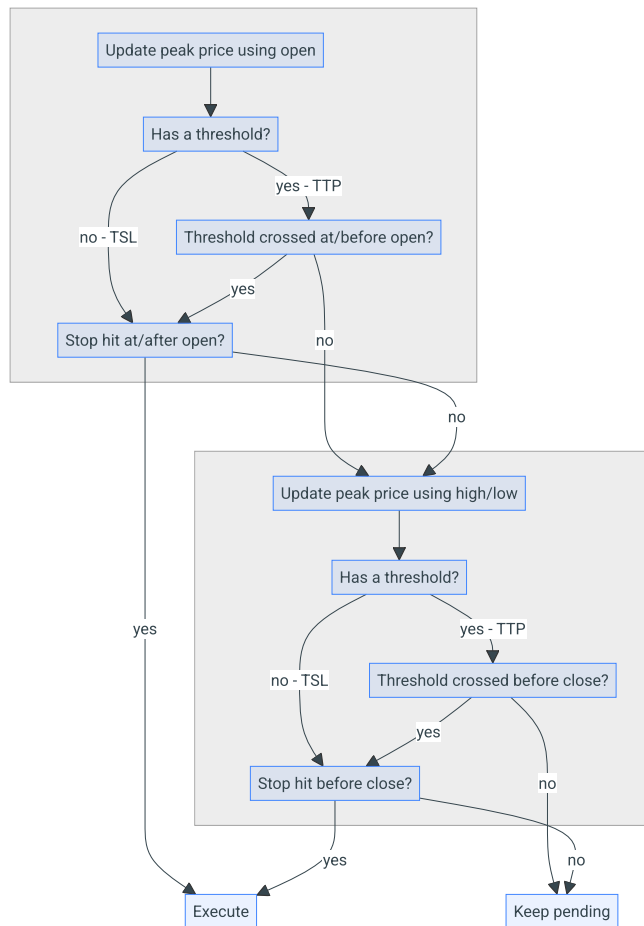
But if the initial price was at \$12 per unit, the stop would match already at open:

```
>>> vbt.pf_nb.check_stop_hit_nb(
...     open=10.0,
...     high=11.0,
...     low=9.0,
...     close=10.5,
...     is_position_long=True,
...     init_price=12.0,
...     stop=0.1
... )
(10.0, True, True)
```

To check a TP order, we would need to set the argument `hit_below` to `False`.

As opposed to fixed stop orders, TSL and TTP orders additionally need to track the peak price the stop price is based upon. Since we have no information on whether the highest price of a candle comes before the lowest price or vice versa, we need to split the candle into distinct zones and update the peak price in a zone that precedes that of the stop check. First, the simulator uses the opening price to update the peak price. Then, it checks whether the stop has been hit during the entire bar. If not, it proceeds to update the peak price using the highest (for long positions) or lowest (for short positions) price of the candle, and performs the stop check again, but now using the closing price alone to avoid the ambiguity that's been mentioned before. This way, the simulator always pessimistically assumes that the worst event (where the stop is being hit) happens before the best event (where the peak price is being updated).

To make the matter even more complex, a TTP order additionally needs to check whether its threshold has been crossed to be properly activated. The check is performed using the function **check\_tsl\_th\_hit\_nb**: it takes the initial and the peak price, and checks whether the difference between them is greater than or equal to the threshold. If so, the order gets converted into a regular TSL order. But if the difference is smaller, the simulator proceeds to update the peak price using the current candle, and makes a second attempt to check for the threshold crossover. If the difference is finally greater, it uses **check\_stop\_hit\_nb**, but it cannot use the current candle anymore since it's unknown whether the stop has been hit after or before the threshold has been crossed, thus only the closing price is used by disabling the argument `can_use_ohlc`. Here's a depiction of what's happening:



What happens if multiple stops have been hit? The simulator pessimistically assumes that **SL** comes first, **TSL** and **TTP** come second, and **TP** comes last. The pending stop that comes before others gets executed while the remaining pending stops get canceled.

#### Resolution

After that, the winner gets converted into the four signals using the function `generate_stop_signal_nb`, which is based on the current position and the default stop exit behavior defined using the argument `stop_exit_type` of the type `StopExitType`. For example, instead of closing out the position, we can make the function reverse the position by using `StopExitType.Reverse`:

```
>>> vbt.pf_nb.generate_stop_signal_nb(
...     position_now=20,
...     stop_exit_type=vbt.pf_enums.StopExitType.Reverse
... )
(False, False, True, False, 0)
```

As we can see, the short entry signal is `True` while other signals are `False`. The number after the signals is the selected accumulation mode of the type `AccumulationMode` for a case where we want to reduce the position instead of closing out or reversing it. What's left is the resolution of the stop exit price, done by the function `resolve_stop_exit_price_nb`. The logic is quite simple: if the argument `stop_exit_price` of the type `StopExitPrice` is `StopExitPrice.Close`, use the closing price, otherwise use the stop price that has been hit. The argument can also be provided as an actual price.

```
>>> vbt.pf_nb.resolve_stop_exit_price_nb(
...     stop_price=9.0,
...     close=10.5,
...     stop_exit_price=vbt.pf_enums.StopExitPrice.Stop
... )
9.0

>>> vbt.pf_nb.resolve_stop_exit_price_nb(
...     stop_price=9.0,
...     close=10.5,
...     stop_exit_price=9.5
... )
9.5
```

Finally, the order signal gets converted into a market or limit order specification the same way as a user-defined signal. See [Signal conversion](#).

#### Updating

Apart from the possibility to update any stop in a callback, there is a possibility to update the stop automatically once the current position has increased. In such a case, the argument `upon_stop_update` of the type `StopUpdateMode` controls whether any current stop should remain or be reset. This decision is done by the

function `should_update_stop_nb`. For example, our position has increased, and we want to know whether the current stop should be updated given the option

`StopUpdateMode.Override`:

```
>>> vbt.pf_nb.should_update_stop_nb(
...     new_stop=0.1,
...     upon_stop_update=vbt.pf_enums.StopUpdateMode.Override
... )
True
```

But if the new stop value is NaN (i.e., no stop), we shouldn't update:

```
>>> vbt.pf_nb.should_update_stop_nb(
...     new_stop=np.nan,
...     upon_stop_update=vbt.pf_enums.StopUpdateMode.Override
... )
False
```

Unless the option is `StopUpdateMode.OverrideNaN`, which will effectively disable all stops:

```
>>> vbt.pf_nb.should_update_stop_nb(
...     new_stop=np.nan,
...     upon_stop_update=vbt.pf_enums.StopUpdateMode.OverrideNaN
... )
True
```

This won't apply to the case where the current position has decreased. But why should we care about updates if signals usually either open or close positions, and don't increase or decrease them? When using accumulation, signals can add to the position or remove from the position. In this case, we should ask ourselves: should this position change invalidate the stops we defined previously, or define new ones? The argument `upon_stop_update` controls this behavior.

### Cancellation

Similarly to updating, cancellation of the currently pending stop orders happens when the position has been closed out. This will clear all the stops automatically. But also similarly to limit orders, there is a possibility of a conflict with an active user-defined signal, which is resolved using the function `resolve_pending_conflict_nb`. The arguments that are used to resolve any pending conflicts for stop orders are `upon_adj_stop_conflict` and `upon_opp_stop_conflict`, both of the type `PendingConflictMode`. For example, if the user has decided to reduce the position and clear all the pending stops along with it, they can set the argument `upon_adj_stop_conflict` (reducing the position and attempting to close it are considered adjacent signals) to the option `PendingConflictMode.CancelExecute`.

## Signals

We've covered some theory on how this simulation method works. Let's take a break from reading and concern ourselves with signal arrays, which together with a signal function are the main input to this method. As we already know, signals come in two flavors:

1. Direction-unaware signals: entries and exits enhanced by direction
2. Direction-aware signals: Long entries, long exits, short entries, short exits

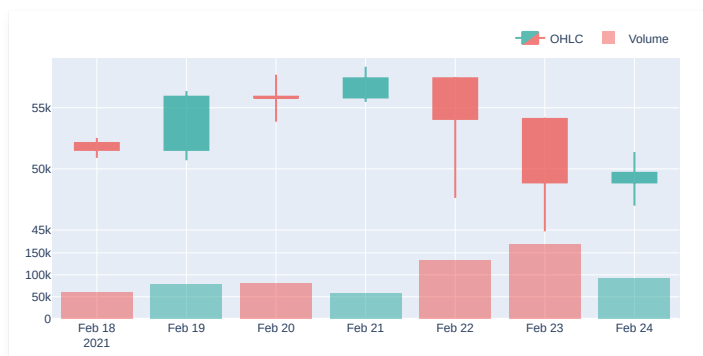
The first flavor is a compressed form of the second flavor, so to say: we can always convert direction-unaware signals into direction-aware, but not the other way round since the first format defines a total of  $2 * 2 * 3 = 12$  combinations and the second format defines a total of  $2 * 2 * 2 * 2 = 16$  combinations. On the other hand, the first format is easier to use since we can set the direction globally and just operate with two arrays instead of four.

But first, let's fetch the entire history of `BTCUSD` and `ETHUSD` for our examples below:

```
>>> data = vbt.BinanceData.pull(["BTCUSD", "ETHUSD"])
```

As we won't need the entire history to illustrate most concepts, let's select the week of data between the 18th and 24th February 2021, where there is a substantial change in price in both directions:

```
>>> sub_data = data.loc["2021-02-18":"2021-02-24"]
>>> sub_data.plot(symbol="BTCUSD").show()
```



Let's try passing the data without signals:

```
>>> pf = vbt.Portfolio.from_signals(sub_data) ❶
>>> pf.orders.count()
symbol
BTCUSDT    0
ETHUSDT    0
Name: count, dtype: int64
```

- ❶ Even though the first argument expects the closing price (`close`), we can pass the entire data instance, from which the OHLC features will be extracted automatically

By default, all signals are set to `False`, thus no orders were generated.

Now, let's assume that our ML model has correctly predicted the peak on the 21st February and ordered us to enter a position on the 18th February and close it on the 21st February. For this, we need to build our entry and exit arrays of the same shape as our data. But instead of specifying the same signals for each asset redundantly, we can provide a Series instead of a DataFrame, which will be applied to each asset thanks to **broadcasting**:

```
>>> entries = pd.Series([X, 0, 0, 0, 0, 0, 0])
>>> exits = pd.Series([0, 0, 0, X, 0, 0, 0])
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=entries, ❶
...     exits=exits
... )
>>> pf.orders.readable
```

| Order | Id | Column  | Signal Index              | Creation Index            | \ |
|-------|----|---------|---------------------------|---------------------------|---|
| 0     |    | BTCUSDT | 2021-02-18 00:00:00+00:00 | 2021-02-18 00:00:00+00:00 |   |
| 1     | 1  | BTCUSDT | 2021-02-21 00:00:00+00:00 | 2021-02-21 00:00:00+00:00 |   |
| 2     | 0  | ETHUSDT | 2021-02-18 00:00:00+00:00 | 2021-02-18 00:00:00+00:00 |   |
| 3     | 1  | ETHUSDT | 2021-02-21 00:00:00+00:00 | 2021-02-21 00:00:00+00:00 |   |

|   | Fill Index                | Size     | Price    | Fees | Side | Type   | Stop | Type |
|---|---------------------------|----------|----------|------|------|--------|------|------|
| 0 | 2021-02-18 00:00:00+00:00 | 0.001940 | 51552.60 | 0.0  | Buy  | Market |      | None |
| 1 | 2021-02-21 00:00:00+00:00 | 0.001940 | 57408.57 | 0.0  | Sell | Market |      | None |
| 2 | 2021-02-18 00:00:00+00:00 | 0.051557 | 1939.61  | 0.0  | Buy  | Market |      | None |
| 3 | 2021-02-21 00:00:00+00:00 | 0.051557 | 1933.53  | 0.0  | Sell | Market |      | None |

- ❶ We could have also specified it as a two-dimensional NumPy array with one column

We can see that the first order in the column `BTCUSDT` is a buy market order that opened a new long position. The second order is of the same size but the opposite side, thus it was used to close out the long position. Reading orders isn't always straight-forward, especially when it comes to determining when positions are opened and closed. To get a better overview, let's calculate and print the position for each symbol at each bar:

```
>>> pf.assets
```

| symbol                    | BTCUSDT | ETHUSDT  |
|---------------------------|---------|----------|
| Open time                 |         |          |
| 2021-02-18 00:00:00+00:00 | 0.00194 | 0.051557 |
| 2021-02-19 00:00:00+00:00 | 0.00194 | 0.051557 |
| 2021-02-20 00:00:00+00:00 | 0.00194 | 0.051557 |
| 2021-02-21 00:00:00+00:00 | 0.00000 | 0.000000 |
| 2021-02-22 00:00:00+00:00 | 0.00000 | 0.000000 |
| 2021-02-23 00:00:00+00:00 | 0.00000 | 0.000000 |
| 2021-02-24 00:00:00+00:00 | 0.00000 | 0.000000 |

The returned array represents the position at the end of each bar, hence we're still in the market on the 20th February but went out of the market on the 21st February.

We provided the same array for each symbol, but what if our ML model told us that the peak for `ETHUSDT` is one day ahead of that for `BTCUSDT`? As soon as our signal specification starts varying with columns, we need to build the respective signal array as a DataFrame with values defined per element. Let's keep the entry array the same for both symbols (entry signals do not vary with columns in our case) and only expand the exit array:

```
>>> entries = pd.Series([X, 0, 0, 0, 0, 0, 0])
>>> exits = pd.DataFrame({
...     0: [0, 0, 0, X, 0, 0, 0],
...     1: [0, 0, X, 0, 0, 0, 0],
... }) ❶
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=entries,
...     exits=exits
... )
>>> pf.assets
```

| symbol                    | BTCUSDT | ETHUSDT  |
|---------------------------|---------|----------|
| Open time                 |         |          |
| 2021-02-18 00:00:00+00:00 | 0.00194 | 0.051557 |
| 2021-02-19 00:00:00+00:00 | 0.00194 | 0.051557 |
| 2021-02-20 00:00:00+00:00 | 0.00194 | 0.000000 |
| 2021-02-21 00:00:00+00:00 | 0.00000 | 0.000000 |
| 2021-02-22 00:00:00+00:00 | 0.00000 | 0.000000 |
| 2021-02-23 00:00:00+00:00 | 0.00000 | 0.000000 |
| 2021-02-24 00:00:00+00:00 | 0.00000 | 0.000000 |

- ❶ We could have also used symbols as column names. Defining rows and columns as a simple range of values (from `0` to `n`) will ignore the labels and broadcast only using shapes.

We can now observe that the long position in the column `ETHUSDT` was cleared one day before that of `BTCUSDT`, as our imaginary model wished. To make array creation a bit easier and to avoid setting each element manually, we can use the symbol wrapper of the data instance to create empty boolean arrays of the same shape as the data, and fill them on specific dates:



```
>>> exits = sub_data.symbol_wrapper.fill(False) ❶
>>> exits.loc["2021-02-21", "BTCUSD"] = True
>>> exits.loc["2021-02-20", "ETHUSD"] = True
>>> exits
symbol          BTCUSD  ETHUSD
Open time
2021-02-18 00:00:00+00:00  False  False
2021-02-19 00:00:00+00:00  False  False
2021-02-20 00:00:00+00:00  False   True
2021-02-21 00:00:00+00:00   True  False
2021-02-22 00:00:00+00:00  False  False
2021-02-23 00:00:00+00:00  False  False
2021-02-24 00:00:00+00:00  False  False
```

❶ Create an array with the same number of columns as we have symbols, and fill it with `False`

For readers who appreciate hot features, here's how to let the vectorbt's broadcaster create both arrays and fill them dynamically using [index dictionaries](#):

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=vbt.index_dict({0: True}),
...     exits=vbt.index_dict({
...         vbt.idx("2021-02-21", "BTCUSD"): True,
...         vbt.idx("2021-02-20", "ETHUSD"): True
...     })
... )
>>> pf.assets
symbol          BTCUSD  ETHUSD
Open time
2021-02-18 00:00:00+00:00  0.00194  0.051557
2021-02-19 00:00:00+00:00  0.00194  0.051557
2021-02-20 00:00:00+00:00  0.00194  0.000000
2021-02-21 00:00:00+00:00  0.00000  0.000000
2021-02-22 00:00:00+00:00  0.00000  0.000000
2021-02-23 00:00:00+00:00  0.00000  0.000000
2021-02-24 00:00:00+00:00  0.00000  0.000000
```

The best in the approach above is that the broadcaster won't create arrays bigger than necessary: it will notice that the entry specification is the same for both symbols and create an array with one column instead of two, saving us some memory.

## Direction-unaware

In all the examples above, we provided only two arrays: `entries` and `exits`. Whenever this happens, the method treats the provided signals as direction-unaware, meaning that an additional argument `direction` is used to control their direction. By default, the direction is `Direction.LongOnly` (see [signal\\_direction](#) in [portfolio settings](#)). To change the direction, override `direction` with any option available in [Direction](#). An option can be provided either as an integer or a field name. For example, let's allow *both* directions to reverse the position on exit instead of just closing it out:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=entries,
...     exits=exits,
...     direction="both" ❶
... )
>>> pf.assets
symbol          BTCUSD  ETHUSD
Open time
2021-02-18 00:00:00+00:00  0.00194  0.051557
2021-02-19 00:00:00+00:00  0.00194  0.051557
2021-02-20 00:00:00+00:00  0.00194 -0.051557
2021-02-21 00:00:00+00:00 -0.00194 -0.051557
2021-02-22 00:00:00+00:00 -0.00194 -0.051557
2021-02-23 00:00:00+00:00 -0.00194 -0.051557
2021-02-24 00:00:00+00:00 -0.00194 -0.051557
```

❶ Can also be provided in the numeric format as `Direction.Both`

Zero values have changed to negative values, which means that positions are now being reversed. Similarly to the arguments for signals, the argument for `direction` can also be provided as an array. This makes possible defining different directions for different dates and symbols. For example, let's long `BTCUSD` and short `ETHUSD`. Thanks to broadcasting, to provide any information per column, we can represent it as a two-dimensional array with just one row:

```
>>> direction = pd.DataFrame([["longonly", "shortonly"]])
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=entries,
...     exits=exits,
...     direction=direction
... )
>>> pf.assets
symbol          BTCUSD  ETHUSD
Open time
2021-02-18 00:00:00+00:00  0.00194 -0.051557
2021-02-19 00:00:00+00:00  0.00194 -0.051557
2021-02-20 00:00:00+00:00  0.00194  0.000000
2021-02-21 00:00:00+00:00  0.00000  0.000000
2021-02-22 00:00:00+00:00  0.00000  0.000000
2021-02-23 00:00:00+00:00  0.00000  0.000000
2021-02-24 00:00:00+00:00  0.00000  0.000000
```

The position values under the column `ETHUSD` have turned negative indicating that we're in a short position. The following example illustrates the usage of direction defined per element by entering a long position at the first bar, exiting it at the peak, then entering a short one at the next bar, and finally exiting it at the last bar:

```
>>> L = vbt.pf_enums.Direction.LongOnly ❶
>>> S = vbt.pf_enums.Direction.ShortOnly

>>> pf = vbt.Portfolio.from_signals(
...     sub_data.select("BTCUSD"), ❷
...     entries= pd.Series([X, 0, 0, 0, X, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, X]),
...     direction=pd.Series([L, L, L, L, S, S, S])
... )
>>> pf.assets
Open time
2021-02-18 00:00:00+00:00    0.001940
2021-02-19 00:00:00+00:00    0.001940
2021-02-20 00:00:00+00:00    0.001940
2021-02-21 00:00:00+00:00    0.000000
2021-02-22 00:00:00+00:00   -0.002059
2021-02-23 00:00:00+00:00   -0.002059
2021-02-24 00:00:00+00:00    0.000000
Freq: D, dtype: float64
```

❶ Shortcuts for better readability, numeric format

❷ Select one symbol of data

#### Note

The numeric format should be preferred over the string format for bigger arrays since strings must be converted to integers prior to the simulation, which is a relatively slow operation.

## Direction-aware

Direction-aware signals are a more flexible form of signals as they allow for more signal combinations. To switch to this mode, we need to provide the arguments `short_entries` and `short_exits` acting as short signals along with the arguments `entries` and `exits` acting as long signals. This will disable the `direction` argument completely as the direction is now expressed by the signals themselves. Let's adapt the example above:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data.select("BTCUSD"),
...     entries= pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     short_entries=pd.Series([0, 0, 0, 0, X, 0, 0]),
...     short_exits= pd.Series([0, 0, 0, 0, 0, 0, X]),
... )
>>> pf.assets
Open time
2021-02-18 00:00:00+00:00    0.001940
2021-02-19 00:00:00+00:00    0.001940
2021-02-20 00:00:00+00:00    0.001940
2021-02-21 00:00:00+00:00    0.000000
2021-02-22 00:00:00+00:00   -0.002059
2021-02-23 00:00:00+00:00   -0.002059
2021-02-24 00:00:00+00:00    0.000000
Freq: D, dtype: float64
```

So, when do we use which signal mode? Use direction-unaware signals when working with a single direction throughout the entire column, and direction-aware signals for more granular decisions, especially when positions need to be closed when both directions are allowed, for example, to close out any position at the end of a day.

## Signal function

Passing signals as pre-defined arrays has one major advantage: caching. Even after we restart the runtime, there won't any recompilation when we pass signal arrays of the same format again. But sometimes there is a need to trade in a bit of performance for more flexibility. Such use cases include path-dependency scenarios where signals depend on the previous or current simulation state such that they cannot be generated in advance. Another use case is to reduce RAM usage: the entire indicator and signal generation logic can be encapsulated into a single signal function such that there is no need for intermediate arrays anymore. This is useful when a huge number of parameters needs to be tested, or when the user wants to choose a number of assets to trade out of a big universe of assets, which usually involves keeping very wide arrays in RAM but using a signal function would make them redundant.

Let's implement the latest example above but without any arrays!

```
>>> @njit
... def signal_func_nb(c):
...     ts = c.index[c.i] ❶
...     if vbt.dt_nb.matches_date_nb(ts, 2021, 2, 18): ❷
...         return True, False, False, False ❸
...     if vbt.dt_nb.matches_date_nb(ts, 2021, 2, 21):
...         return False, True, False, False
...     if vbt.dt_nb.matches_date_nb(ts, 2021, 2, 22):
...         return False, False, True, False
...     if vbt.dt_nb.matches_date_nb(ts, 2021, 2, 24):
...         return False, False, False, True
...     return False, False, False, False

>>> pf = vbt.Portfolio.from_signals(
...     sub_data.select("BTCUSD"),
```

```
...     signal_func_nb=signal_func_nb
... )
>>> pf.assets
Open time
2021-02-18 00:00:00+00:00    0.001940
2021-02-19 00:00:00+00:00    0.001940
2021-02-20 00:00:00+00:00    0.001940
2021-02-21 00:00:00+00:00    0.000000
2021-02-22 00:00:00+00:00   -0.002059
2021-02-23 00:00:00+00:00   -0.002059
2021-02-24 00:00:00+00:00    0.000000
Freq: D, dtype: float64
```

- 1 `c.index` returns an array with timestamps in the nanosecond format while `c.i` returns the current row. By applying the latter on the former, we can get the current timestamp.
- 2 Use `matches_date_nb` to check whether the current day matches a date
- 3 Signal functions must return a set of direction-aware signals

We've switched a vectorized logic in favor of an iterative logic, which is usually more verbose but also much more flexible and similar to the format used in most open-source backtesting frameworks. But that doesn't mean that we have to define everything iteratively, we can still pass one or more arrays and make decisions based on them. All we have to do is to make the signal function accept arrays as positional arguments and select one element out of them at each time step to generate the four signals, and then pass the actual arrays to `FS` as a tuple using `signal_args`. Just note that any array-like object must be a NumPy array since Numba doesn't understand Pandas.

```
>>> @njit
... def signal_func_nb(c, entries, exits, short_entries, short_exits):
...     long_entry = entries[c.i]
...     long_exit = exits[c.i]
...     short_entry = short_entries[c.i]
...     short_exit = short_exits[c.i]
...     return long_entry, long_exit, short_entry, short_exit

>>> pf = vbt.Portfolio.from_signals(
...     sub_data.select("BTCUSD"),
...     signal_func_nb=signal_func_nb,
...     signal_args=(
...         pd.Series([X, 0, 0, 0, 0, 0, 0]).values,
...         pd.Series([0, 0, 0, X, 0, 0, 0]).values,
...         pd.Series([0, 0, 0, 0, X, 0, 0]).values,
...         pd.Series([0, 0, 0, 0, 0, 0, X]).values
...     )
... )
>>> pf.assets
Open time
2021-02-18 00:00:00+00:00    0.001940
2021-02-19 00:00:00+00:00    0.001940
2021-02-20 00:00:00+00:00    0.001940
2021-02-21 00:00:00+00:00    0.000000
2021-02-22 00:00:00+00:00   -0.002059
2021-02-23 00:00:00+00:00   -0.002059
2021-02-24 00:00:00+00:00    0.000000
Freq: D, dtype: float64
```

But what if we wanted to expand our data to multiple assets? The example above would only work if each of the arrays is kept as one-dimensional since we only select rows in the signal function. To make our logic shape-agnostic though, we need to make each of the arrays two-dimensional and additionally select the current column in the signal function. But that's not enough: we also need to take care of broadcasting, which can be flexibly done in the signal function either manually with `flex_select_nb` or automatically with `select_nb`:

```
>>> @njit
... def signal_func_nb(c, entries, exits, short_entries, short_exits):
...     long_entry = vbt.pf_nb.select_nb(c, entries)
...     long_exit = vbt.pf_nb.select_nb(c, exits)
...     short_entry = vbt.pf_nb.select_nb(c, short_entries)
...     short_exit = vbt.pf_nb.select_nb(c, short_exits)
...     return long_entry, long_exit, short_entry, short_exit

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     signal_func_nb=signal_func_nb,
...     signal_args=(
...         vbt.to_2d_array(pd.Series([X, 0, 0, 0, 0, 0, 0])),
...         vbt.to_2d_array(pd.Series([0, 0, 0, X, 0, 0, 0])),
...         vbt.to_2d_array(pd.Series([0, 0, 0, 0, X, 0, 0])),
...         vbt.to_2d_array(pd.Series([0, 0, 0, 0, 0, 0, X]))
...     )
... )
>>> pf.assets
symbol          BTCUSD  ETHUSD
Open time
2021-02-18 00:00:00+00:00  0.001940  0.051557
2021-02-19 00:00:00+00:00  0.001940  0.051557
2021-02-20 00:00:00+00:00  0.001940  0.051557
2021-02-21 00:00:00+00:00  0.000000  0.000000
2021-02-22 00:00:00+00:00 -0.002059 -0.056000
2021-02-23 00:00:00+00:00 -0.002059 -0.056000
2021-02-24 00:00:00+00:00  0.000000  0.000000
```

Our strategy can now be applied on an arbitrary number of columns, great! But even this isn't the most flexible design 😊 What if the user provides a signal array that doesn't have the same number of rows as the data? If bound checking is enabled, we would get the "index is out of bounds" error since the signal function

would attempt to select an element that simply doesn't exist. To make any array broadcast against the data prior to the simulation, we can define it in the dictionary `broadcast_named_args`, and then use a template to substitute its name by the broadcasted array in `signal_args`:

```
>>> entries = pd.Series({vbt.utc_timestamp("2021-02-18"): True})
>>> exits = pd.Series({vbt.utc_timestamp("2021-02-21"): True})
>>> short_entries = pd.Series({vbt.utc_timestamp("2021-02-22"): True})
>>> short_exits = pd.Series({vbt.utc_timestamp("2021-02-24"): True})

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     signal_func_nb=signal_func_nb,
...     signal_args=(
...         vbt.Rep("entries"), ❶
...         vbt.Rep("exits"),
...         vbt.Rep("short_entries"),
...         vbt.Rep("short_exits")
...     ),
...     broadcast_named_args=dict(
...         entries=entries,
...         exits=exits,
...         short_entries=short_entries,
...         short_exits=short_exits
...     )
... )
>>> pf.assets
symbol      BTCUSDT  ETHUSDT
2021-02-18 00:00:00+00:00  0.001940  0.051557
2021-02-19 00:00:00+00:00  0.001940  0.051557
2021-02-20 00:00:00+00:00  0.001940  0.051557
2021-02-21 00:00:00+00:00  0.000000  0.000000
2021-02-22 00:00:00+00:00 -0.002059 -0.056000
2021-02-23 00:00:00+00:00 -0.002059 -0.056000
2021-02-24 00:00:00+00:00  0.000000  0.000000
```

❶ `entries` is recognized as a key in `broadcast_named_args`.

Our setup now behaves the same way as the built-in arguments `entries`, `exits`, `short_entries`, and `short_exits` 🚀 We don't even have to convert them to NumPy arrays anymore since this is being taken care of automatically by the broadcaster. It allows us also to use index dictionaries and other hot broadcasting features:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     signal_func_nb=signal_func_nb,
...     signal_args=(
...         vbt.Rep("entries"),
...         vbt.Rep("exits"),
...         vbt.Rep("short_entries"),
...         vbt.Rep("short_exits")
...     ),
...     broadcast_named_args=dict(
...         entries= vbt.index_dict({"2021-02-18": True, "_def": False}),
...         exits=    vbt.index_dict({"2021-02-21": True, "_def": False}),
...         short_entries=vbt.index_dict({"2021-02-22": True, "_def": False}),
...         short_exits= vbt.index_dict({"2021-02-24": True, "_def": False})
...     )
... )
```

But we're rarely interested in backtesting signals on some fixed dates. Let's create a signal function that yields a long entry signal when there is an above-crossover and a short entry signal when there is a below-crossover of two moving average arrays. In addition, we'll parametrize this strategy by introducing a flexible parameter `wait` that controls the number of bars after which a signal should be placed after a crossover has been detected; if an opposite crossover happens during that time, the signal gets canceled, thus `wait` also acts as a confirmation period. The parameter will broadcast together with data to be able to define it per row, column, and element.

```
>>> @njit
... def signal_func_nb(c, fast_sma, slow_sma, wait):
...     curr_wait = vbt.pf_nb.select_nb(c, wait) ❶
...     i_wait = c.i - curr_wait ❷
...     if i_wait < 0: ❸
...         return False, False, False, False
...
...     if vbt.nb.iter_crossed_above_nb(fast_sma, slow_sma, i_wait, c.col): ❹
...         cross_confirmed = True
...         for j in range(i_wait + 1, c.i + 1): ❺
...             if not vbt.nb.iter_above_nb(fast_sma, slow_sma, j, c.col):
...                 cross_confirmed = False
...                 break
...         if cross_confirmed:
...             return True, False, False, False
...
...     if vbt.nb.iter_crossed_below_nb(fast_sma, slow_sma, i_wait, c.col): ❻
...         cross_confirmed = True
...         for j in range(i_wait + 1, c.i + 1):
...             if not vbt.nb.iter_below_nb(fast_sma, slow_sma, j, c.col):
...                 cross_confirmed = False
...                 break
...         if cross_confirmed:
...             return False, False, True, False
...
...     return False, False, False, False

>>> fast_sma = data.run("sma", 20, short_name="fast_sma").real
>>> slow_sma = data.run("sma", 50, short_name="slow_sma").real
```

```
>>> pf = vbt.Portfolio.from_signals(
...     data,
...     signal_func_nb=signal_func_nb,
...     signal_args=(
...         vbt.Rep("fast_sma"),
...         vbt.Rep("slow_sma"),
...         vbt.Rep("wait")
...     ),
...     broadcast_named_args=dict(
...         fast_sma=fast_sma,
...         slow_sma=slow_sma,
...         wait=0 ⑦
...     )
... )
>>> pf.orders.count()
fast_sma_timeperiod  slow_sma_timeperiod  symbol
20                  50                  BTCUSD  40
                        ETHUSD  32
Name: count, dtype: int64
```

- ① Since the confirmation period is an array-like parameter, get the value defined for the current row and column
- ② Get the row where the crossover should have taken place
- ③ If the current period is smaller than the confirmation period, return no signal
- ④ Check whether the faster SMA has crossed the slower SMA at the current bar
- ⑤ If true, loop over the confirmation period (including the current bar) and check whether the faster SMA has always stayed above the slower SMA for confirmation
- ⑥ The same for the below-crossover
- ⑦ Waiting period is a flexible parameter, that is, it must broadcast together with other arrays

To confirm that our strategy has generated a correct number of orders, let's get the total number of crossover signals manually:

```
>>> n_crossed_above = fast_sma.vbt.crossed_above(slow_sma).sum()
>>> n_crossed_below = fast_sma.vbt.crossed_below(slow_sma).sum()
>>> n_crossed_above + n_crossed_below
fast_sma_timeperiod  slow_sma_timeperiod  symbol
20                  50                  BTCUSD  40
                        ETHUSD  32
Name: count, dtype: int64
```

To demonstrate the full power of the vectorbt's broadcaster, let's test the confirmation period of 0, 1, 7, and 30 bars by wrapping the parameter with [Param](#):

```
>>> pf = vbt.Portfolio.from_signals(
...     data,
...     signal_func_nb=signal_func_nb,
...     signal_args=(
...         vbt.Rep("fast_sma"),
...         vbt.Rep("slow_sma"),
...         vbt.Rep("wait")
...     ),
...     broadcast_named_args=dict(
...         fast_sma=fast_sma,
...         slow_sma=slow_sma,
...         wait=vbt.Param([0, 1, 7, 30])
...     )
... )
>>> pf.orders.count()
wait  fast_sma_timeperiod  slow_sma_timeperiod  symbol
0     20                  50                  BTCUSD  40
                        ETHUSD  32
1     20                  50                  BTCUSD  38
                        ETHUSD  32
7     20                  50                  BTCUSD  36
                        ETHUSD  30
30    20                  50                  BTCUSD  14
                        ETHUSD  16
Name: count, dtype: int64
```

We can observe how the number of orders gradually decreases with a longer confirmation period. If you love vectorbt for its performance, you might argue that even though the number of crossovers is low, having the second loop is not exactly optimal performance-wise: we can rewrite the logic to iterate over the data just once. For this, we need to introduce a temporary array that stores the index of the latest crossover that has been confirmed so far, and once the confirmation period is fully over, we can issue a signal. This use case is a perfect example on how to temporarily store and then share data between multiple signal function calls!

The temporary array that we will create will be a one-dimensional NumPy array where the latest crossover index is stored per column. We could have also used a regular typed list but remember that NumPy arrays enjoy superiority in Numba. Why per column and not just an array with one value? This would work for ungrouped portfolios where columns are processed strictly one after another, but in a case where the portfolio is grouped, columns are processed in a zigzag manner inside their groups, thus we should always structure our temporary data per column to be on a safe side. Another challenge is the creation of such an array: how do we know the number of columns in advance? Thankfully, we can use a template!

```
>>> @njit
... def signal_func_nb(c, fast_sma, slow_sma, wait, temp_coi): ①
...     if temp_coi[c.col] != -1: ②
...         crossed_above = vbt.nb.iter_crossed_above_nb(
...             fast_sma, slow_sma, temp_coi[c.col], c.col
...         )
```

```

...     crossed_below = vbt.nb.iter_crossed_below_nb(
...         fast_sma, slow_sma, temp_coi[c.col], c.col
...     )
...     if crossed_above: ❸
...         if not vbt.pf.nb.iter_above_nb(c, fast_sma, slow_sma): ❹
...             temp_coi[c.col] = -1
...         if crossed_below:
...             if not vbt.pf.nb.iter_below_nb(c, fast_sma, slow_sma):
...                 temp_coi[c.col] = -1
...     curr_wait = vbt.pf.nb.select_nb(c, wait)
...     if temp_coi[c.col] != -1: ❺
...         if c.i - temp_coi[c.col] == curr_wait:
...             if crossed_above:
...                 temp_coi[c.col] = -1
...                 return True, False, False, False
...             if crossed_below:
...                 temp_coi[c.col] = -1
...                 return False, False, True, False
...         else: ❻
...             if vbt.pf.nb.iter_crossed_above_nb(c, fast_sma, slow_sma):
...                 if curr_wait == 0:
...                     return True, False, False, False
...                 temp_coi[c.col] = c.i
...             if vbt.pf.nb.iter_crossed_below_nb(c, fast_sma, slow_sma):
...                 if curr_wait == 0:
...                     return False, False, True, False
...                 temp_coi[c.col] = c.i
...     return False, False, False, False

>>> pf = vbt.Portfolio.from_signals(
...     data,
...     signal_func_nb=signal_func_nb,
...     signal_args=(
...         vbt.Rep("fast_sma"),
...         vbt.Rep("slow_sma"),
...         vbt.Rep("wait"),
...         vbt.RepEval("np.full(wrapper.shape_2d[1], -1)") ❼
...     ),
...     broadcast_named_args=dict(
...         fast_sma=fast_sma,
...         slow_sma=slow_sma,
...         wait=vbt.Param([0, 1, 7, 30])
...     )
... )
>>> pf.orders.count()
wait fast_sma_timeperiod slow_sma_timeperiod symbol
0      20                50                BTCUSD  40
      ETHUSD  32
1      20                50                BTCUSD  38
      ETHUSD  32
7      20                50                BTCUSD  36
      ETHUSD  30
30     20                50                BTCUSD  14
      ETHUSD  16
Name: count, dtype: int64

```

- ❶ The temporary array is taken as a regular argument
- ❷ Check whether there is a crossover index stored under this column
- ❸ If true, check the type of the crossover and whether it can still be confirmed. If it cannot be confirmed at this bar, remove the index.
- ❹ Use generic iterative functions (starting with `vbt.nb.iter_`, see [generic.iter\\_](#)) when an operation must be performed on a custom row/column, and portfolio iterative functions (starting with `vbt.pf.nb.iter_`, see [portfolio.iter\\_](#)) to perform an operation on the current row/column taken from the context
- ❺ If there is still an index in the temporary array, the crossover has been confirmed so far, thus check if the confirmation period is over, and if true, return the signal
- ❻ If there is no crossover index, check whether there is a crossover at this bar, and if true, store it inside the temporary array and move on. If the confirmation period is zero, return the signal right away.
- ❼ Create the temporary array `temp_coi`. Use an evaluation template to run a code as an expression after all the arrays have been broadcasted and the final shape has been established. Use `wrapper` to access the shape information.

The code above isn't even complex: it would take the same amount of code to define this logic in a conventional backtesting software. The only difference is that vectorbt relies on functional programming whereas other frameworks rely on object-oriented programming where functions such as `crossed_above_nb` are methods of the current backtesting instance (`self.crossed_above()`) and variables such as `temp_coi` are attributes of that instance (`self.temp_coi`).

## Conflicts

When signals are generated in an automated way, it often happens that multiple signals of the same type come one after another, or multiple signals of different types appear at the same bar. The first case is effectively solved by the FS method by taking into consideration only the first signal while ignoring others (unless [accumulation](#) is turned on), as demonstrated in the following example where we're repeatedly issuing a long entry signal:

```

>>> pf = vbt.Portfolio.from_signals(sub_data, entries=True) ❶
>>> pf.asset_flow
symbol      BTCUSD  ETHUSD
Open time
2021-02-18 00:00:00+00:00  0.00194  0.051557
2021-02-19 00:00:00+00:00  0.00000  0.000000
2021-02-20 00:00:00+00:00  0.00000  0.000000

```

```

2021-02-21 00:00:00+00:00 0.00000 0.000000
2021-02-22 00:00:00+00:00 0.00000 0.000000
2021-02-23 00:00:00+00:00 0.00000 0.000000
2021-02-24 00:00:00+00:00 0.00000 0.000000

```

- 1 When signals are provided as a single value, the value gets broadcast against the entire data shape such that it appears under each single row and column

But what if we start issuing a long exit signal at the same time?

```

>>> pf = vbt.Portfolio.from_signals(sub_data, entries=True, exits=True)
>>> pf.asset_flow
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00    0.0    0.0
2021-02-19 00:00:00+00:00    0.0    0.0
2021-02-20 00:00:00+00:00    0.0    0.0
2021-02-21 00:00:00+00:00    0.0    0.0
2021-02-22 00:00:00+00:00    0.0    0.0
2021-02-23 00:00:00+00:00    0.0    0.0
2021-02-24 00:00:00+00:00    0.0    0.0

```

We can see that the simulator simply ignored conflicting signals. But sometimes, we may want to prefer one signal type over another. In the example above, we have a so-called "long signal conflict", the resolution of which is controlled by the argument `upon_long_conflict` of the type `ConflictMode`. For example, if long entries are more important than long exits:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=True,
...     exits=True,
...     upon_long_conflict="entry"
... )
>>> pf.asset_flow
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.00194  0.051557
2021-02-19 00:00:00+00:00  0.00000  0.000000
2021-02-20 00:00:00+00:00  0.00000  0.000000
2021-02-21 00:00:00+00:00  0.00000  0.000000
2021-02-22 00:00:00+00:00  0.00000  0.000000
2021-02-23 00:00:00+00:00  0.00000  0.000000
2021-02-24 00:00:00+00:00  0.00000  0.000000

```

What about use cases where we allow both directions for an exit signal to become a short entry signal? If so, we would have a so-called "signal direction conflict", the resolution of which is controlled by the argument `upon_dir_conflict` of the type `DirectionConflictMode`. Let's prefer short signals over long signals, in any direction:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=True,
...     exits=True,
...     direction="both",
...     upon_dir_conflict="short"
... )
>>> pf.asset_flow
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00 -0.00194 -0.051557
2021-02-19 00:00:00+00:00  0.00000  0.000000
2021-02-20 00:00:00+00:00  0.00000  0.000000
2021-02-21 00:00:00+00:00  0.00000  0.000000
2021-02-22 00:00:00+00:00  0.00000  0.000000
2021-02-23 00:00:00+00:00  0.00000  0.000000
2021-02-24 00:00:00+00:00  0.00000  0.000000

```

We can see that both orders have become sell orders. Let's combine both use cases and apply our knowledge to the scenario where all four signals are set! We will open a long position at the first bar, while at other bars only the one signal that's opposite to the current position will win and reverse the position:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=True,
...     exits=vbt.index_dict({0: False, "_def": True}),
...     short_entries=vbt.index_dict({0: False, "_def": True}),
...     short_exits=vbt.index_dict({0: False, "_def": True}),
...     upon_long_conflict="entry",
...     upon_short_conflict="entry",
...     upon_dir_conflict="opposite",
...     upon_opposite_entry="reverse"
... )
>>> pf.asset_flow
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.001940  0.051557
2021-02-19 00:00:00+00:00 -0.003880 -0.103114
2021-02-20 00:00:00+00:00  0.003884  0.105377
2021-02-21 00:00:00+00:00 -0.003889 -0.107641
2021-02-22 00:00:00+00:00  0.004127  0.117085
2021-02-23 00:00:00+00:00 -0.004366 -0.126528
2021-02-24 00:00:00+00:00  0.004297  0.122982

```

Great, we forced vectorbt to reverse the position at each bar 🤖

## Orders

As we've learned, signals are just another level of abstraction over orders; they control the timing and direction of orders. But how do we specify the parameters of a typical order a signal should be converted into? Identically to `Portfolio.from_orders`, the class method `Portfolio.from_signals` takes a variety of order-related arguments; in fact, it takes all the arguments that can be found as a field in the class `Order`. If any argument remains `None`, the method takes the default value defined in the `portfolio settings`. For example, the default size is `np.inf`, meaning that each signal instructs the simulator to use up the entire capital. Also, each order-related argument is array-like and broadcasts together with signals. This way, by setting an argument to a single value, we can enable that value for each signal. Let's make each entry signal order \$1 worth of each asset by tweaking the size and size type arguments:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     size=1,
...     size_type="value"
... )
>>> pf.assets
symbol          BTCUSDT  ETHUSD
Open time
2021-02-18 00:00:00+00:00  0.000019  0.000516
2021-02-19 00:00:00+00:00  0.000019  0.000516
2021-02-20 00:00:00+00:00  0.000019  0.000516
2021-02-21 00:00:00+00:00  0.000000  0.000000
2021-02-22 00:00:00+00:00  0.000000  0.000000
2021-02-23 00:00:00+00:00  0.000000  0.000000
2021-02-24 00:00:00+00:00  0.000000  0.000000
```

As we can see, the simulator ordered  $1 / 51552.60 = 0.000019$  units of `BTCUSDT` and then closed out the position. To have a more granular control over any order information, we can specify the information as an array. For example, let's enter a position with 50% of the available cash, close the position, then open a new position of the opposite direction with 25% of the available cash, and close the position again:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries= pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     short_entries=pd.Series([0, 0, 0, 0, X, 0, 0]),
...     short_exits= pd.Series([0, 0, 0, 0, 0, X, 0]),
...     size= pd.Series([0.5, 0, 0, 0, 0.25, 0, 0]),
...     size_type="valuepercent", ①
... )
>>> pf.asset_value / pf.value ②
symbol          BTCUSDT  ETHUSD
Open time
2021-02-18 00:00:00+00:00  0.500000  0.500000 << long entry of 50%
2021-02-19 00:00:00+00:00  0.520256  0.501976
2021-02-20 00:00:00+00:00  0.519967  0.496546
2021-02-21 00:00:00+00:00  0.000000  0.000000 << long exit
2021-02-22 00:00:00+00:00 -0.250000 -0.250000 << short entry of 25%
2021-02-23 00:00:00+00:00 -0.220600 -0.215853
2021-02-24 00:00:00+00:00  0.000000  0.000000 << short exit
```

- ① Treat the size as a percentage of the total portfolio value
- ② Get the allocation at each bar, that is, the ratio of the asset value in respect to the total portfolio value

Thanks to the vectorbt's powerful broadcasting mechanism, we can backtest arbitrary configurations with a couple lines of code. Below, we're testing three different mutual configurations of the size and size type arguments:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     size= vbt.Param([np.inf, 1, 0.5], level=0), ①
...     size_type=vbt.Param(["amount", "value", "valuepercent"], level=0)
... )
>>> pf.total_return
size size_type symbol
inf amount BTCUSDT 0.113592
ETHUSD -0.003135
1.0 value BTCUSDT 0.001136
ETHUSD -0.000031
0.5 valuepercent BTCUSDT 0.056796
ETHUSD -0.001567
Name: total_return, dtype: float64
```

- ① Without the same `level`, both parameters would build a Cartesian product

Each configuration is getting applied on the entire set of signal arrays.

## Accumulation

If we wanted to sell 1\$ worth of each asset instead of closing out the entire position whenever an exit signal is encountered, we need to turn on accumulation:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     size=1,
```



```
...     size_type="value",
...     accumulate=True ❶
... )
>>> pf.assets
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.000019  0.000516
2021-02-19 00:00:00+00:00  0.000019  0.000516
2021-02-20 00:00:00+00:00  0.000019  0.000516
2021-02-21 00:00:00+00:00  0.000002  0.000000
2021-02-22 00:00:00+00:00  0.000002  0.000000
2021-02-23 00:00:00+00:00  0.000002  0.000000
2021-02-24 00:00:00+00:00  0.000002  0.000000
```

#### ❶ See `AccumulationMode`

There is a leftover in the column `BTCUSDT` since we made a profit, while the position has been closed entirely in the column `ETHUSDT` since we made a loss (\$1 is worth less than at the beginning of the simulation). But there is another implication: whenever the accumulation is turned on, the method starts behaving effectively like [Portfolio.from\\_orders](#). For instance, it treats each signal as an order, irrespective of whether we're in the market or not. This is best illustrated by the following example, where we're issuing a long entry signal at each single bar, without and with the accumulation:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=True,
...     size=1,
...     size_type="value",
...     accumulate=vbt.Param([False, True])
... )
>>> pf.asset_flow ❶
accumulate
symbol          False      True
Open time
2021-02-18 00:00:00+00:00  0.000019  0.000516  0.000019  0.000516
2021-02-19 00:00:00+00:00  0.000000  0.000000  0.000018  0.000512
2021-02-20 00:00:00+00:00  0.000000  0.000000  0.000018  0.000523
2021-02-21 00:00:00+00:00  0.000000  0.000000  0.000017  0.000517
2021-02-22 00:00:00+00:00  0.000000  0.000000  0.000018  0.000563
2021-02-23 00:00:00+00:00  0.000000  0.000000  0.000020  0.000634
2021-02-24 00:00:00+00:00  0.000000  0.000000  0.000020  0.000616
```

#### ❶ Calculate and print the asset flow at each bar, that is, the absolute amount of units bought (positive) or sold (negative)

We can see that without the accumulation, only the first signal is getting executed while all the signals that follow are getting ignored since we're already in a position. With the accumulation though, each signal gets executed irrespective of the current position. This allows for pyramiding and other trading schemes that require binary position requirements to be lifted.

## Size types

There is a variety of size types supported. To get the full list, refer to the enumerated type `SizeType`. Each size type can be provided either as a (case-insensitive) string representing a field name, or an integer representing a value, such that the arguments `size_type="value"` and `size_type=SizeType.Value` will behave identically. Most size types will be internally converted into the size type `Amount`, which is the absolute amount of units to order. The conversion is mostly done using the valuation price `val_price`, which defaults to the order price and is meant to be the latest price at the point of decision-making.

### Note

When working with Numba-compiled functions directly, only the integer format is supported.

But not all size types are supported in `Portfolio.from_signals`: any target size that defines a target, such as `TargetAmount`, `TargetValue`, and `TargetPercent(100)`, cannot be safely used since the final order size may contradict the signal. For example: if we're in a position of 10 units, and we've issued an entry signal with the size of 3 units and the size type of `TargetAmount`, the actual order will be a sell order of the size 7, which is opposite to the direction of the signal that we've issued. Additionally, the size type `Percent` cannot be used in certain circumstances when both directions are allowed, such as when reversing a position, because such a percentage cannot be simply "flipped".

## Size granularity

Size is always represented as a (usually 64-bit) floating-point number and the entire simulation logic of vectorbt is also based on this number format. But, as you might have heard, floating numbers aren't exactly ideal for monetary calculations because the floating-point arithmetic often causes a loss of precision. Since Numba doesn't allow us to use fixed-point numbers, vectorbt is forced to use floating-point numbers and apply a range of tricks to compare them with a certain confidence, such as by using `numpy.isclose` and `numpy.round_`.

But what about use cases where the size needs to be an integer, such as when trading stocks? For this, we can use the argument `size_granularity`, which will round off the final absolute size to some number of decimal places. As a rule of thumb: use `1` for whole shares, `0.001` for fractional shares, and some custom value for crypto. For example, Binance provides the step size of each trading pair, which can be directly used as `size_granularity`.

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     size_granularity=vbt.Param([1, 0.001]),
...     init_cash=1000_000
... )
>>> pf.asset_flow
size_granularity    1.000    0.001
```

| symbol                    |  | BTCUSDT | ETHUSDT | BTCUSDT | ETHUSDT  |
|---------------------------|--|---------|---------|---------|----------|
| Open time                 |  |         |         |         |          |
| 2021-02-18 00:00:00+00:00 |  | 19.0    | 515.0   | 19.397  | 515.567  |
| 2021-02-19 00:00:00+00:00 |  | 0.0     | 0.0     | 0.000   | 0.000    |
| 2021-02-20 00:00:00+00:00 |  | 0.0     | 0.0     | 0.000   | 0.000    |
| 2021-02-21 00:00:00+00:00 |  | -19.0   | -515.0  | -19.397 | -515.566 |
| 2021-02-22 00:00:00+00:00 |  | 0.0     | 0.0     | 0.000   | 0.000    |
| 2021-02-23 00:00:00+00:00 |  | 0.0     | 0.0     | 0.000   | 0.000    |
| 2021-02-24 00:00:00+00:00 |  | 0.0     | 0.0     | 0.000   | 0.000    |

**Info**

Even though the traded size appears as an integer, it's still represented by a float.

## Price

By default, vectorbt executes an order right away using the current closing price. This behavior can be changed by tweaking the argument `price`, which can either take a price array, or a price option of the type **PriceType**. If we take a look into the portfolio settings, we'll find that `price` is set to the string `"close"`, which upon running is getting translated into the option `PriceType.Close`. This option is just an alias for the value `np.inf` (positive infinity). What does an infinity do here? Since an order price must be defined within the price bounds of a bar, the negative and positive infinity represent the opening and closing price respectively (see **Price resolution**); both can be used inside arrays. The other two options `NextOpen` and `NextClose` are standalone options and cannot be used inside arrays because they involve other arguments.

**Important**

Which option to choose depends on which price you used to generate your signals. Mostly, signals are generated and executed using the same closing price. To account for potential time gaps between signal generation and execution, use the next open or close, or shift the signals manually. If you generated the signals using the opening price or any other price preceding it, you can also use the option `"open"` and a bit of slippage.

Let's execute entries using open and exits using close. Remember that each order-related argument is defined for all signals types: long entries and exits, and short entries and exits. That means that there isn't any argument that defines price specifically for exits, otherwise, the number of arguments would skyrocket. To make any argument value valid only for a subset of signal types, we should set it by using the signal types a mask:

```
>>> price = sub_data.symbol_wrapper.fill() ❶
>>> entries = pd.Series([X, 0, 0, 0, 0, 0, 0]).vbt.broadcast_to(price) ❷
>>> exits = pd.Series([0, 0, 0, X, 0, 0, 0]).vbt.broadcast_to(price)
>>> price[entries] = sub_data.open ❸
>>> price[exits] = sub_data.close

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=entries,
...     exits=exits,
...     price=price
... )

>>> pf.orders.price.to_pd() == sub_data.open ❹
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00    True    True
2021-02-19 00:00:00+00:00   False   False
2021-02-20 00:00:00+00:00   False   False
2021-02-21 00:00:00+00:00   False   False
2021-02-22 00:00:00+00:00   False   False
2021-02-23 00:00:00+00:00   False   False
2021-02-24 00:00:00+00:00   False   False

>>> pf.orders.price.to_pd() == sub_data.close ❺
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00   False   False
2021-02-19 00:00:00+00:00   False   False
2021-02-20 00:00:00+00:00   False   False
2021-02-21 00:00:00+00:00    True    True
2021-02-22 00:00:00+00:00   False   False
2021-02-23 00:00:00+00:00   False   False
2021-02-24 00:00:00+00:00   False   False
```

- ❶ Create a new price array of the same shape as the data
- ❷ Define signals and broadcast them to the price array to be able to use them as a mask
- ❸ Set any element in price that falls under an entry signal to the opening price
- ❹ Generate a mask of filled orders which price is open (such a comparison works only if slippage is zero!)
- ❺ Generate a mask of filled orders which price is close

Let's order using the opening price of the next bar instead:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits=pd.Series([0, 0, 0, X, 0, 0, 0]),
...     price="nextopen"
... )
>>> pf.orders.price.to_pd() == sub_data.open
symbol      BTCUSDT  ETHUSDT
```

```

Open time
2021-02-18 00:00:00+00:00    False    False
2021-02-19 00:00:00+00:00     True     True
2021-02-20 00:00:00+00:00    False    False
2021-02-21 00:00:00+00:00    False    False
2021-02-22 00:00:00+00:00     True     True
2021-02-23 00:00:00+00:00    False    False
2021-02-24 00:00:00+00:00    False    False

```

As we can see, the simulator waited one bar and then executed each signal using the opening price. This is one of the safest approaches when it comes to backtesting because now we have the freedom of running indicators on any price, and we don't have to worry about the look-ahead bias during the execution anymore. The most bulletproof approach would be using the next close because the difference between the previous close and the next open is usually negligible.

## Shifting

We could have done the same as above by shifting the signal arrays by one bar manually:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]).vbt.signals.fshift(),
...     exits= pd.Series([0, 0, 0, X, 0, 0]).vbt.signals.fshift(),
...     price="open" ❶
... )
>>> pf.orders.price.to_pd() == sub_data.open
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00    False    False
2021-02-19 00:00:00+00:00     True     True
2021-02-20 00:00:00+00:00    False    False
2021-02-21 00:00:00+00:00    False    False
2021-02-22 00:00:00+00:00     True     True
2021-02-23 00:00:00+00:00    False    False
2021-02-24 00:00:00+00:00    False    False

```

❶ When shifting manually, use open/close of the current bar

And this is one of the most underrated features of vectorbt: since we're working with array data, we can shift the data to make any current bar use information from the past. In the example above, after we've forward-shifted the signal arrays, the long entry signal on the 18th February has moved to the 19th February while the index has stayed the same. Thus, the surrounding price information in form of OHLC (the arguments `open`, `high`, `low`, and `close`) must be fixed and should never be shifted! That also means that we not only have to shift the signals, but any information those signals link to, for example, the order direction and price:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries= pd.Series([X, 0, 0, 0, X, 0, 0]).vbt.fshift(1, False),
...     exits= pd.Series([0, 0, 0, 0, X, 0, 0, X]).vbt.fshift(1, False),
...     direction=pd.Series([L, L, L, L, S, S, S]).vbt.fshift(1, -1)
... )
>>> pf.assets
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00    0.000000    0.000000
2021-02-19 00:00:00+00:00    0.001789    0.051151
2021-02-20 00:00:00+00:00    0.001789    0.051151
2021-02-21 00:00:00+00:00    0.001789    0.051151
2021-02-22 00:00:00+00:00    0.000000    0.000000
2021-02-23 00:00:00+00:00   -0.001979   -0.057624
2021-02-24 00:00:00+00:00   -0.001979   -0.057624

```

### Hint

As a rule of thumb: if an argument is signal-anchored, it should be shifted as well. If an argument is date-anchored though, it should stay the same.

To reduce the burden of shifting manually, vectorbt can do it for us automatically! For this, it requires from us providing the argument `from_ago`, which represents how far ago was the bar from which all the signal and order information should be taken. For example, when `from_ago=1`, the related information is taken from the previous bar:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries= pd.Series([X, 0, 0, 0, X, 0, 0]),
...     exits= pd.Series([0, 0, 0, 0, X, 0, 0, X]),
...     direction=pd.Series([L, L, L, L, S, S, S]),
...     from_ago=1
... )
>>> pf.assets
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00    0.000000    0.000000
2021-02-19 00:00:00+00:00    0.001789    0.051151
2021-02-20 00:00:00+00:00    0.001789    0.051151
2021-02-21 00:00:00+00:00    0.001789    0.051151
2021-02-22 00:00:00+00:00    0.000000    0.000000
2021-02-23 00:00:00+00:00   -0.001979   -0.057624
2021-02-24 00:00:00+00:00   -0.001979   -0.057624

```

This argument can also be provided as an array.

## Slippage

By default, an order is executed as a market order. This can be seen in the portfolio settings, under the key `order_type`. Market orders are transactions meant to execute as quickly as possible at the current market price, which is always `price` (the closing price by default). In reality though, the price at which an order is executed often does not match the price at which it was requested. To take into account this discrepancy, we need to introduce a slippage. Assuming the trading volume is high, we should see less slippage. And when the trading volume is slow, we should expect to see more slippage. An optimal slippage can be calculated from order book data (see [this blog](#)).

But let for a second assume that the average slippage is 0.5%. Using it together with the default price is generally a bad idea since the closing price is meant to be the latest price seen at each bar. To make our simulation more realistic, we need to apply it on the next open instead:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     price="nextopen",
...     slippage=0.005 ❶
... )
>>> pf.orders.price.to_pd() ❷
```

| symbol                    | BTCUSDT     | ETHUSDT    |
|---------------------------|-------------|------------|
| Open time                 |             |            |
| 2021-02-18 00:00:00+00:00 | NaN         | NaN        |
| 2021-02-19 00:00:00+00:00 | 51810.37305 | 1948.60455 |
| 2021-02-20 00:00:00+00:00 | NaN         | NaN        |
| 2021-02-21 00:00:00+00:00 | NaN         | NaN        |
| 2021-02-22 00:00:00+00:00 | 57125.28825 | 1923.87230 |
| 2021-02-23 00:00:00+00:00 | NaN         | NaN        |
| 2021-02-24 00:00:00+00:00 | NaN         | NaN        |

```
>>> pf.orders.price.to_pd() / sub_data.open ❸
```

| symbol                    | BTCUSDT | ETHUSDT |
|---------------------------|---------|---------|
| Open time                 |         |         |
| 2021-02-18 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-19 00:00:00+00:00 | 1.005   | 1.005   |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-22 00:00:00+00:00 | 0.995   | 0.995   |
| 2021-02-23 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-24 00:00:00+00:00 | NaN     | NaN     |

❶ 0.01 is 1%

❷ Get the filled order price

❸ Get the filled order price in relation to the opening price

We can see that the slippage increased the price by 0.5% when buying and decreased the price by 0.5% when selling. Introducing a slippage will always result in a fixed price penalty, thus the slippage should always reflect the average penalty that is being recorded in the market for transactions of this magnitude. Since slippage isn't static but depending on various factors, we can provide it as an array.

## Limit orders

To help eliminate or reduce slippage, traders use limit orders instead of market orders. A limit order only fills at the price we want, or better. Unlike a market order, it won't fill at a worse price. But there is a catch: while the price is guaranteed, the filling of the order is not, and limit orders will not be executed unless the asset price meets the order qualifications. If the asset does not reach the specified price, the order is not filled, and we may miss out on the trading opportunity. So, what happens when we execute our signals using limit orders? Let's switch the default order type by setting the argument `order_type` to `"limit"`:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     order_type="limit"
... )
>>> pf.orders.price.to_pd()
```

| symbol                    | BTCUSDT  | ETHUSDT |
|---------------------------|----------|---------|
| Open time                 |          |         |
| 2021-02-18 00:00:00+00:00 | NaN      | NaN     |
| 2021-02-19 00:00:00+00:00 | 51552.60 | 1938.91 |
| 2021-02-20 00:00:00+00:00 | NaN      | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN      | NaN     |
| 2021-02-22 00:00:00+00:00 | 57412.35 | 1933.54 |
| 2021-02-23 00:00:00+00:00 | NaN      | NaN     |
| 2021-02-24 00:00:00+00:00 | NaN      | NaN     |

Since the default order price is the closing price, the simulator registered it as the target limit price and skipped the entry bar. At the next bar, the simulator checked whether the limit price has been hit by comparing it against the full candle. As we can see above, each of the target prices could be satisfied already at the next bar, which is quite similar to using the next opening price as the order execution price. But such a rapid match isn't always the case: what if we wanted to enter a short trade on the 22nd February using the previous high as the limit price?

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([0, 0, 0, 0, X, 0, 0]),
...     direction="shortonly",
...     price=sub_data.high.vbt.fshift(), ❶
...     order_type="limit"
... )
>>> pf.orders.price.to_pd()
```

| symbol                    | BTCUSDT | ETHUSDT |
|---------------------------|---------|---------|
| Open time                 |         |         |
| 2021-02-18 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-19 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-22 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-23 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-24 00:00:00+00:00 | NaN     | NaN     |

- Shift the high price by one bar to use the previous high price

Suddenly, no limit order could be filled as the same price or higher (we're selling) couldn't be found at any time during or after the 22nd February. We have now an order that can potentially remain pending forever. How do we limit its lifetime? There are various possibilities.

#### Time in force

Time-in-force orders can be created using the argument `limit_tif`, which expects a time delta, in any format that can be translated into `np.timedelta64` and then to a 64-bit integer representing nanoseconds, that is, a string, `pd.Timedelta`, `datetime.timedelta`, `np.timedelta64`, or an integer. The time-in-force specification starts counting at the **beginning** of the entry bar (even if the limit order has been issued at the end of the bar) and will be checked at the beginning of each bar starting from the second one. Let's create a buy limit order on the 20th February using the previous low price:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([0, 0, X, 0, 0, 0, 0]),
...     price=sub_data.low.vbt.fshift(),
...     order_type="limit"
... )
>>> pf.orders.price.to_pd()
```

| symbol                    | BTCUSDT | ETHUSDT |
|---------------------------|---------|---------|
| Open time                 |         |         |
| 2021-02-18 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-19 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN     | 1891.0  |
| 2021-02-22 00:00:00+00:00 | 50710.2 | NaN     |
| 2021-02-23 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-24 00:00:00+00:00 | NaN     | NaN     |

We can see that the limit order in the column `BTCUSDT` was executed in two bars while the limit order in the column `ETHUSDT` was executed in just one bar. Below, we're testing a TIF option passed as a frequency string, which works only outside of arrays:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([0, 0, X, 0, 0, 0, 0]),
...     price=sub_data.low.vbt.fshift(),
...     order_type="limit",
...     limit_tif="2d"
... )
>>> pf.orders.price.to_pd()
```

| symbol                    | BTCUSDT | ETHUSDT |
|---------------------------|---------|---------|
| Open time                 |         |         |
| 2021-02-18 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-19 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN     | 1891.0  |
| 2021-02-22 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-23 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-24 00:00:00+00:00 | NaN     | NaN     |

- Gets translated into a time delta using `to_timedelta64`

Why is there is no order in `BTCUSDT`? Because 2 days from `2021-02-20 00:00:00` is `2021-02-22 00:00:00`, which is the timestamp at which the order gets cancelled, and since each timestamp represents the opening time of a bar, the pending order doesn't exist during the fifth bar anymore. To provide a TIF specification inside an array or to test multiple configurations, instead of a string, we must use either a Pandas or NumPy format, or a total duration in nanoseconds. Let's test all formats at once!

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([0, 0, X, 0, 0, 0, 0]),
...     price=sub_data.low.vbt.fshift(),
...     order_type="limit",
...     limit_tif=vbt.Param([
...         -1,
...         pd.Timedelta(days=1),
...         2 * np.timedelta64(864000000000),
...         3 * vbt.dt_nb_d_ns
...     ], keys=["none", "1 days", "2 days", "3 days"])
... )
>>> pf.orders.price.to_pd()
```

| limit_tif                 | none    | 1 days  | 2 days  | \       |
|---------------------------|---------|---------|---------|---------|
| symbol                    | BTCUSDT | ETHUSDT | BTCUSDT | ETHUSDT |
| Open time                 |         |         |         |         |
| 2021-02-18 00:00:00+00:00 | NaN     | NaN     | NaN     | NaN     |
| 2021-02-19 00:00:00+00:00 | NaN     | NaN     | NaN     | NaN     |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN     | NaN     | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN     | 1891.0  | NaN     | 1891.0  |
| 2021-02-22 00:00:00+00:00 | 50710.2 | NaN     | NaN     | NaN     |
| 2021-02-23 00:00:00+00:00 | NaN     | NaN     | NaN     | NaN     |
| 2021-02-24 00:00:00+00:00 | NaN     | NaN     | NaN     | NaN     |

| limit_tif                 | 3 days  |        |
|---------------------------|---------|--------|
| symbol                    | BTCUSD  | ETHUSD |
| Open time                 |         |        |
| 2021-02-18 00:00:00+00:00 | NaN     | NaN    |
| 2021-02-19 00:00:00+00:00 | NaN     | NaN    |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN    |
| 2021-02-21 00:00:00+00:00 | NaN     | 1891.0 |
| 2021-02-22 00:00:00+00:00 | 50710.2 | NaN    |
| 2021-02-23 00:00:00+00:00 | NaN     | NaN    |
| 2021-02-24 00:00:00+00:00 | NaN     | NaN    |

- 1 Don't use TIF
- 2 TIF of 1 day in Pandas format
- 3 TIF of 2 days in NumPy format
- 4 TIF of 3 days in integer format (nanoseconds)

**Note**

NumPy and integer formats are preferred when building large arrays.

There is also a way to specify a number of rows as opposed to a time delta by tweaking the time delta format (`time_delta_format`), which is represented by the enumerated type **TimeDeltaFormat**. This is required in cases where the index is not datetime-like, such as after splitting the data for cross-validation. Let's change the time delta format to `Rows` and do the same as above but now waiting for a specific number of rows to pass:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([0, 0, X, 0, 0, 0, 0]),
...     price=sub_data.low.vbt.fshift(),
...     order_type="limit",
...     limit_tif=vbt.Param(
...         [-1, 1, 2, 3],
...         keys=["none", "1 rows", "2 rows", "3 rows"]
...     ),
...     time_delta_format="rows"
... )
```

If we're confident that our index has a fixed frequency (for example, days instead of business days) and doesn't have gaps, we can also determine the number of rows by dividing time deltas:

```
>>> index_td = vbt.timedelta("1 minute")
>>> tif_td = vbt.timedelta("1 day")
>>> int(tif_td / index_td)
1440
```

### Expiration date

Another way to let limit orders expire based on dates and times is by setting an expiration date with `limit_expiry`. We've got two meaningful options here: setting a frequency at which limit orders should expire, or providing a datetime-like array that can incorporate `pd.Timestamp`, `datetime.datetime`, and `np.datetime64` objects. In the end, the argument will be converted into the 64-bit integer format representing a Unix time (total nanoseconds since 1970). Let's play with the first option first by making limit orders behave like day orders! By receiving a frequency, the simulation method will determine the right bound of each timestamp based by using the method `ArrayWrapper.get_period_ns_index`:

```
>>> sub_data.symbol_wrapper.get_period_ns_index("1d")
array([1613692800000000000, 1613779200000000000, 1613865600000000000,
       1613952000000000000, 1614038400000000000, 1614124800000000000,
       1614211200000000000])
```

But since our data is already daily, let's make a pending limit order expire at the end of the week in which it was created:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([0, 0, X, 0, 0, 0, 0]),
...     price=sub_data.low.vbt.fshift(),
...     order_type="limit",
...     limit_expiry="monday" ①
... )
>>> pf.orders.price.to_pd()
symbol      BTCUSD  ETHUSD
Open time
2021-02-18 00:00:00+00:00  NaN    NaN
2021-02-19 00:00:00+00:00  NaN    NaN
2021-02-20 00:00:00+00:00  NaN    NaN
2021-02-21 00:00:00+00:00  NaN  1891.0
2021-02-22 00:00:00+00:00  50710.2  NaN
2021-02-23 00:00:00+00:00  NaN    NaN
2021-02-24 00:00:00+00:00  NaN    NaN

>>> sub_data.symbol_wrapper.index.to_period(vbt.offset("monday")) ②
PeriodIndex(['2021-02-16/2021-02-22', '2021-02-16/2021-02-22',
            '2021-02-16/2021-02-22', '2021-02-16/2021-02-22',
            '2021-02-16/2021-02-22', '2021-02-23/2021-03-01',
            '2021-02-23/2021-03-01'],
            dtype='period[W-MON]', name='Open time')
```

1 See [Anchored offsets](#)

2 User warning "Converting to PeriodArray/Index representation will drop timezone information" can be ignored

Both orders could be executed because the dates `2021-02-20`, `2021-02-21`, and `2021-02-22` belong to the same week `2021-02-16/2021-02-22`. As soon as we had changed the week layout to start on a Sunday though, only the first two dates would belong to the same week, thus resulting in the expiration of the pending order in the column `BTCUSDT`:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([0, 0, X, 0, 0, 0, 0]),
...     price=sub_data.low.vbt.fshift(),
...     order_type="limit",
...     limit_expiry="sunday"
... )
>>> pf.orders.price.to_pd()
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00      NaN      NaN
2021-02-19 00:00:00+00:00      NaN      NaN
2021-02-20 00:00:00+00:00      NaN      NaN
2021-02-21 00:00:00+00:00      NaN    1891.0
2021-02-22 00:00:00+00:00      NaN      NaN
2021-02-23 00:00:00+00:00      NaN      NaN
2021-02-24 00:00:00+00:00      NaN      NaN

>>> sub_data.symbol_wrapper.index.to_period(vbt.offset("sunday"))
PeriodIndex(['2021-02-15/2021-02-21', '2021-02-15/2021-02-21',
            '2021-02-15/2021-02-21', '2021-02-15/2021-02-21',
            '2021-02-22/2021-02-28', '2021-02-22/2021-02-28',
            '2021-02-22/2021-02-28'],
            dtype='period[W-SUN]', name='Open time')
```

We can also build our own `limit_expiry` array. For instance, let's simulate a TIF of 2 days by relying on the expiration dates alone - the same as passing `limit_tif="2d"`:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([0, 0, X, 0, 0, 0, 0]),
...     price=sub_data.low.vbt.fshift(),
...     order_type="limit",
...     limit_expiry=vbt.RepEval("""
...     expiry_index = wrapper.index + pd.Timedelta(days=2)
...     expiry_arr = vbt.to_2d_array(vbt.dt.to_ns(expiry_index))
...     expiry_arr
...     """) 1
... )
>>> pf.orders.price.to_pd()
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00      NaN      NaN
2021-02-19 00:00:00+00:00      NaN      NaN
2021-02-20 00:00:00+00:00      NaN      NaN
2021-02-21 00:00:00+00:00      NaN    1891.0
2021-02-22 00:00:00+00:00      NaN      NaN
2021-02-23 00:00:00+00:00      NaN      NaN
2021-02-24 00:00:00+00:00      NaN      NaN
```

1 Since, after broadcasting, the final index might be different from the original one, we need to create a template that adds 2 days to the final index and converts it into a two-dimensional array. Leaving it as one-dimensional will broadcast values per column (!)

Note

Don't attempt to pass a `pd.Index` directly, it will be converted into parameters. Use `pd.Series` instead.

### Conflicts

What happens if the user issues a signal when there is a limit order pending? Since FS can track only one limit order per column at a time, we need to choose a winner. There are two arguments that are relevant: `upon_adj_limit_conflict` and `upon_opp_limit_conflict`, both of the type `PendingConflictMode`. The first controls the decision process if both orders attempt to go in the same direction, the second if their directions are opposite. Let's introduce a sell market order on the 21st February and test various conflict resolution options:

```
>>> all_conflict_modes = vbt.pf_enums.PendingConflictMode._fields 1
>>> pf = vbt.Portfolio.from_signals(
...     sub_data.select("BTCUSDT"),
...     entries= pd.Series([0, 0, X, 0, 0, 0, 0]),
...     short_entries=pd.Series([0, 0, 0, X, 0, 0, 0]),
...     price=sub_data.select("BTCUSDT").low.vbt.fshift(),
...     order_type=vbt.index_dict({2: "limit", "_def": "market"}), 2
...     upon_opp_limit_conflict=vbt.Param(all_conflict_modes)
... )
>>> pf.orders.price.to_pd()
upon_opp_limit_conflict  KeepIgnore  KeepExecute  CancelIgnore  \
Open time
2021-02-18 00:00:00+00:00      NaN      NaN      NaN
2021-02-19 00:00:00+00:00      NaN      NaN      NaN
2021-02-20 00:00:00+00:00      NaN      NaN      NaN
2021-02-21 00:00:00+00:00      NaN    53863.93      NaN
2021-02-22 00:00:00+00:00    50710.2    50710.20      NaN
2021-02-23 00:00:00+00:00      NaN      NaN      NaN
```



|                           |               |     |     |
|---------------------------|---------------|-----|-----|
| 2021-02-24 00:00:00+00:00 | NaN           | NaN | NaN |
| upon_opp_limit_conflict   | CancelExecute |     |     |
| Open time                 |               |     |     |
| 2021-02-18 00:00:00+00:00 | NaN           |     |     |
| 2021-02-19 00:00:00+00:00 | NaN           |     |     |
| 2021-02-20 00:00:00+00:00 | NaN           |     |     |
| 2021-02-21 00:00:00+00:00 | 53863.93      |     |     |
| 2021-02-22 00:00:00+00:00 | NaN           |     |     |
| 2021-02-23 00:00:00+00:00 | NaN           |     |     |
| 2021-02-24 00:00:00+00:00 | NaN           |     |     |

- 1 Get the list with all options
- 2 Only the first order should be a limit order

As you might have noticed, the first word in the option's name means what to do with the pending limit order, and the second word means what to do with the user-defined order. By default, the limit order wins when the user-defined signal is of the same direction (buy and buy, sell and sell, option `KeepIgnore`) to avoid repeatedly executing similar signals, and the user-defined signal wins when both orders are of the opposite direction (buy and sell, sell and buy, option `CancelExecute`) to account for a change in the market regime.

### Delta

The target price of a limit order is taken from the argument `price`. But what if we wanted to place a limit order at a price some distance (a.k.a. "delta") above or below some another price, such as 10% from the current closing price? For this, we would need to know whether a signal is a buy or sell order beforehand: if it's a buy order, the price should be lower than the closing price, and if it's a sell order, the price should be higher than the closing price. To avoid determining such information in advance and building a price array manually, there is a handy argument `limit_delta` that specifies how far away from `price` the target limit price should be. The format of this argument is controlled by another argument `delta_format` of the type `DeltaFormat`. The default format is a percentage (`Percent`), such as passing `limit_delta=0.1` will be understood as 10%. Let's try out different deltas for a buy limit order at the very first bar:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     order_type="limit",
...     limit_delta=vbt.Param([0, 0.1, 0.5])
... )
>>> pf.orders.price.to_pd()
```

| limit_delta               | 0.0     | 0.1     | 0.5      |
|---------------------------|---------|---------|----------|
| symbol                    | BTCUSDT | ETHUSDT | BTCUSDT  |
| Open time                 |         |         |          |
| 2021-02-18 00:00:00+00:00 | NaN     | NaN     | NaN      |
| 2021-02-19 00:00:00+00:00 | 51552.6 | 1938.91 | NaN      |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN     | NaN      |
| 2021-02-21 00:00:00+00:00 | NaN     | NaN     | NaN      |
| 2021-02-22 00:00:00+00:00 | NaN     | NaN     | 1745.649 |
| 2021-02-23 00:00:00+00:00 | NaN     | NaN     | 46397.34 |
| 2021-02-24 00:00:00+00:00 | NaN     | NaN     | NaN      |

| limit_delta               | ETHUSDT |
|---------------------------|---------|
| symbol                    |         |
| Open time                 |         |
| 2021-02-18 00:00:00+00:00 | NaN     |
| 2021-02-19 00:00:00+00:00 | NaN     |
| 2021-02-20 00:00:00+00:00 | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN     |
| 2021-02-22 00:00:00+00:00 | NaN     |
| 2021-02-23 00:00:00+00:00 | NaN     |
| 2021-02-24 00:00:00+00:00 | NaN     |

As we can see, without a delta, the target limit price is essentially the current closing price. With a limit delta of 10%, the target limit price becomes `close * (1 - limit_delta)`, which in our example have only been matched after a few bars. The final limit delta of 50% haven't been matched at all since the price window that we chose doesn't have dips of such a magnitude. If our limit order was a sell order, the calculation would be `close * (1 + limit_delta)`.

### Reversing

We've covered regular buy/sell limit orders that get matched whenever the price is found to be the same or lower/higher than the requested price. Now, we can reverse the matching logic with the argument `limit_reverse` to simulate buy/sell **stop** orders. For instance, for a buy stop order with an absolute delta of \$100, it would search for a price that is \$100 higher (as opposed to lower) than the current closing price and execute the limit order using that price:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data.select("BTCUSDT"),
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     order_type="limit",
...     limit_delta=vbt.Param([0, 100, 5000, 10000]),
...     delta_format="absolute",
...     limit_reverse=True
... )
>>> pf.orders.price.to_pd()
```

| limit_delta               | 0        | 100     | 5000    | 10000 |
|---------------------------|----------|---------|---------|-------|
| Open time                 |          |         |         |       |
| 2021-02-18 00:00:00+00:00 | NaN      | NaN     | NaN     | NaN   |
| 2021-02-19 00:00:00+00:00 | 51552.61 | 51652.6 | NaN     | NaN   |
| 2021-02-20 00:00:00+00:00 | NaN      | NaN     | 56552.6 | NaN   |
| 2021-02-21 00:00:00+00:00 | NaN      | NaN     | NaN     | NaN   |
| 2021-02-22 00:00:00+00:00 | NaN      | NaN     | NaN     | NaN   |
| 2021-02-23 00:00:00+00:00 | NaN      | NaN     | NaN     | NaN   |
| 2021-02-24 00:00:00+00:00 | NaN      | NaN     | NaN     | NaN   |



### Info

Why does the same delta of zero result in a different fill price without and with reversing? Because upon hitting the second bar, the limit price is getting compared against the opening price. In the first example, the opening price is higher than the limit price, thus the limit order is executed using the limit price. In the second example, because the opening price is higher, the limit order is executed using the opening price.

### Adjustment

In the theoretical section we discussed how it's possible to change any limit order from a callback. There are four callbacks that we can use: a signal function ( `signal_func_nb` ), an adjustment function ( `adjust_func_nb` ), a post-signal function ( `post_signal_func_nb` ), or a post-segment function ( `post_segment_func_nb` ). The signal function is called at the beginning of a bar and should only be used if we need to define the four user-defined signals dynamically. The adjustment function is called at the beginning of the default signal function `dir_signal_func_nb` for direction-unaware and `ls_signal_func_nb` for direction-aware signals, which are provided via boolean arrays (statically). The post-segment function is called at the end of a bar for pre-calculating metrics such as portfolio returns.

### Note

The adjustment function gets called only if the signals are provided statically, that is, `signal_func_nb` hasn't been overridden. Also note that overriding any of the functions above will disable caching of `Portfolio.from_signals`!

Each of those functions accepts a named tuple with the current simulation context, which includes a one-dimensional record array `SignalContext.last_limit_info` with the latest limit order information defined per column. By changing this array, we can change the limit order that is yet to be processed at this bar (or the next one for the post-segment function). We will focus on the adjustment function since it's meant to work together with signal arrays. This function should be Numba-compiled, take the current context variable (usually denoted with `c`) and other custom arguments ( `adjust_args` ), and return nothing ( `None` ). Why nothing? Because we can change all the relevant information in the context. Let's create a function that cancels any pending limit order (that is, it hasn't been executed at the entry bar). Limit orders will be created using the opening price and a parametrized custom delta:

```
>>> @njit
... def adjust_func_nb(c): ❶
...     limit_info = c.last_limit_info[c.col] ❷
...     if limit_info.creation_idx != -1: ❸
...         if c.i - limit_info.creation_idx >= 1: ❹
...             vbt.pf_nb.clear_limit_info_nb(limit_info) ❺

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     price="open",
...     order_type="limit",
...     limit_delta=vbt.Param([0, 0.1]),
...     adjust_func_nb=adjust_func_nb
... )
>>> pf.orders.price.to_pd()
limit_delta      0.0      0.1
symbol          BTCUSDT ETHUSDT BTCUSDT ETHUSDT
Open time
2021-02-18 00:00:00+00:00 52117.67 1849.7   NaN   NaN
2021-02-19 00:00:00+00:00   NaN   NaN   NaN   NaN
2021-02-20 00:00:00+00:00   NaN   NaN   NaN   NaN
2021-02-21 00:00:00+00:00   NaN   NaN   NaN   NaN
2021-02-22 00:00:00+00:00   NaN   NaN   NaN   NaN
2021-02-23 00:00:00+00:00   NaN   NaN   NaN   NaN
2021-02-24 00:00:00+00:00   NaN   NaN   NaN   NaN
```

- ❶ Must take `SignalContext`
- ❷ Get the pending limit order information for the current column (asset)
- ❸ Check whether there is any limit order pending
- ❹ Check whether the order has been pending for one bar or longer
- ❺ Use `clear_limit_info_nb` to cancel the order

We can see that without a delta, each order could be filled already at the entry bar. With a delta of 10% though, the orders couldn't be filled at the entry bar, and thus they were cancelled at the beginning of the second bar (19th February). Let's do the opposite: if a limit order is still pending, override its price with the valuation price and delta with 1% to execute at this bar:

```
>>> @njit
... def adjust_func_nb(c):
...     limit_info = c.last_limit_info[c.col]
...     if limit_info.creation_idx != -1:
...         if c.i - limit_info.creation_idx >= 1:
...             limit_info.init_price = -np.inf ❶
...             limit_info.delta = 0.01

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     price="open",
...     order_type="limit",
...     limit_delta=vbt.Param([0.1, 0.2]),
...     adjust_func_nb=adjust_func_nb
... )
>>> pf.orders.price.to_pd()
limit_delta      0.1      0.2
symbol          BTCUSDT ETHUSDT BTCUSDT ETHUSDT
Open time
```

|                           |            |           |            |           |
|---------------------------|------------|-----------|------------|-----------|
| 2021-02-18 00:00:00+00:00 | NaN        | NaN       | NaN        | NaN       |
| 2021-02-19 00:00:00+00:00 | 51037.0839 | 1919.5209 | 51037.0839 | 1919.5209 |
| 2021-02-20 00:00:00+00:00 | NaN        | NaN       | NaN        | NaN       |
| 2021-02-21 00:00:00+00:00 | NaN        | NaN       | NaN        | NaN       |
| 2021-02-22 00:00:00+00:00 | NaN        | NaN       | NaN        | NaN       |
| 2021-02-23 00:00:00+00:00 | NaN        | NaN       | NaN        | NaN       |
| 2021-02-24 00:00:00+00:00 | NaN        | NaN       | NaN        | NaN       |

- 1 Negative infinity will be translated into the valuation price, that is, the latest known price at the time of calling a callback, which is the opening price for adjustment and signal callbacks, and the closing price for post-segment callbacks. Note that you cannot use positive infinity for the closing price because it's unknown!

The same way as we cancelled an order, we can also create a new order without the need of user-defined signals - basically out of nothing!

```
>>> @njit
... def adjust_func_nb(c, custom_delta):
...     limit_info = c.last_limit_info[c.col]
...     if c.i == 0: ❶
...         curr_delta = vbt.pf_nb.select_nb(c, custom_delta)
...         vbt.pf_nb.set_limit_info_nb(limit_info, c.i, delta=curr_delta) ❷

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     broadcast_named_args=dict(custom_delta=vbt.Param([0, 0.1])), ❸
...     adjust_func_nb=adjust_func_nb,
...     adjust_args=(vbt.Rep("custom_delta"),) ❹
... )
>>> pf.orders.price.to_pd()
custom_delta      0.0      0.1
symbol      BTCUSDT ETHUSDT BTCUSDT ETHUSDT
Open time
2021-02-18 00:00:00+00:00 52117.67 1849.7      NaN      NaN
2021-02-19 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-20 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-21 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-22 00:00:00+00:00      NaN      NaN      NaN 1664.73
2021-02-23 00:00:00+00:00      NaN      NaN 46905.903      NaN
2021-02-24 00:00:00+00:00      NaN      NaN      NaN      NaN
```

- 1 Create a signal order at the very first bar only
- 2 Use `set_limit_info_nb`. Take a look at its signature to learn more about defaults. For example, it uses the current valuation price, the infinity size, and both directions by default.
- 3 To make a custom argument parameterizable, we need to define it as a flexible argument in `broadcast_named_args`, that is, it must be able to be defined per element
- 4 Don't forget to put a comma after a single element in a tuple

We just created limit orders dynamically, without any signals 🌟

## Stop orders

Just like limit orders, stop orders are executed only once a price condition is met. Their information is also stored per column inside a one-dimensional record array. But in contrast to limit orders, they are just deterred user-defined signals that result in a market or limit order to mainly close out an existing position. They are created upon a successful entry order that either opens, increases, or reverses a position (i.e., closes out and then enters a new position in the opposite direction). There are three stop order types: SL (`sl_stop`), TSL and TTP (`tsl_stop` and `tsl_th`, acting as one order), and TP (`tp_stop`). Each one of them is stored and tracked separately, but they all act as a one-cancels-all (OCA) order where one order is triggered in full while the others are automatically canceled. Any pending stop orders get also canceled if the entered position has been closed out by other means.

Let's place a stop loss order of 10%:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     sl_stop=0.1,
... )
>>> pf.orders.price.to_pd()
symbol      BTCUSDT ETHUSDT
Open time
2021-02-18 00:00:00+00:00 51552.60 1939.610
2021-02-19 00:00:00+00:00      NaN      NaN
2021-02-20 00:00:00+00:00      NaN      NaN
2021-02-21 00:00:00+00:00      NaN      NaN
2021-02-22 00:00:00+00:00      NaN 1745.649
2021-02-23 00:00:00+00:00 46397.34      NaN
2021-02-24 00:00:00+00:00      NaN      NaN
```

### Important

When passing an instance of `Data` as the first argument (as we did with `sub_data`), the method will extract OHLC from it automatically. If you're passing the closing price as the first argument instead, make sure to pass other price features (OHL) as well, using the arguments `open`, `high`, and `low`. Without them, candles will be incomplete and vectorbt will make decisions solely based on `close`!

We can see that the entered long position has been exited at the price  $51552.60 * 0.9 = 46397.34$ . To test multiple stop values, we can wrap it with the class `Param`:

```
>>> pf = vbt.Portfolio.from_signals(
```

```

...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     sl_stop=vbt.Param([np.nan, 0.1, 0.2]), ❶
... )
>>> pf.orders.price.to_pd()
sl_stop
symbol          BTCUSDT  ETHUSDT  BTCUSDT  ETHUSDT  BTCUSDT  \
Open time
2021-02-18 00:00:00+00:00  51552.6  1939.61  51552.60  1939.610  51552.6
2021-02-19 00:00:00+00:00    NaN    NaN    NaN    NaN    NaN
2021-02-20 00:00:00+00:00    NaN    NaN    NaN    NaN    NaN
2021-02-21 00:00:00+00:00    NaN    NaN    NaN    NaN    NaN
2021-02-22 00:00:00+00:00    NaN    NaN    NaN    1745.649  NaN
2021-02-23 00:00:00+00:00    NaN    NaN    46397.34    NaN    NaN
2021-02-24 00:00:00+00:00    NaN    NaN    NaN    NaN    NaN

sl_stop
symbol          ETHUSDT
Open time
2021-02-18 00:00:00+00:00  1939.610
2021-02-19 00:00:00+00:00    NaN
2021-02-20 00:00:00+00:00    NaN
2021-02-21 00:00:00+00:00    NaN
2021-02-22 00:00:00+00:00  1551.688
2021-02-23 00:00:00+00:00    NaN
2021-02-24 00:00:00+00:00    NaN

```

❶ NaN means no stop order

Since each stop argument and its accompanying information broadcast together with the data, we can provide them as fully-fledged arrays. Below, we're defining a stop loss based on the ATR:

```

>>> atr = data.run("atr").atr
>>> sub_atr = atr.loc["2021-02-18":"2021-02-24"]

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     sl_stop=sub_atr / sub_data.close
... )
>>> pf.orders.price.to_pd()
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  51552.600000  1939.610000
2021-02-19 00:00:00+00:00    NaN    NaN
2021-02-20 00:00:00+00:00    NaN  1805.283249
2021-02-21 00:00:00+00:00    NaN    NaN
2021-02-22 00:00:00+00:00  48335.716826    NaN
2021-02-23 00:00:00+00:00    NaN    NaN
2021-02-24 00:00:00+00:00    NaN    NaN

```

What about absolute stop values? They can be specified the same way as for limit orders:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data.select("BTCUSDT"),
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     sl_stop=100,
...     delta_format="absolute"
... )
>>> pf.orders.price.to_pd()
Open time
2021-02-18 00:00:00+00:00  51552.6
2021-02-19 00:00:00+00:00  51452.6
2021-02-20 00:00:00+00:00    NaN
2021-02-21 00:00:00+00:00    NaN
2021-02-22 00:00:00+00:00    NaN
2021-02-23 00:00:00+00:00    NaN
2021-02-24 00:00:00+00:00    NaN
Freq: D, dtype: float64

```

We can also provide a target stop price directly:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data.select("BTCUSDT"),
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     sl_stop=51452.6,
...     delta_format="target"
... )
>>> pf.orders.price.to_pd()
Open time
2021-02-18 00:00:00+00:00  51552.6
2021-02-19 00:00:00+00:00  51452.6
2021-02-20 00:00:00+00:00    NaN
2021-02-21 00:00:00+00:00    NaN
2021-02-22 00:00:00+00:00    NaN
2021-02-23 00:00:00+00:00    NaN
2021-02-24 00:00:00+00:00    NaN
Freq: D, dtype: float64

```

Since the simulator can keep track of multiple stop types at the same time, we can simulate a specific risk-to-reward (R/R) ratio by setting a stop loss and a take profit. For instance, let's simulate the risk-to-reward ratio of 1/2 based on the ATR:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     sl_stop=1 * sub_atr / sub_data.close,
...     tp_stop=2 * sub_atr / sub_data.close
... )
>>> pf.orders.price.to_pd()
symbol      BTCUSDT      ETHUSDT
Open time
2021-02-18 00:00:00+00:00  51552.600000  1939.610000
2021-02-19 00:00:00+00:00         NaN         NaN
2021-02-20 00:00:00+00:00         NaN  1805.283249
2021-02-21 00:00:00+00:00  57986.366348         NaN
2021-02-22 00:00:00+00:00         NaN         NaN
2021-02-23 00:00:00+00:00         NaN         NaN
2021-02-24 00:00:00+00:00         NaN         NaN
```

The TP order won in the first column, the SL order won in the second column. But is there a way to see the order type and preferably the date of the signal that initiated it? Sure, vectorbt can fulfill all of your wishes! 🤖

```
>>> pf.orders.readable
Order Id Column      Signal Index      Creation Index \
0      0  BTCUSDT 2021-02-18 00:00:00+00:00 2021-02-18 00:00:00+00:00
1      1  BTCUSDT 2021-02-18 00:00:00+00:00 2021-02-21 00:00:00+00:00
2      0  ETHUSDT 2021-02-18 00:00:00+00:00 2021-02-18 00:00:00+00:00
3      1  ETHUSDT 2021-02-18 00:00:00+00:00 2021-02-20 00:00:00+00:00

      Fill Index      Size      Price      Fees      Side      Type \
0 2021-02-18 00:00:00+00:00  0.001940  51552.600000  0.0  Buy  Market
1 2021-02-21 00:00:00+00:00  0.001940  57986.366348  0.0  Sell  Market
2 2021-02-18 00:00:00+00:00  0.051557  1939.610000  0.0  Buy  Market
3 2021-02-20 00:00:00+00:00  0.051557  1805.283249  0.0  Sell  Market

Stop Type
0      None
1      TP
2      None
3      SL

>>> pf.orders.stop_type.to_pd(mapping=True)
symbol      BTCUSDT      ETHUSDT
Open time
2021-02-18 00:00:00+00:00      None      None
2021-02-19 00:00:00+00:00      None      None
2021-02-20 00:00:00+00:00      None      SL
2021-02-21 00:00:00+00:00      TP      None
2021-02-22 00:00:00+00:00      None      None
2021-02-23 00:00:00+00:00      None      None
2021-02-24 00:00:00+00:00      None      None
```

① Orders are represented as rows in a compressed record array. Specific information as a column.

② Specific information as a time series

As we can read from the first DataFrame, the first order is a buy market order (Side and Type) that was filled on the 18th February (Fill Index). The second order in the same column is a sell market order resulting from a TP order (Stop Type) that was issued on the 18th February (Signal Index) but actually filled on the 21st February (Fill Index). The column Creation Index is the same as the column Fill Index since all orders are market orders. The second DataFrame is a common representation of a single piece of information in relation to the time and asset; it's best used when the information should be analyzed as a time series with Pandas.

Finally, let's talk about multiple stop configurations. Here's how to test each stop type independently, that is, SL of 10%, TSL of 10%, TTP of 10% with a 10%-threshold, and TP of 10%:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     sl_stop= vbt.Param([0.1, np.nan, np.nan, np.nan], level=0),
...     tsl_stop=vbt.Param([np.nan, 0.1, 0.1, np.nan], level=0),
...     tsl_th= vbt.Param([np.nan, np.nan, 0.1, np.nan], level=0),
...     tp_stop= vbt.Param([np.nan, np.nan, np.nan, 0.1], level=0),
... )
>>> pf.total_return
sl_stop  tsl_stop  tsl_th  tp_stop  symbol
0.1      NaN      NaN      NaN      BTCUSDT  -0.100000
      NaN      NaN      NaN      ETHUSDT  -0.100000
NaN      0.1      NaN      NaN      BTCUSDT   0.018717
      NaN      NaN      NaN      ETHUSDT  -0.052332
NaN      0.1      0.1      NaN      BTCUSDT   0.018717
      NaN      NaN      NaN      ETHUSDT  -0.163033
NaN      NaN      NaN      0.1      BTCUSDT   0.100000
      NaN      NaN      NaN      ETHUSDT  -0.163033
Name: total_return, dtype: float64
```

To build a product of various stop types, we can specify unique (non-repeating) values and omit the level argument:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     sl_stop=vbt.Param([np.nan, 0.1]),
...     tsl_stop=vbt.Param([np.nan, 0.1]),
...     tsl_th=vbt.Param([np.nan, 0.1]),
```

```
... tp_stop=vbt.Param([np.nan, 0.1]),
... )
```

#### Warning

The number of columns generated from a Cartesian product of multiple parameter combinations grows exponentially with the number of parameter combinations - show mercy to your RAM!

But this would include a combination where `tsl_th` is not NaN but `tsl_stop` is NaN, which doesn't make any sense. What we need though are three combinations: no TSL/TTP stop, TSL stop, and TTP stop. Thus, we need to combine those two arguments manually and link them by the same `level`:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     sl_stop=vbt.Param([np.nan, 0.1], level=0),
...     tsl_stop=vbt.Param([np.nan, 0.1, 0.1], level=1),
...     tsl_th=vbt.Param([np.nan, np.nan, 0.1], level=1),
...     tp_stop=vbt.Param([np.nan, 0.1], level=2),
... )
```

#### Note

If some arguments require a level, it must be defined for all arguments.

We can then call some metric and analyze it using Pandas. For example, let's calculate the average return of using 1) either SL or TP, or 2) SL and TP together:

```
>>> total_return = pf.total_return
>>> sl_stops = total_return.index.get_level_values("sl_stop")
>>> tp_stops = total_return.index.get_level_values("tp_stop")

>>> total_return[np.isnan(sl_stops) | np.isnan(tp_stops)].mean()
-0.045462468872515135

>>> total_return[~np.isnan(sl_stops) & ~np.isnan(tp_stops)].mean()
0.0
```

Seems like combining SL and TP works better on our "huge" time series with 7 points 🤔

The examples above work only if we're testing arguments provided as scalars (single values). How can we backtest various combinations of stop values provided as arrays though, such as those computed from the ATR? Performing that manually would be highly difficult because we would need to tile each stop type array by the number of stop values and then combine all stop type arrays using the Cartesian product. An easier approach is to build a basic indicator that does the preparation of arrays for us:

```
>>> StopOrderPrep = vbt.IF.from_expr("""
...     sl_stop = @p_sl_mult * @in_atr / @in_close
...     tp_stop = @p_tp_mult * @in_atr / @in_close
...     sl_stop, tp_stop
... """)
>>> stop_order_prep = StopOrderPrep.run(
...     close=sub_data.close,
...     atr=sub_data.atr,
...     sl_mult=[np.nan, 1], ❶
...     tp_mult=[np.nan, 1],
...     param_product=True
... )
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     sl_stop=stop_order_prep.sl_stop,
...     tp_stop=stop_order_prep.tp_stop,
... )
>>> pf.total_return
custom_sl_mult  custom_tp_mult  symbol
NaN            NaN            BTCUSD  -0.036398
NaN            NaN            ETHUSD  -0.163033
NaN            1.0            BTCUSD  0.062400
NaN            1.0            ETHUSD  -0.163033
1.0            NaN            BTCUSD  -0.062400
1.0            NaN            ETHUSD  -0.069255
1.0            1.0            BTCUSD  0.062400
1.0            1.0            ETHUSD  -0.069255
Name: total_return, dtype: float64
```

❶ NaN means no order, 1 means one ATR divided by the close

#### Entry point

The entry price of stop orders can be configured using the argument `stop_entry_price`: it can take either the price itself, or any option from the enumerated type **StopEntryPrice**. To be able to distinguish between real prices and options, the options have a negative sign. By default, the entry price is the closing price, even if the entry order itself was filled before the closing price, such as at the opening price; this is because we cannot fill a stop order at the same bar as the entry order, that's why we're postponing the first check to the next bar. Let's see what happens if we define a stop order that can be executed already at the first bar:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     price="open",
...     tp_stop=0.005,
```

```

...     stop_entry_price="fillprice"
... )
>>> pf.orders.price.to_pd()
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  52117.67000  1849.70
2021-02-19 00:00:00+00:00  52378.25835  1938.91
2021-02-20 00:00:00+00:00      NaN      NaN
2021-02-21 00:00:00+00:00      NaN      NaN
2021-02-22 00:00:00+00:00      NaN      NaN
2021-02-23 00:00:00+00:00      NaN      NaN
2021-02-24 00:00:00+00:00      NaN      NaN

```

As we can see, even though the target price 52378.26 could theoretically be filled at the first bar as it's lower than the highest price of 52530.00 of that bar, the check was still postponed to the next bar because of the FS's limitation of one order per bar and asset.

#### Exit point

The number of options to define a stop **exit** price is relatively small: the stop price and the closing price. If the stop has been matched at the beginning of a bar, the stop price will become the opening price. The same for the closing price. By default, a stop order is executed using the stop price, but if the entire simulation should be done on the closing prices only, the second option might be more appropriate.

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     tp_stop=0.05,
...     stop_exit_price=vbt.Param(["stop", "close"])
... )
>>> pf.orders.price.to_pd()
stop_exit_price
symbol      BTCUSDT  ETHUSDT  close  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  51552.60  1939.6100  51552.6  1939.61
2021-02-19 00:00:00+00:00  54130.23      NaN  55906.0      NaN
2021-02-20 00:00:00+00:00      NaN  2036.5905      NaN  1913.00
2021-02-21 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-22 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-23 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-24 00:00:00+00:00      NaN      NaN      NaN      NaN

```

This way, we're effectively delaying the execution of a stop order until the end of a bar. Not only we can change the default stop exit price, but also the default stop exit behavior: using the argument `stop_exit_type` of the type `StopExitType` we can reduce/reverse/reverse+reduce the position instead of closing it. By default, the position is being closed out by issuing an exit signal and disabling accumulation, but we can also reverse it by issuing an opposite entry signal and/or reduce it by keeping the default accumulation:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     tp_stop=0.01, ①
...     stop_exit_type=vbt.Param(["close", "reverse"])
... )
>>> pf.orders.price.to_pd()
stop_exit_type
symbol      close  ETHUSDT  reverse  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  51552.600  1939.6100  51552.6000  1939.6100
2021-02-19 00:00:00+00:00  52068.126  1959.0061  52068.1260  1959.0061
2021-02-20 00:00:00+00:00      NaN      NaN  55346.9400  1935.4500
2021-02-21 00:00:00+00:00      NaN      NaN  56399.6019  1932.1300
2021-02-22 00:00:00+00:00      NaN      NaN  56834.4843  1914.1947
2021-02-23 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-24 00:00:00+00:00      NaN      NaN      NaN      NaN

```

#### ① TP of 1%

Take a look at both columns under the parameter value `reverse`: they now have five orders instead of two, how so? By reversing, we're closing the existing position and opening a new one but in the opposite direction, using a single order. Opening a new position will create a new stop order, thus we've created a never-ending cycle of conditional reversals 😬 If such a cycle is unwanted, we can define `stop_exit_type` as an array where only a particular stop is closed or reversed. Each value in this array should be located at the entry point rather than exit point because this information (along with `stop_exit_price`) is order-anchored and not date-anchored! For example, let's reverse the first order only:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     tp_stop=0.01,
...     stop_exit_type=vbt.index_dict({0: "reverse", "_def": "close"})
... )
>>> pf.orders.price.to_pd()
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  51552.600  1939.6100
2021-02-19 00:00:00+00:00  52068.126  1959.0061
2021-02-20 00:00:00+00:00  55346.940  1935.4500
2021-02-21 00:00:00+00:00      NaN      NaN
2021-02-22 00:00:00+00:00      NaN      NaN
2021-02-23 00:00:00+00:00      NaN      NaN
2021-02-24 00:00:00+00:00      NaN      NaN

```

The first position was reversed while the second position only exited.

What about order type? Upon exit, the order type becomes either a market or limit order. This behavior is controlled by the argument `stop_order_type` of the type **OrderType**. There is also an argument `stop_limit_delta` to specify a delta for the resulting limit order! Let's test a 5% TP order as a stop market order vs. stop limit order with a delta of 1%:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     tp_stop=0.05,
...     stop_order_type=vbt.Param(["market", "limit"]),
...     stop_limit_delta=0.01
... )
>>> pf.orders.price.to_pd()
stop_order_type  market  ETHUSD  limit  ETHUSD
symbol          BTCUSD          BTCUSD
Open time
2021-02-18 00:00:00+00:00  51552.60  1939.6100  51552.6000  1939.61
2021-02-19 00:00:00+00:00  54130.23      NaN  54671.5323      NaN
2021-02-20 00:00:00+00:00      NaN  2036.5905      NaN      NaN
2021-02-21 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-22 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-23 00:00:00+00:00      NaN      NaN      NaN      NaN
2021-02-24 00:00:00+00:00      NaN      NaN      NaN      NaN
```

The latest stop limit order couldn't be filled because the highest price in the entire column `ETHUSD` is `2042.34` while the requested limit price is `1939.61 * 1.05 * 1.01 = 2056.96`. But even if there was a price higher on the 20th February: unless the stop price was triggered at open, the simulator wouldn't be able to use the entire candle to match the limit price because there is no intra-bar data to prove that the limit price can be hit strictly after the stop price. The only information the simulator could have used in this case is the closing price: only if it was higher than the limit price, there would be a guarantee that the limit price has actually been hit.

### Conflicts

Conflicts between stops orders and user-defined signals enjoy the same resolution logic as limit orders, the only difference is in the argument naming: `upon_adj_stop_conflict` and `upon_opp_stop_conflict`. By default, whenever a user-defined signal in any direction is encountered, nothing happens: any stop order remains pending and the signal gets executed as usual; that's because pending stop orders are automatically cancelled whenever the position gets closed out. Let's cancel any signal when there is a stop order pending:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, X, 0, X, 0, X]),
...     exits= pd.Series([0, X, 0, X, 0, X, 0]),
...     direction="both",
...     sl_stop=0.1,
...     upon_adj_stop_conflict="keepignore",
...     upon_opp_stop_conflict="keepignore",
... )
>>> pf.orders.price.to_pd()
symbol          BTCUSD  ETHUSD
Open time
2021-02-18 00:00:00+00:00  51552.60  1939.610
2021-02-19 00:00:00+00:00      NaN      NaN
2021-02-20 00:00:00+00:00      NaN      NaN
2021-02-21 00:00:00+00:00      NaN      NaN
2021-02-22 00:00:00+00:00      NaN  1745.649
2021-02-23 00:00:00+00:00  46397.34  1577.890
2021-02-24 00:00:00+00:00  49676.20      NaN
```

### Adjustment

Similarly to limit orders, stop orders are also stored in one-dimensional record arrays where information is laid out per column: `SignalContext.last_sl_info` for SL, `SignalContext.last_tsl_info` for TSL and TTP, and `SignalContext.last_tp_info` for TP. For instance, create an adjustment function that creates one TP and one SL order based on a maximum allowed, absolute, parameterizable profit and loss (PnL):

```
>>> @njit
... def adjust_func_nb(c, max_loss, max_profit):
...     position_now = c.last_position[c.col] ①
...     if position_now != 0: ②
...         sl_info = c.last_sl_info[c.col]
...         tp_info = c.last_tp_info[c.col]
...         ml = vbt.pf_nb.select_nb(c, max_loss) ③
...         mp = vbt.pf_nb.select_nb(c, max_profit)
...
...         if not vbt.pf_nb.is_stop_info_active_nb(sl_info): ④
...             sl_info.stop = ml / (sl_info.init_price * position_now) ⑤
...         if not vbt.pf_nb.is_stop_info_active_nb(tp_info):
...             tp_info.stop = mp / (sl_info.init_price * position_now)

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     adjust_func_nb=adjust_func_nb,
...     adjust_args=(vbt.Rep("max_loss"), vbt.Rep("max_profit")),
...     broadcast_named_args=dict(
...         max_loss=10,
...         max_profit=10
...     )
... )
>>> pf.exit_trades.pnl.to_pd() ⑥
symbol          BTCUSD  ETHUSD
Open time
2021-02-18 00:00:00+00:00      NaN      NaN
```



|                           |      |       |
|---------------------------|------|-------|
| 2021-02-19 00:00:00+00:00 | NaN  | NaN   |
| 2021-02-20 00:00:00+00:00 | 10.0 | NaN   |
| 2021-02-21 00:00:00+00:00 | NaN  | NaN   |
| 2021-02-22 00:00:00+00:00 | NaN  | -10.0 |
| 2021-02-23 00:00:00+00:00 | NaN  | NaN   |
| 2021-02-24 00:00:00+00:00 | NaN  | NaN   |

- 1 Get the size of the current position. Remember that fields that are not associated with cash are accessible via column!
- 2 We're in the market when the position is not zero
- 3 Our parameters broadcast against the data, thus we need to select the current element flexibly
- 4 There is a handy function `is_stop_info_active_nb` to check whether the current stop information record is active (i.e., the stop value is not NaN)
- 5 Divide the maximum loss/profit by the notional value to get the optimal stop loss/take profit
- 6 Get PnL of each trade as a time series

#### Hint

To avoid re-compiling the entire simulation function with each new run, define the adjustment function in a different cell or even file. Caching won't help though since Numba doesn't allow for caching callbacks.

Our losses and profits are now capped by \$10! Here's what happened. Whenever we provide a custom adjustment function or any other callback, the simulator switches to the flexible mode and enables stop orders. Then, whenever a new position is opened or an existing one is increased, the simulator initializes each stop type with the provided arguments. But since we haven't passed anything related to `SL` and `TP`, their default stop value is NaN, thus they are inactive by default. But regardless of that, they still contain valuable information, such as the initial price. Hence, what's left for us is just to check whether the respective stop is inactive and override the stop value to activate it. We could have also used the functions `set_sl_info_nb` and `set_tp_info_nb` to set the entire information, but there was no need for that.

#### Laddering

We know how to define a single stop to close out an entire position, but what about moving out of a position gradually? For this, we need a way to provide multiple stop values. But there is a problem: the stop arguments should broadcast together with other broadcastable arguments, that is, when providing (for example) `tp_stop` as an array, the first axis should represent time and the second axis should represent columns. But thanks to a new single-axis broadcasting feature, we can notify the broadcaster that the array's rows don't represent time but something completely different.

On practice, `Portfolio.from_signals` has an argument `stop_ladder`, which when enabled can switch to single-axis broadcasting such that providing a list (or one-dimensional array) of values will be considered as a single ladder. We can also provide a two-dimensional array to specify the ladder for each single column.

#### Note

Enabling this argument makes **all** stop arguments behave like ladders - there's no way we can make one argument to be a ladder series and another argument to be a time series! Also, the argument `stop_ladder` is effective on all columns and cannot be provided per column.

In a ladder, each value represents a step with a stop value. Just like in real-world ladders, steps must be ordered from low to high such that a step is executed only after the previous one has been executed first. There's no limit on the number of steps. Let's enable laddering through the argument `stop_ladder` and build our first `TP` ladder with two values: 1% and 5%.

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     stop_ladder=True,
...     tp_stop=[0.01, 0.05]
... )
>>> pf.asset_flow
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.00194  0.051557
2021-02-19 00:00:00+00:00 -0.00097 -0.025778
2021-02-20 00:00:00+00:00 -0.00097 -0.025778
2021-02-21 00:00:00+00:00  0.00000  0.000000
2021-02-22 00:00:00+00:00  0.00000  0.000000
2021-02-23 00:00:00+00:00  0.00000  0.000000
2021-02-24 00:00:00+00:00  0.00000  0.000000
```

Both ladder steps were successfully executed and removed an equal chunk of the position. By default, the exit size distribution of the steps is uniform - the exit size depends only on the number of steps. But what if we wanted remove an amount from the position that is proportional to the step size? Step size in this scenario is the difference of the current stop price and the previous stop price (step range), relative to the difference of the last stop price and the entry price (full range). For this, we can provide the argument `stop_ladder` with the value "weighted", which corresponds to the mode `StopLadderMode.Weighted` (for possible ladder modes see `StopLadderMode`):

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0]),
...     stop_ladder="weighted",
...     tp_stop=[0.01, 0.05]
... )
>>> pf.asset_flow
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.001940  0.051557
2021-02-19 00:00:00+00:00 -0.000388 -0.010311
2021-02-20 00:00:00+00:00 -0.001552 -0.041245
2021-02-21 00:00:00+00:00  0.000000  0.000000
```



```

2021-02-22 00:00:00+00:00 0.000000 0.000000
2021-02-23 00:00:00+00:00 0.000000 0.000000
2021-02-24 00:00:00+00:00 0.000000 0.000000

```

The first step removed  $(0.01 - 0) / (0.05 - 0) = 20\%$  of the initial position while the second stop removed the remaining  $(0.05 - 0.01) / (0.05 - 0) = 80\%$ .

Let's say we want to have two parallel ladders: SL and TP. What happens to the exit size distribution if both ladders are executed partially?

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     stop_ladder="uniform",
...     tp_stop=[0.1, 0.2],
...     sl_stop=[0.05, 0.1, 0.15]
... )

>>> pf.orders.stop_type.to_pd(mapping=True)
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  None    None
2021-02-19 00:00:00+00:00  None    None
2021-02-20 00:00:00+00:00    TP     SL
2021-02-21 00:00:00+00:00  None    None
2021-02-22 00:00:00+00:00    SL     SL
2021-02-23 00:00:00+00:00    SL     SL
2021-02-24 00:00:00+00:00  None    None

>>> pf.asset_flow
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.001940  0.051557
2021-02-19 00:00:00+00:00  0.000000  0.000000
2021-02-20 00:00:00+00:00 -0.000970 -0.017186
2021-02-21 00:00:00+00:00  0.000000  0.000000
2021-02-22 00:00:00+00:00 -0.000647 -0.017186
2021-02-23 00:00:00+00:00 -0.000323 -0.017186
2021-02-24 00:00:00+00:00  0.000000  0.000000

```

We see that the first TP step in the column `BTCUSDT` removed the half of the position because there are only two steps in the TP ladder. But then an SL step was hit and removed 33.3% of the initial position because there are three steps in the SL ladder. The final step then removed the remainder of the position. But what if our intention is to move out of the position respective to the current position and not the initial one? If the position suddenly changed, the exit sizes would be redistributed based on this change. This can be done by providing `StopLadderMode.AdaptUniform` or `StopLadderMode.AdaptWeighted` mode:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     stop_ladder="adaptuniform",
...     tp_stop=[0.1, 0.2],
...     sl_stop=[0.05, 0.1, 0.15]
... )

>>> pf.orders.stop_type.to_pd(mapping=True)
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  None    None
2021-02-19 00:00:00+00:00  None    None
2021-02-20 00:00:00+00:00    TP     SL
2021-02-21 00:00:00+00:00  None    None
2021-02-22 00:00:00+00:00    SL     SL
2021-02-23 00:00:00+00:00    SL     SL
2021-02-24 00:00:00+00:00  None    None

>>> pf.asset_flow
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.001940  0.051557
2021-02-19 00:00:00+00:00  0.000000  0.000000
2021-02-20 00:00:00+00:00 -0.000970 -0.017186
2021-02-21 00:00:00+00:00  0.000000  0.000000
2021-02-22 00:00:00+00:00 -0.000323 -0.017186
2021-02-23 00:00:00+00:00 -0.000323 -0.017186
2021-02-24 00:00:00+00:00  0.000000  0.000000

```

The both SL steps in the first column now remove exactly  $1 / 3$  of the new position and act as if they were created right after the position change by the first TP step.

Now, let's illustrate how to specify a different ladder for a different column. For this, we need to construct a two-dimensional array. Since some ladders may have a smaller number of steps than the others, we need to pad them with NaN:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     stop_ladder=True,
...     tp_stop=np.column_stack((
...         np.array([0.1, 0.2, np.nan]), ①
...         np.array([0.01, 0.02, 0.03]) ②
...     ))
... )

>>> pf.asset_flow
symbol      BTCUSDT  ETHUSDT
Open time

```

```

2021-02-18 00:00:00+00:00 0.00194 0.051557
2021-02-19 00:00:00+00:00 0.00000 -0.017186
2021-02-20 00:00:00+00:00 -0.00097 -0.017186
2021-02-21 00:00:00+00:00 0.00000 0.000000
2021-02-22 00:00:00+00:00 0.00000 0.000000
2021-02-23 00:00:00+00:00 0.00000 0.000000
2021-02-24 00:00:00+00:00 0.00000 0.000000

```

① Gets applied on `BTCUSDT`

② Gets applied on `ETHUSDT`

The padding step wouldn't be necessary if we provided the ladders as parameters:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     stop_ladder=True,
...     tp_stop=vbt.Param([
...         np.array([0.1, 0.2]), ①
...         np.array([0.01, 0.02, 0.03]) ②
...     ])
... )
>>> pf.asset_flow

```

| tp_stop                   | array_0  | ETHUSDT  | array_1   | ETHUSDT   |
|---------------------------|----------|----------|-----------|-----------|
| symbol                    | BTCUSDT  |          | BTCUSDT   | ETHUSDT   |
| Open time                 |          |          |           |           |
| 2021-02-18 00:00:00+00:00 | 0.00194  | 0.051557 | 0.001940  | 0.051557  |
| 2021-02-19 00:00:00+00:00 | 0.00000  | 0.000000 | -0.000647 | -0.017186 |
| 2021-02-20 00:00:00+00:00 | -0.00097 | 0.000000 | -0.000647 | -0.017186 |
| 2021-02-21 00:00:00+00:00 | 0.00000  | 0.000000 | -0.000647 | 0.000000  |
| 2021-02-22 00:00:00+00:00 | 0.00000  | 0.000000 | 0.000000  | 0.000000  |
| 2021-02-23 00:00:00+00:00 | 0.00000  | 0.000000 | 0.000000  | 0.000000  |
| 2021-02-24 00:00:00+00:00 | 0.00000  | 0.000000 | 0.000000  | 0.000000  |

① Gets applied on both columns

② Gets applied on both columns

Finally, let's discuss how to specify our own exit size for each ladder step. Since there are no arguments for specifying the exit size, we need change it inside an adjustment or signal function. Remember to use an adjustment function if you already have signal arrays, and a signal function if you generate your signals dynamically. Inside the callback, we need to 1) retrieve the current information record of the stop type we've defined the ladder for, 2) select the exit size corresponding to the current step (available via the record field `step`), and 3) write the exit size to the record. Once the step is hit, the user-defined exit size will be used instead of the default one.

#### Note

The default size and size type in any record are NaN and -1 respectively. Only once the step is hit, they will be internally replaced by the calculated values. Thus, you can check whether the ladder uses the default exit size by testing these fields against these values.

In the following example we go into a trade with 6 units, and then test two TP ladders: 1) remove 3 units at the level of 10% and the remaining 3 units at the level of 20%, and 2) remove 1 unit at the level of 1%, another 2 units at the level of 2%, and the remaining 3 units at the level of 3%.

```

>>> @njit
... def adjust_func_nb(c, exit_size, exit_size_type):
...     tp_info = c.last_tp_info[c.col]
...     if vbt.pf_nb.is_stop_info_ladder_active_nb(tp_info): ①
...         if np.isnan(tp_info.exit_size): ②
...             tp_info.exit_size = vbt.pf_nb.select_nb(c, exit_size, i=tp_info.step) ③
...             tp_info.exit_size_type = vbt.pf_nb.select_nb(c, exit_size_type, i=tp_info.step)

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     adjust_func_nb=adjust_func_nb,
...     adjust_args=(vbt.Rep("exit_size"), vbt.Rep("exit_size_type")), ④
...     stop_ladder=True,
...     tp_stop=vbt.Param([
...         np.array([0.1, 0.2]),
...         np.array([0.01, 0.02, 0.03])
...     ], level=0), ⑤
...     broadcast_named_args=dict(
...         exit_size=vbt.BCO( ⑥
...             vbt.Param([
...                 np.array([3, 3]),
...                 np.array([1, 2, 3])
...             ], level=0, hide=True), ⑦
...             axis=1,
...             merge_kwargs=dict( ⑧
...                 reset_index="from_start",
...                 fill_value=np.nan,
...             )
...         ),
...         exit_size_type=vbt.BCO(
...             vbt.pf_enums.SizeType.Amount, ⑨
...             axis=1,
...             merge_kwargs=dict(
...                 reset_index="from_start",
...                 fill_value=-1,
...             )
...         )
...     )

```

```

...     ),
...     size=6,
...     init_cash="auto"
... )
>>> pf.asset_flow
tp_stop      array_0      array_1
symbol      BTCUSDT  ETHUSD  BTCUSDT  ETHUSD
Open time
2021-02-18 00:00:00+00:00    6.0    6.0    6.0    6.0
2021-02-19 00:00:00+00:00    0.0    0.0   -1.0   -1.0
2021-02-20 00:00:00+00:00   -3.0    0.0   -2.0   -2.0
2021-02-21 00:00:00+00:00    0.0    0.0   -3.0    0.0
2021-02-22 00:00:00+00:00    0.0    0.0    0.0    0.0
2021-02-23 00:00:00+00:00    0.0    0.0    0.0    0.0
2021-02-24 00:00:00+00:00    0.0    0.0    0.0    0.0

```

- 1 Check whether the TP ladder is active using `is_stop_info_ladder_active_nb`
- 2 We want to override the exit size and type only once per step; if the exit size is not NaN, then it has already been overridden.
- 3 Since the exit size and type arrays at least partially broadcast against the target shape, we can use `select_nb` by providing the current step as `i`
- 4 Exit size and type are passed to `broadcast_named_args`. These templates are then substituted by the actual arrays once the broadcasting is over.
- 5 Specify the same level to avoid building the product between `tp_stop`, `exit_size`, and `exit_size_type`. This way, the three parameters will be considered as a single parameter.
- 6 Exit size and type must broadcast in the same way as our ladders, thus use `BCO` to change the default behavior
- 7 We should provide the exit size in an identical fashion to `tp_stop`. Additionally, we'll hide the parameter from the final columns as it's redundant.
- 8 These keyword arguments are required if parameter arrays have different shapes. In this case, smaller arrays will be padded with NaN to match the length of the longest array.
- 9 In our example, the size type is the same for all sizes, thus we can just use a single value here

## Dynamic

We've learned that order-related arguments such as `size` act as a facade of the backtesting process with `FS`: they are meant to be anchored by time rather than signals, so we cannot change them dynamically. This is proved by the fact that any signal function (`signal_func_nb`) returns only signals and nothing else. But what if a signal needs to have a size or other order information different from the default one?

Remember that vectorbt takes order information as arrays and processes them iteratively: it goes through each row and column, reads the current element, and uses this element along with the returned signals. Theoretically, if we override the current element in the signal function, vectorbt should pick it and use it as new order information! The only question is how to get access to the respective array if context (`c`) doesn't list it? Use templates! For example, by passing `vbt.Rep("size")` in `signal_args`, the method will search for a variable called `size` in its template context and pass it to the signal function. Conveniently for us, the template context contains all the information passed to the method, including the order-related arguments.

But there is a catch: to avoid consuming too much memory, vectorbt keeps most array-like arguments as two-dimensional arrays with only one element and uses flexible indexing (see [this](#) and [this](#)) to select that one element at each single row and column. That is, we either have to tell the broadcaster to build the full array and then override each element (`arr[c.i, c.col] = ...`), which is wasteful but still may be wanted if there is also a requirement to keep track of all the values used previously. Or, we can make an array with only one element per column and override that one element (`arr[0, c.col] = ...`) to make vectorbt use flexible indexing.

Let's demonstrate both approaches by buying for \$10 when a long signal is encountered and selling for \$5 when a short signal is encountered. Here's the first approach using a full-sized array:

```

>>> @njit
... def signal_func_nb(c, long_signals, short_signals, size):
...     long_signal = vbt.pf_nb.select_nb(c, long_signals)
...     short_signal = vbt.pf_nb.select_nb(c, short_signals)
...     if long_signal:
...         size[c.i, c.col] = 10  ❶
...     if short_signal:
...         size[c.i, c.col] = 5
...     return long_signal, False, short_signal, False

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     signal_func_nb=signal_func_nb,
...     signal_args=(
...         vbt.Rep("long_signals"),
...         vbt.Rep("short_signals"),
...         vbt.Rep("size")  ❷
...     ),
...     size=vbt.RepEval("np.full(wrapper.shape_2d, np.nan)"),  ❸
...     size_type="value",
...     accumulate=True,
...     broadcast_named_args=dict(
...         long_signals= pd.Series([X, 0, 0, 0, X, 0, 0]),
...         short_signals= pd.Series([0, 0, X, 0, 0, 0, X]),
...     )
... )
>>> pf.orders.value.to_pd()
symbol      BTCUSDT  ETHUSD
Open time
2021-02-18 00:00:00+00:00    10.0    10.0
2021-02-19 00:00:00+00:00    NaN     NaN
2021-02-20 00:00:00+00:00    -5.0    -5.0
2021-02-21 00:00:00+00:00    NaN     NaN
2021-02-22 00:00:00+00:00    10.0    10.0

```

|                           |      |      |
|---------------------------|------|------|
| 2021-02-23 00:00:00+00:00 | NaN  | NaN  |
| 2021-02-24 00:00:00+00:00 | -5.0 | -5.0 |

- 1 Set the value under the current row and column to be picked by vectorbt
- 2 Forward `size` we built below to the signal function using templates
- 3 Using this template we can delay the creation of the array to the point where the target shape is known (i.e., after broadcasting all arrays), which gets available via `wrapper.shape`

#### Example

Can we somehow get access to the filled array after the simulation? Yes! The trick is to create an empty dictionary, pass it to the template, and save the created array to the dictionary inside the template expression:

```
>>> memory = {}
>>> pf = vbt.Portfolio.from_signals(
...     # ...
...     size=vbt.RepEval("""
...         size = np.full(wrapper.shape_2d, np.nan)
...         memory["size"] = size
...         return size
...     """,
...     context=dict(memory=memory), ①
...     context_merge_kwargs=dict(nested=False) ②
... ),
...     # ...
... )
>>> memory["size"]
[[10. 10.]
 [nan nan]
 [ 5.  5.]
 [nan nan]
 [10. 10.]
 [nan nan]
 [ 5.  5.]]
```

① Everything that we pass as `context` will be available as a variable in the template

② We don't want to lose the reference to the dictionary `memory`, that's why we disable nested dictionary merging when the local context is being merged with the global context

Here's the second approach using a one-element array:

```
>>> @njit
... def signal_func_nb(c, long_signals, short_signals, size):
...     long_signal = vbt.pf_nb.select_nb(c, long_signals)
...     short_signal = vbt.pf_nb.select_nb(c, short_signals)
...     if long_signal:
...         size[0, c.col] = 10 ①
...     if short_signal:
...         size[0, c.col] = 5
...     return long_signal, False, short_signal, False

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     signal_func_nb=signal_func_nb,
...     signal_args=(
...         vbt.Rep("long_signals"),
...         vbt.Rep("short_signals"),
...         vbt.Rep("size")
...     ),
...     size=vbt.RepEval("np.full((1, wrapper.shape_2d[1]), np.nan)"), ②
...     size_type="value",
...     accumulate=True,
...     broadcast_named_args=dict(
...         long_signals= pd.Series([X, 0, 0, 0, X, 0, 0]),
...         short_signals=pd.Series([0, 0, X, 0, 0, 0, X]),
...     )
... )
>>> pf.orders.value.to_pd()
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00    10.0    10.0
2021-02-19 00:00:00+00:00     NaN     NaN
2021-02-20 00:00:00+00:00    -5.0    -5.0
2021-02-21 00:00:00+00:00     NaN     NaN
2021-02-22 00:00:00+00:00    10.0    10.0
2021-02-23 00:00:00+00:00     NaN     NaN
2021-02-24 00:00:00+00:00    -5.0    -5.0
```

① Since our array has only one element, vectorbt will pick its value for any row and column, thus we need to override this exact element

② In contrast to the previous example, here we only need one row

#### Entry laddering

To demonstrate the full power of this trick, we'll create a custom entry ladder! Whenever a user-defined entry signal is discovered, it will be split into multiple smaller entry signals, each executed after a predefined number of bars and adding a predefined percentage of resources to the current position. Also, we'll show how to group arguments using named tuples and define custom record arrays that hold complex temporary information. Finally, we'll show how to parameterize the ladder and test two completely different ladders with a single simulation.

```
>>> Signals = namedtuple("Signals", [ ①
...     "entries",
...     "exits"
... ])
>>> Order = namedtuple("Order", [ ②
```

```

...     "size",
...     "size_type"
... ]
>>> ladder_dt = np.dtype([ 3
...     ("after_n_bars", int_),
...     ("exit_pct", float_)
... ])
>>> ladder_info_dt = np.dtype([ 4
...     ("init_idx", int_),
...     ("step", int_)
... ], align=True)

>>> @njit 5
... def signal_func_nb(c, signals, order, ladders, last_ladder_info): 6
...     is_entry = vbt.pf_nb.select_nb(c, signals.entries)
...     is_exit = vbt.pf_nb.select_nb(c, signals.exits)
...     position_now = c.last_position[c.col]
...     ladder_info = last_ladder_info[c.col] 7
...
...     if position_now > 0 and is_exit: 8
...         ladder_info["init_idx"] = -1 9
...         ladder_info["step"] = -1
...         order.size[c.i, c.col] = np.inf 10
...         order.size_type[c.i, c.col] = vbt.pf_enums.SizeType.Amount
...         return False, True, False, False 11
...
...     if is_entry: 12
...         ladder_info["init_idx"] = c.i
...         ladder_info["step"] = 0
...         return False, False, False, False
...
...     step = ladder_info["step"]
...     if step != -1 and step < ladders.shape[0]: 13
...         after_n_bars = ladders.after_n_bars[step, c.col] 14
...         if after_n_bars != -1 and c.i >= ladder_info["init_idx"] + after_n_bars: 15
...             ladder_info["step"] = step + 1 16
...             order.size[c.i, c.col] = ladders.exit_pct[step, c.col] 17
...             order.size_type[c.i, c.col] = vbt.pf_enums.SizeType.Percent
...             return True, False, False, False 18
...
...     return False, False, False, False 19

>>> pf = vbt.Portfolio.from_signals(
...     sub_data.select("BTCUSD"),
...     signal_func_nb=signal_func_nb,
...     signal_args=( 20
...         vbt.RepEval(
...             "Signals(entries, exits)",
...             context=dict(Signals=Signals) 21
...         ),
...         vbt.RepEval(
...             "Order(size, size_type)",
...             context=dict(Order=Order)
...         ),
...         vbt.Rep("ladders"), 22
...         vbt.RepEval( 23
...             "np.full(wrapper.shape_2d[1], np.array((-1, -1), ladder_info_dt))",
...             context=dict(ladder_info_dt=ladder_info_dt)
...         )
...     ),
...     size=vbt.RepEval("np.full(wrapper.shape_2d, np.nan)", 24
...     size_type=vbt.RepEval("np.full(wrapper.shape_2d, -1)",
...     accumulate=True, 25
...     broadcast_named_args=dict(
...         entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...         exits= pd.Series([0, 0, 0, 0, 0, 0, 0]),
...         ladders=vbt.BCO( 26
...             vbt.Param([ 27
...                 np.array([ 28
...                     (2, 1/4), 29
...                     (3, 1/3),
...                     (4, 1/2),
...                     (5, 1/1)
...                 ], dtype=ladder_dt),
...                 np.array([
...                     (2, 1/2),
...                     (4, 1/1),
...                 ], dtype=ladder_dt),
...             ]),
...             axis=1, 30
...             merge_kwargs=dict( 31
...                 reset_index="from_start",
...                 fill_value=np.array((-1, np.nan), dtype=ladder_dt),
...             )
...         )
...     ),
...     ),
>>> pf.asset_flow
ladder          array_0  array_1
Open time
2021-02-18 00:00:00+00:00  0.000000  0.000000
2021-02-19 00:00:00+00:00  0.000000  0.000000
2021-02-20 00:00:00+00:00  0.000448  0.000895
2021-02-21 00:00:00+00:00  0.000435  0.000000
2021-02-22 00:00:00+00:00  0.000462  0.000924
2021-02-23 00:00:00+00:00  0.000511  0.000000
2021-02-24 00:00:00+00:00  0.000000  0.000000

```

- ❶ Create a **namedtuple** to define a container for multiple arrays. Otherwise, the amount of arrays passed to the signal function would take a lot of space and be poorly readable. This named tuple contains two signal arrays: entries and exits.
- ❷ The second named tuple contains order-related arrays that are meant to be overridden in the signal function: size and size type. You can add more fields to it if needed.
- ❸ Ladders are stored in a two-dimensional array where each column represents a ladder and each row a step within this ladder. Each step should define how many bars need to pass and how much we need to order (basically a tuple). Regular NumPy arrays cannot store such information, but structured arrays can! Here, we define the data type for such an array.
- ❹ We also need to keep track of the index of the user-defined entry and the current step being processed. We need this per ladder (column), thus we will create a one-dimensional structured array.
- ❺ If you're finished with the function, don't forget to put it into another cell to avoid re-compilation! Also, don't cache it because it might break if you change some named tuples.
- ❻ Our function takes the context, and the initialized named tuples and structured arrays that we defined above
- ❼ Similarly to built-in information arrays such as `last_position` and `last_sl_info`, our latest ladder information is also laid out per column
- ❽ If we encounter a user-defined exit signal, close out the position
- ❾ Reset the temporary information to deactivate our laddering logic at the next bars
- ❿ Override the order-related information for this signal, here we go all out
- ⓫ Long exit
- ⓬ If we encounter a user-defined entry signal, write the current bar index to the temporary information, set the current step to zero to activate our laddering logic, and return no signal to skip to the next bar
- ⓭ Check whether the ladder is active and the current step is lower than the total number of steps
- ⓮ Remember that ladders are stored in a two-dimensional array with a ladder per column, that is, the first axis represents steps and the second axis represents columns
- ⓯ Next, we need to check whether the condition of the current step has been satisfied, that is, whether the current bar index comes after the target bar index. But first, we must check whether the value is defined; this is because ladders may have a different number of steps.
- ⓰ Increment the current step
- ⓱ Execute the signal by ordering the requested percentage of the available resources
- ⓲ Long entry
- ⓳ If there was no action during this bar, return no signal
- ⓴ Use templates to construct named tuples and record arrays. The arrays defined in `broadcast_named_args` will be automatically recognized in template expressions.
- ⓵ We need to provide the classes via a context since they are not automatically available in the expressions
- ⓶ We will define ladders in `broadcast_named_args` to broadcast it together with data
- ⓷ Create a structured NumPy array that holds temporary information per ladder (column). Here, `np.array((-1, -1), ladder_info_dt)` is a single value that instructs NumPy to use the data type `ladder_info_dt` and fill both fields with -1.
- ⓸ Size and size type are built-in arguments that we want to override, thus make them of the full shape
- ⓹ Don't forget to enable accumulation to be able to gradually increase the position
- ⓺ Use **BCO** to control broadcasting of a single argument
- ⓻ Use **Param** to test multiple values (here ladder arrays)
- ⓼ Luckily, any structured array can be initialized using regular tuples
- ⓽ For example, in the first step, once we're two bars away from the user-defined entry signal, execute an order for 25% of the available cash balance
- ⓿ Remember that the first axis are steps, not bars, thus our array isn't broadcastable along the time axis (`axis=0`), only along the column axis (`axis=1`)
- ⓾ What happens if one array has fewer steps than another? Under the hood, both arrays will be stacked along columns, but stacking cannot work if they have different shapes. Here, we specify that whenever an array has insufficient number of rows, it should be padded using `fill_value`.

The strategy has correctly split the entry signal into a set of smaller entry orders distributed over time. The code above is a great template for defining dynamic signal strategies of any complexity.

## Grouping

Until now, we processed the columns `BTCUSDT` and `ETHUSDT` separately, that is, the second column is processed strictly after the first column. But what if there is a need to put both columns into the same portfolio for analysis? By introducing a grouping, we can make a group of columns to be treated as a single column (read more [here](#)). Here, we need to distinguish between grouping after and before the simulation.

## After simulation

When the main scenario is to simulate columns **separately** and then subsequently analyze them as a whole, we can group-by the columns after the simulation, either when querying a specific metric of interest:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries= pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits=   pd.Series([0, 0, 0, X, 0, 0, 0]),
```

```
...     short_entries=pd.Series([0, 0, 0, 0, X, 0, 0]),
...     short_exits= pd.Series([0, 0, 0, 0, 0, 0, X]),
... )
>>> pf.get_value(group_by=True)
Open time
2021-02-18 00:00:00+00:00    200.000000
2021-02-19 00:00:00+00:00    209.238037
2021-02-20 00:00:00+00:00    206.946937
2021-02-21 00:00:00+00:00    211.045749
2021-02-22 00:00:00+00:00    211.045749
2021-02-23 00:00:00+00:00    232.943589
2021-02-24 00:00:00+00:00    228.775332
Freq: D, Name: group, dtype: float64
```

Or, by quick-fixing the entire portfolio:

```
>>> grouped_pf = pf.replace(wrapper=pf.wrapper.replace(group_by=True)) ❶
>>> grouped_pf.value
Open time
2021-02-18 00:00:00+00:00    200.000000
2021-02-19 00:00:00+00:00    209.238037
2021-02-20 00:00:00+00:00    206.946937
2021-02-21 00:00:00+00:00    211.045749
2021-02-22 00:00:00+00:00    211.045749
2021-02-23 00:00:00+00:00    232.943589
2021-02-24 00:00:00+00:00    228.775332
Freq: D, Name: group, dtype: float64
```

❶ Group-by instruction is part of the portfolio's wrapper

What the portfolio above did was aggregate various metrics such as the initial capital and time series such as the cash flows along the column axis. There are some time series that cannot be aggregated though, such as the asset flows:

```
>>> grouped_pf.asset_flow
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.001940  0.051557
2021-02-19 00:00:00+00:00  0.000000  0.000000
2021-02-20 00:00:00+00:00  0.000000  0.000000
2021-02-21 00:00:00+00:00 -0.001940 -0.051557
2021-02-22 00:00:00+00:00 -0.002059 -0.056080
2021-02-23 00:00:00+00:00  0.000000  0.000000
2021-02-24 00:00:00+00:00  0.002059  0.056080
```

We can still disable the grouping for individual metrics by passing `group_by=False`. What we cannot do after the simulation though is to introduce cash sharing because it can be only applied during the simulation where it has real consequences on the trading logic.

## Before simulation

Adding a group-by instruction before the simulation may or may not have consequences on the simulation. For instance, grouping with cash sharing will always have consequences, while grouping without cash sharing will only have consequences when `FS` is run in the flexible mode (that is, when any of the default callbacks are overridden); in this case, columns are grouped for the user to take advantage of the grouping in the callbacks, while all the calculations are still done on the per-column basis as if there was no grouping at all. Let's make use of this fact and limit the number of active positions to just one in each group:

```
>>> @njit
... def signal_func_nb(c, entries, exits):
...     is_entry = vbt.pf_nb.select_nb(c, entries)
...     is_exit = vbt.pf_nb.select_nb(c, exits)
...     other_in_position = False
...     for col in range(c.from_col, c.to_col): ❶
...         if col != c.col and c.last_position[col] != 0:
...             other_in_position = True
...             break
...     if other_in_position: ❷
...         return False, False, False, False
...     return is_entry, is_exit, False, False ❸

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     signal_func_nb=signal_func_nb,
...     signal_args=(vbt.Rep("entries"), vbt.Rep("exits")),
...     broadcast_named_args=dict(
...         entries=pd.DataFrame({
...             0: [X, 0, 0, 0, 0, 0, X],
...             1: [0, X, 0, X, 0, 0, 0]
...         }),
...         exits=pd.DataFrame({
...             0: [0, 0, X, 0, X, 0, 0],
...             1: [0, 0, 0, 0, 0, 0, X]
...         })
...     ),
...     group_by=True ❹
... )
>>> pf.asset_flow
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.00194  0.000000
2021-02-19 00:00:00+00:00  0.00000  0.000000
2021-02-20 00:00:00+00:00 -0.00194  0.000000
2021-02-21 00:00:00+00:00  0.00000  0.051719
```

```

2021-02-22 00:00:00+00:00 0.00000 0.000000
2021-02-23 00:00:00+00:00 0.00000 -0.051719
2021-02-24 00:00:00+00:00 0.00218 0.000000

```

- ❶ Go through the columns in this group and check whether there are open positions apart from this one
- ❷ If true, return no signal
- ❸ If false, return the signal
- ❹ Put all columns into the same group. For multiple groups, use `vbt.ExceptLevel("symbol")`

The first signal in the column `ETHUSDT` couldn't execute because there was already a position in the column `BTCUSDT`, but once it was closed, the second signal could finally go through.

### Sorting

Orders are sorted only in groups with cash sharing, and only in two cases: when orders in different columns should be executed in different bar zones (i.e., at different times) such that they need to be sorted by time, and when orders are instructed to be sorted by order value as part of the automatic call sequence enabled by `call_seq="auto"`. Both cases cannot be combined! Let's take a look at the first case where the first column is executed using the closing price and the second column using the opening price:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     price=["close", "open"], ❶
...     size=[[100, 50]],
...     size_type="value",
...     group_by=True,
...     cash_sharing=True ❷
... )
>>> pf.orders.get_value(group_by=False).to_pd()

```

| symbol                    | BTCUSDT | ETHUSDT |
|---------------------------|---------|---------|
| Open time                 |         |         |
| 2021-02-18 00:00:00+00:00 | 50.0    | 50.0    |
| 2021-02-19 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-22 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-23 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-24 00:00:00+00:00 | NaN     | NaN     |

- ❶ If we want to provide any information per column, we need to build a two-dimensional array with exactly one row (any array-like object will be transformed into a NumPy array, thus passing a list works too)
- ❷ Turn on cash sharing

As we can see, the second column was executed first and has ended up using the half of the capital. If both assets were executed in the same bar zone though, they wouldn't be sorted and would be just normally executed from the left column to the right:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     price="close",
...     size=[[100, 50]],
...     size_type="value",
...     group_by=True,
...     cash_sharing=True
... )
>>> pf.orders.get_value(group_by=False).to_pd()

```

| symbol                    | BTCUSDT | ETHUSDT |
|---------------------------|---------|---------|
| Open time                 |         |         |
| 2021-02-18 00:00:00+00:00 | 100.0   | NaN     |
| 2021-02-19 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-22 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-23 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-24 00:00:00+00:00 | NaN     | NaN     |

Even though the second column requested \$50 of assets, the simulator couldn't fulfill that request because the default initial capital is \$100, which has been all used up by the first column. To sort by order value, use the automatic call sequence:

```

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     price="close",
...     size=[[100, 50]],
...     size_type="value",
...     group_by=True,
...     cash_sharing=True,
...     call_seq="auto" ❶
... )
>>> pf.orders.get_value(group_by=False).to_pd()

```

| symbol                    | BTCUSDT | ETHUSDT |
|---------------------------|---------|---------|
| Open time                 |         |         |
| 2021-02-18 00:00:00+00:00 | 50.0    | 50.0    |
| 2021-02-19 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-20 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-21 00:00:00+00:00 | NaN     | NaN     |
| 2021-02-22 00:00:00+00:00 | NaN     | NaN     |



|                           |     |     |
|---------------------------|-----|-----|
| 2021-02-23 00:00:00+00:00 | NaN | NaN |
| 2021-02-24 00:00:00+00:00 | NaN | NaN |

### 1 Here

This has made the second column execute first. But if we attempted to sort by order value when the columns had to execute their orders at different times, the simulation would fail:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     price=[["close", "open"]],
...     size=[100, 50],
...     size_type="value",
...     group_by=True,
...     cash_sharing=True,
...     call_seq="auto"
... )
ValueError: Cannot sort orders by value if they are executed at different times
```

And it makes sense if we think more about it: if the simulator sorted both columns by value, the second column would come first, but how it's possible if it should execute at the beginning of the bar? Remember that each bar is divided into three zones: open, middle, and close. The only possibility to sort orders by value is to use either the opening or the closing point of the bar. We cannot use the middle of the bar because we don't really know what happens there. Since we give vectorbt clues on where in the bar an order should execute by using `-np.inf` (bar open) and `np.inf` (bar close) as order price, passing any custom prices would be considered happening in the middle of a bar and thus fail as well:

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries=pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     price=sub_data.close,
...     size=[100, 50],
...     size_type="value",
...     group_by=True,
...     cash_sharing=True,
...     call_seq="auto"
... )
ValueError: Cannot sort orders by value if they are executed at different times
```

Even if we know that the provided array represents the closing price, vectorbt doesn't know about it. Thus, make sure to use exclusively any option from [PriceType](#) when the automatic call sequence should be enabled. If you're using stop orders, make sure to use `PriceType.Close` and `StopExitPrice.Close` to execute all orders at the closing price, for instance, for rebalancing. Limit orders, on the other hand, are never executed at the closing price, thus using them together with the automatic call sequence is hardly possible.

## Custom outputs

When the portfolio has been simulated, and we've got our new shiny portfolio instance, the first thing we usually do is run various statistics. Since most statistics are based on returns, the portfolio instance may sometimes need to repeatedly re-calculate the returns, which is a slow process because it has to translate order records into asset flows, cash flows, assets, cash, asset value, portfolio value, and finally, returns. Such operations are known as "reconstructions" because they aim to derive a certain property of the simulation post factum. Hence, performance analysis is usually the main performance bottleneck in a typical backtesting pipeline. Gladly, there is an argument `save_returns` that, when enabled, pre-calculates the returns during the simulation and makes them available for all the metrics that need them (read more [here](#)):

```
>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries= pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     short_entries=pd.Series([0, 0, 0, 0, X, 0, 0]),
...     short_exits= pd.Series([0, 0, 0, 0, 0, 0, X]),
...     save_returns=True
... )
>>> pf.returns
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.000000  0.000000
2021-02-19 00:00:00+00:00  0.084446  0.007935
2021-02-20 00:00:00+00:00 -0.001159 -0.021483
2021-02-21 00:00:00+00:00  0.028069  0.010732
2021-02-22 00:00:00+00:00  0.000000  0.000000
2021-02-23 00:00:00+00:00  0.096079  0.112338
2021-02-24 00:00:00+00:00 -0.013245 -0.023012
```

To check whether they make sense, compare them to the reconstructed returns:

```
>>> pf.get_returns()
symbol      BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.000000  0.000000
2021-02-19 00:00:00+00:00  0.084446  0.007935
2021-02-20 00:00:00+00:00 -0.001159 -0.021483
2021-02-21 00:00:00+00:00  0.028069  0.010732
2021-02-22 00:00:00+00:00  0.000000  0.000000
2021-02-23 00:00:00+00:00  0.096079  0.112338
2021-02-24 00:00:00+00:00 -0.013245 -0.023012
```

1 Every attribute with `get_` triggers a reconstruction

Under the hood, FS creates a named tuple with an uninitialized returns array and gradually fills it at the end of each bar. Once the simulation is over, this tuple gets attached to the created portfolio instance under the attribute **Portfolio.in\_outputs**:

```
>>> pf.in_outputs
FSInOutputs(returns=array([[ 0.,          0.],
 [ 0.08444579,  0.00793458],
 [-0.00115927, -0.02148338],
 [ 0.02806853,  0.01073183],
 [ 0.,          0.],
 [ 0.09607864,  0.11233812],
 [-0.01324464, -0.02301153]]))
```

Since most things returned by the simulator are in NumPy format, we can use the method **Portfolio.get\_in\_output** to wrap any array with Pandas:

```
>>> pf.get_in_output("returns")
symbol          BTCUSDT  ETHUSDT
Open time
2021-02-18 00:00:00+00:00  0.000000  0.000000
2021-02-19 00:00:00+00:00  0.084446  0.007935
2021-02-20 00:00:00+00:00 -0.001159 -0.021483
2021-02-21 00:00:00+00:00  0.028069  0.010732
2021-02-22 00:00:00+00:00  0.000000  0.000000
2021-02-23 00:00:00+00:00  0.096079  0.112338
2021-02-24 00:00:00+00:00 -0.013245 -0.023012
```

But there is one catch: the argument `save_returns` is only available in the fixed simulation mode, that is, when none of the default callbacks have been overridden. As soon as we enter the flexible mode, we need to define our own in-output tuple under the argument `in_outputs` and fill it with any information we want, usually in the segment post-processing function `post_segment_func`. This callback takes the context **SignalSegmentContext**, which is the same as **SignalContext** but without the information on the current column since a segment encompasses multiple columns in a particular group at a particular row, hence, we need to go over the columns in the current segment manually.

For demonstration, let's develop a post-segment callback that fills the portfolio returns the same way as done by the fixed FS mode! In addition, we will store the total portfolio return per column to avoid reconstructing it during the post-analysis phase:

```
>>> @njit
... def post_segment_func_nb(c):
...     returns = c.in_outputs.returns  ❶
...     total_return = c.in_outputs.total_return
...     i = c.i
...     g = c.group
...     if c.cash_sharing:  ❷
...         returns[i, g] = c.last_return[g]
...         total_return[g] = c.last_value[g] / c.init_cash[g] - 1
...     else:  ❸
...         for col in range(c.from_col, c.to_col):  ❹
...             returns[i, col] = c.last_return[col]
...             total_return[col] = c.last_value[col] / c.init_cash[col] - 1

>>> pf = vbt.Portfolio.from_signals(
...     sub_data,
...     entries= pd.Series([X, 0, 0, 0, 0, 0, 0]),
...     exits= pd.Series([0, 0, 0, X, 0, 0, 0]),
...     short_entries= pd.Series([0, 0, 0, 0, X, 0, 0]),
...     short_exits= pd.Series([0, 0, 0, 0, 0, 0, X]),
...     post_segment_func_nb=post_segment_func_nb,  ❺
...     in_outputs=dict(  ❻
...         returns=vbt.RepEval(  ❼
...             "np.full((target_shape[0], len(cs_group_lens)), np.nan)"
...         ),
...         total_return=vbt.RepEval(
...             "np.full(len(cs_group_lens), np.nan)"
...         )
...     )
... )
>>> pd.testing.assert_frame_equal(  ❸
...     pf.get_in_output("returns"),
...     pf.get_returns()
... )
>>> pd.testing.assert_series_equal(
...     pf.get_in_output("total_return"),
...     pf.get_total_return()
... )
```

- ❶ **SignalSegmentContext.in\_outputs** contains exactly the in-output tuple that we passed
- ❷ If cash sharing is enabled, returns must be computed per group
- ❸ If cash sharing is disabled, returns must be computed per column
- ❹ Iterate over each column in the current group
- ❺ Overriding this callback is enough to switch to the flexible mode
- ❻ In-output tuple can be provided either as a named tuple or a dict. If the latter, it will be automatically transformed into a named tuple by the method.
- ❼ Use expression evaluation templates to build arrays only after the target shape is established. Use `len(cs_group_lens)` to get the number of groups when cash sharing is enabled, and the number of columns when cash sharing is disabled.
- ❽ Check the pre-computed and reconstructed arrays for equality

The most impressive in the approach above is that any pre-computed metric that can be found among the attributes of the `Portfolio` class is automatically picked by any built-in property and method that requires it, such as `Portfolio.stats`. The only limitation is the inability to change the grouping after the array has been created; when this happens, the reconstructed version of the metric will be returned instead.

## Summary

Backtesting shouldn't be complex! One of the most crucial factors in successful trading is a proper entry and exit timing. By parametrizing the timing and direction of our trades, we can theoretically achieve the worst and best-possible returns in any market, thus the representation of a trading strategy as a permutation of buy, sell, short-sell, and short-cover signals is almost "**Turing-complete**" in this regard. Reducing complex strategies into such a limited signal set has another advantage: we can enable and standardize backtesting of many trading strategies using the same piece of code. The method `Portfolio.from_signals` not only makes all of that possible, but it also encourages us to hack into the simulation to define our custom trading logic iteratively, or to change the default simulation behavior, making it the best of both vectorized and event-driven worlds. But not everything is appropriate to be represented using signals: strategies that heavily rely upon variations in order information, such as rebalancing strategies, are best implemented using other means. Gladly, vectorbt offers solutions to such problems as well 🙌

 [Python code](#)